Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет инженерно-экономический
Кафедра экономической информатики
Дисциплина «Распределенные информационные системы»

«К ЗАЩИТЕ ДОПУСТИТЬ»
Руководитель курсового проекта
Ассистент кафедры ЭИ
_____А.Д. Русецкий
___.____.2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:
«ПЛАТФОРМА ДЛЯ УПРАВЛЕНИЯ ЛИЧНЫМИ ФИНАНСАМИ С МОДУЛЕМ КАТЕГОРИЗАЦИИ РАСХОДОВ И ПЛАНИРОВАНИЯ БЮДЖЕТА»

БГУИР КП 6-05-0611-01 028 ПЗ

Выполнила студент группы 372302
Усова Виктория Александровна

_____
(подпись студента)
Курсовой проект представлен на проверку ___.____.2026

_____
(подпись студента)

Минск 2026

Бланк заданий

Бланк заданий

РЕФЕРАТ

БГУИР КР 6-05-0611-01 028 ПЗ

Усова В. А. Платформа для управления личными финансами с модулем категоризации расходов и планирования бюджета / В.А. Усова. – Минск: БГУИР, 2026. – 103 с.

Пояснительная записка 103 с., 55 рис., 5 табл., 23 источника, 4 приложения.

Ключевые слова: веб-приложение, личные финансы, управление бюджетом, категоризация расходов, финансовые цели, аутентификация, REST API, база данных, аналитика, совместные листы.

Цель курсового проекта: разработка программной платформы для управления личными финансами с модулем категоризации расходов и планирования бюджета, обеспечивающей автоматизацию учёта доходов и расходов, аналитику финансового состояния и поддержку достижения финансовых целей пользователя.

Методология проведения работы: при разработке использовались методы системного анализа, моделирование бизнес-процессов с применением нотаций BPMN и IDEF0, а также UML-диаграмм. Проектирование базы данных основано на реляционной модели с нормализацией до третьей нормальной формы. Серверная часть реализована на языке Java с использованием фреймворка Spring Boot, клиентская часть – на TypeScript с применением библиотеки React и сборщика Vite, обмен данными организован по протоколу HTTP в формате JSON, база данных – PostgreSQL. Для проверки корректности системы применялось модульное и интеграционное тестирование с использованием JUnit 5 и Mockito.

Результаты работы: выполнен анализ предметной области и изучены бизнес-процессы управления личными финансами, включая учёт операций, категоризацию расходов и бюджетное планирование. Построены диаграммы бизнес-процессов в нотациях BPMN и IDEF0. Спроектирована реляционная база данных. Созданы диаграммы классов, состояний, компонентов и последовательностей, описывающие структуру и поведение системы. Реализованы модули категоризации расходов, планирования бюджета, управления финансовыми целями и вкладами, аналитики, совместных финансовых листов, курсов валют и обучающих материалов по финансовой грамотности. Проведено функциональное, модульное и интеграционное тестирование, подтвердившее корректность реализованной бизнес-логики.

Область применения результатов: веб-приложение предназначено для конечных пользователей, заинтересованных в систематическом контроле личных финансов, планировании бюджета и достижении финансовых целей, а также для администраторов, осуществляющих управление системой и обучающими материалами по финансовой грамотности.

СОДЕРЖАНИЕ

Введение 6
1 Анализ литературных источников и программных решений 8
1.1 Описание и анализ предметной области 8
1.2 Обзор функциональности аналогов программного средства 9
2 Моделирование предметной области и разработка требований к программному средству 15
2.1 Анализ и формализация бизнес-процессов предметной области 15
2.2 Разработка спецификации требований к программному средству 27
2.3 Образ предлагаемого решения 31
3 Проектирование и разработка программного средства 34
3.1 Архитектурные решения и технологии реализации программного средства 34
3.2 Проектирование и разработка пользовательского интерфейса 39
3.3 Разработка модели данных 43
3.4 Модели представления программного средства на основе UML 47
3.5 Описание организации бизнес-логики программного средств 52
3.6 Спецификация к разработанному API и механизмы обеспечения информационной безопасности 56
4 Тестирование и программная документация к программному средству 59
4.1 Функциональное тестирование программного средства 59
4.2 Модульное и интеграционное тестирование бизнес-логики программного средства 61
4.3 Руководство по развертыванию программного средства 65
4.4 Руководство пользователя 67
Заключение 83
Список использованных источников 84
Приложение А (обязательное) Листинг кода алгоритмов, реализующих основную бизнес-логику 86
Приложение Б (обязательное) Листинг скрипта базы данных 92
Приложение В (не обязательное) BPMN-модель процессов предметной области 101
Приложение Г (не обязательное) Диаграмма классов 102
Ведомость документов курсового проекта 103

ВВЕДЕНИЕ

Современные условия цифровой экономики характеризуются активным развитием финансовых технологий и ростом интереса населения к вопросам управления личными финансами [1]. Увеличение числа банковских продуктов, электронных платёжных систем и сервисов онлайн-покупок приводит к усложнению структуры доходов и расходов граждан [2]. В результате всё больше людей сталкиваются с необходимостью систематического контроля финансовых потоков, анализа структуры затрат и планирования бюджета для достижения краткосрочных и долгосрочных целей. В этих условиях создание удобной и функциональной платформы для управления личными финансами становится особенно актуальным.

Традиционные способы ведения учёта – записи в блокноте или использование электронных таблиц – нередко оказываются недостаточно эффективными. Они требуют значительных временных затрат, подвержены ошибкам и не обеспечивают наглядной аналитики. Применение современных информационных технологий позволяет автоматизировать процессы фиксации доходов и расходов, классифицировать финансовые операции и формировать отчётность в удобной форме. Автоматизация в данном случае выступает не только средством упрощения расчётов, но и инструментом повышения финансовой грамотности пользователя.

Особое значение приобретает модуль категоризации расходов. Разделение затрат по категориям позволяет выявлять наиболее ресурсоёмкие направления расходов, анализировать динамику потребления и принимать обоснованные решения по оптимизации бюджета. Автоматическая или полуавтоматическая классификация операций снижает нагрузку на пользователя и повышает точность учёта данных.

Не менее важной задачей является планирование бюджета. Формирование лимитов по категориям расходов и контроль их соблюдения способствует более ответственному отношению к финансовым ресурсам. Наличие инструментов прогнозирования и визуализации помогает пользователю оценивать текущее финансовое состояние, отслеживать отклонения от запланированных показателей и своевременно корректировать стратегию управления средствами.

Создание единой цифровой платформы объединяет процессы учёта, анализа и планирования в рамках одной системы. Пользователь получает возможность регистрировать доходы и расходы, просматривать статистику за выбранные периоды, формировать отчёты и устанавливать финансовые цели. Для обеспечения стабильной работы и масштабируемости системы требуется комплексный подход, включающий анализ предметной области, проектирование архитектуры, реализацию серверной и клиентской частей, а также обеспечение безопасности хранения персональных данных.

Разработка платформы для управления личными финансами с модулем категоризации расходов и планирования бюджета предполагает комплексный

подход, включающий анализ предметной области, проектирование архитектуры, реализацию клиент-серверного взаимодействия и обеспечение защиты персональных данных. Поскольку система работает с финансовой информацией пользователей, особое внимание уделяется безопасности, разграничению прав доступа и сохранности данных [3].

Программное решение создаётся с использованием современных технологий распределённых информационных систем. Серверная часть реализуется на базе Spring Boot [4], что обеспечивает построение устойчивой и масштабируемой backend-архитектуры с разделением на уровни контроллеров, сервисов и доступа к данным. Клиентская часть разрабатывается с применением современных JavaScript-фреймворков [5], позволяющих создать удобный и интуитивно понятный интерфейс. Использование PostgreSQL обеспечивает надёжное хранение операций и эффективную обработку аналитических запросов [6].

Архитектура системы строится по принципу разделения ответственности между модулями учёта операций, категоризации расходов, планирования бюджета и формирования отчётности, что упрощает сопровождение и дальнейшее развитие функционала. Тестирование бизнес-логики гарантирует корректность расчётов, контроля лимитов и формирования статистики даже при одновременной работе нескольких пользователей. В результате создаётся надёжная и масштабируемая платформа, соответствующая современным требованиям к цифровым финансовым сервисам.

Цель работы заключается в разработке и реализации программной платформы, обеспечивающей автоматизацию учёта личных финансов, категоризацию расходов и поддержку планирования бюджета.

Для достижения поставленной цели необходимо решить следующие задачи:

– провести анализ существующих программных решений в области управления личными финансами;

– выполнить моделирование предметной области и сформировать требования к системе;

– разработать архитектуру и реализовать программное средство;

– провести тестирование разработанной системы и подготовить документацию.

Объектом исследования является процесс управления личными финансовыми ресурсами пользователя. Предметом исследования – автоматизация учёта доходов и расходов с использованием инструментов категоризации и бюджетного планирования.

Цитирования оформлены ссылками на источники, приведённые в «Списке использованных источников».

1 АНАЛИЗ ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ПРОГРАММНЫХ РЕШЕНИЙ

1.1 Описание и анализ предметной области

В условиях цифровой трансформации общества финансовое поведение населения претерпевает существенные изменения. Расширение спектра банковских услуг, развитие интернет-банкинга, мобильных приложений и систем бесконтактной оплаты приводит к увеличению скорости и частоты совершения финансовых операций. Денежные средства становятся более «виртуальными», что психологически снижает контроль над расходами и усложняет оценку реального финансового положения. В такой среде особую актуальность приобретает систематизированный подход к управлению личными денежными ресурсами.

Предметная область управления личными финансами охватывает процессы формирования доходов, осуществления расходов, распределения средств по направлениям потребления, накопления и инвестирования. В отличие от корпоративных финансов, где применяются строгие регламенты и стандарты бухгалтерского учёта, в сфере личных финансов поведение носит более гибкий и зачастую спонтанный характер. Это повышает риск нерационального использования средств и возникновения дефицита бюджета.

Базовыми элементами являются пользователь, финансовая операция, источник дохода, статья расхода, категория, бюджетный период и финансовая цель. Финансовая операция фиксирует факт движения денежных средств и характеризуется суммой, датой, типом (доход или расход) и дополнительным описанием. Категория служит инструментом группировки однородных расходов, формируя логическую структуру затрат. Бюджетный период определяет временные рамки анализа (месяц, квартал, год), в пределах которых осуществляется планирование и контроль.

Ключевым процессом является учёт финансовых операций. При отсутствии централизованной системы данные о расходах часто рассредоточены: часть хранится в банковских приложениях, часть – в электронных кошельках, а мелкие траты нередко остаются незафиксированными. Это приводит к фрагментарности информации и затрудняет формирование целостной картины. Ручное сведение данных из различных источников требует значительных временных затрат и сопровождается арифметическими ошибками.

Следующим значимым процессом выступает категоризация расходов. Без структурирования финансовых операций анализ теряет информативность, поскольку совокупная сумма затрат не отражает их экономического содержания. Разделение расходов по категориям позволяет определить приоритетные направления потребления, выявить дисбаланс между обязательными и необязательными тратами, а также оценить потенциал для

оптимизации бюджета. Однако при ручном присвоении категорий возможны неточности и несогласованность классификации.

Процесс бюджетного планирования предполагает установление лимитов по выбранным категориям и контроль их соблюдения. Пользователь определяет допустимый объём расходов на определённый период, после чего сопоставляет фактические значения с запланированными. Отсутствие автоматизированного механизма контроля приводит к запаздыванию реакции на перерасход. В большинстве случаев превышение бюджета становится очевидным лишь по окончании периода, когда возможности корректировки уже ограничены.

Дополнительным элементом предметной области является формирование финансовых целей. Накопление на крупные покупки или создание резервного фонда требует систематического распределения средств и отслеживания прогресса. Без инструментов визуализации и расчёта сроков достижения цели мотивация пользователя может снижаться, что негативно влияет на дисциплину накоплений.

Современные программные продукты в сфере личных финансов различаются по уровню функциональности и сложности. Часть решений ориентирована на базовый учёт операций, другие предлагают расширенные аналитические инструменты и интеграцию с банковскими сервисами. Однако многие из них либо перегружены дополнительными функциями, не востребованными широкой аудиторией, либо не обеспечивают достаточной гибкости в настройке категорий и бюджетных параметров.

С точки зрения информационных технологий данная предметная область характеризуется необходимостью обработки персональных данных, хранения истории операций и выполнения регулярных вычислительных операций по агрегированию статистики. Это требует построения надёжной архитектуры с обеспечением целостности данных, поддержкой многопользовательского режима и защитой от несанкционированного доступа.

Таким образом, управление личными финансами представляет собой сложную совокупность взаимосвязанных процессов, включающих учёт операций, их классификацию, анализ и планирование. Недостаточная автоматизация данных процессов приводит к ошибкам, потере контроля и снижению эффективности распределения средств. Создание специализированной платформы позволяет объединить перечисленные элементы в едином цифровом пространстве, обеспечив системность, прозрачность и удобство финансового управления.

1.2 Обзор функциональности аналогов программного средства

Анализ существующих программных решений в сфере управления личными финансами является важным этапом разработки платформы с модулем категоризации расходов и планирования бюджета. Современный

рынок мобильных и веб-приложений предлагает широкий спектр инструментов для учёта доходов и расходов, различающихся по функциональности, уровню автоматизации, модели монетизации и целевой аудитории.

Исследование аналогов позволяет определить наиболее востребованные функции, выявить типовые архитектурные решения, а также установить ограничения и недостатки существующих продуктов. Особое внимание уделяется удобству пользовательского интерфейса, гибкости настройки категорий и бюджетов, возможностям аналитики, а также уровню автоматизации обработки финансовых операций.

В рамках анализа были рассмотрены наиболее популярные решения, представленные на международном и локальном рынках: CoinKeeper, Zenmoney, Money Manager и YNAB (You Need A Budget).

CoinKeeper представляет собой мобильное приложение для учёта личных финансов, ориентированное на визуальное управление бюджетом [7]. Основной особенностью системы является графический интерфейс, в котором операции добавляются посредством перетаскивания денежных потоков между «кошельками» и категориями (рисунок 1.1).

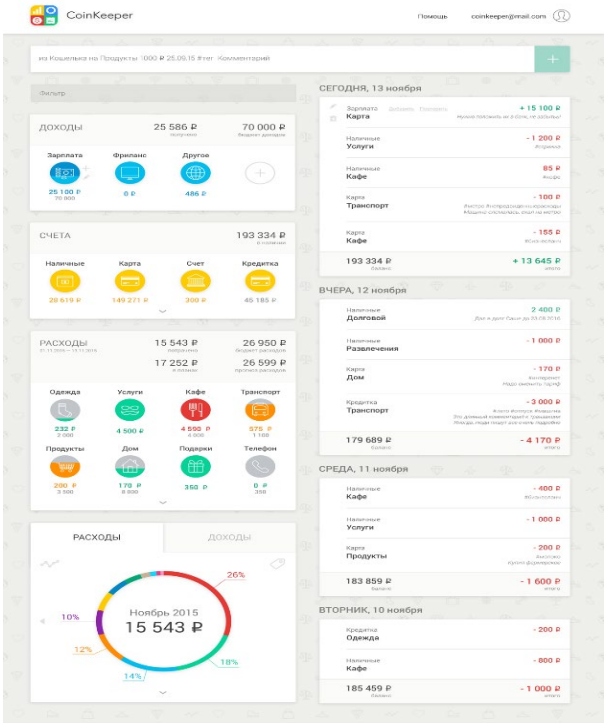


Рисунок 1.1 – Система CoinKeeper

В качестве функциональных возможностей данной системы можно выделить учёт доходов и расходов с ручным вводом операций, настройка категорий расходов и доходов, формирование бюджетов по категориям на разные периоды (неделя, месяц), наглядные диаграммы и графики распределения

средств, синхронизация данных между устройствами, возможность делиться доступом с другими пользователями (семья, партнёры).

Можно выделить следующие преимущества:

– удобный и визуально ориентированный интерфейс, минимальный порог вхождения для пользователей;

– интерактивное управление бюджетом с визуальными элементами.

– поддержка нескольких валют и кошельков;

– быстрый ввод операций и мгновенная аналитика.

В качестве недостатков выделяют:

– автоматическая классификация операций отсутствует – требуется ручной ввод и корректировка;

– ограниченные инструменты аналитики – нет детальных отчётов с прогнозами;

– функциональность бюджетного планирования недостаточно гибкая – нет продвинутых сценариев, шаблонов или рекомендаций;

– основные расширенные функции доступны только по подписке.

CoinKeeper хорошо подходит как базовый инструмент учёта, но не обеспечивает глубокого планирования бюджета и автоматического анализа расходов, что ограничивает его применение для серьёзной финансовой аналитики.

Zenmoney – система, ориентированная на автоматизацию финансового учёта с возможностью интеграции с банковскими счетами и картами (рисунок 1.2). Приложение автоматически получает данные о транзакциях, классифицирует операции и предлагает аналитику по бюджетам [8].

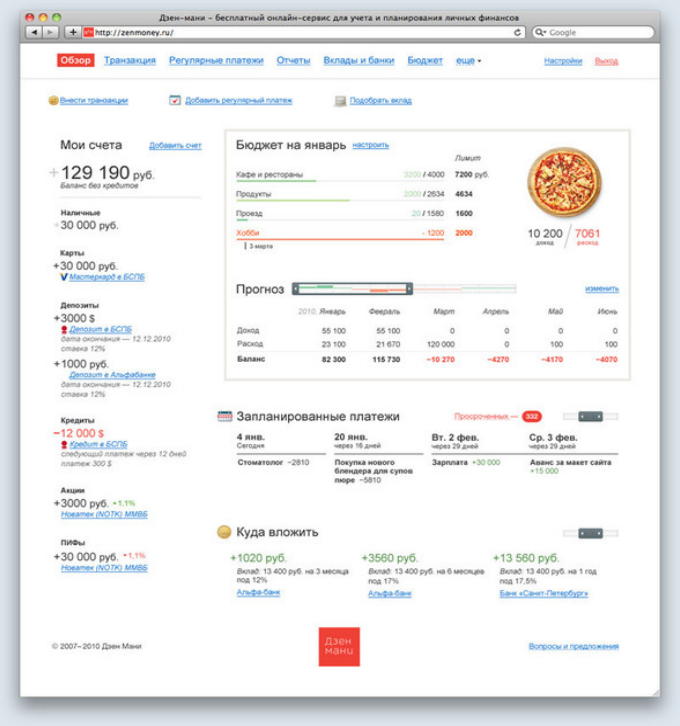


Рисунок 1.2 – Платформа Zenmoney

В качестве функциональных возможностей данной платформы можно выделить интеграцию с банковскими счетами и платёжными картами (автоматический импорт операций), автоматическая категоризация расходов с возможностью корректировок, формирование бюджетов и контроль их исполнения, детальные отчёты по периодам и категориям, прогнозирование остатка бюджета и анализ тенденций, поддержка нескольких счетов, валют и долговых обязательств.

Преимущества данной платформы:

– высокий уровень автоматизации – минимальный ручной ввод данных;

– возможность отслеживания долгов и кредитов;

– расширенная аналитика с отчётами, диаграммами и трендами;

– удобный интерфейс с мобильной и веб-версиями.

Недостатки:

– зависимость от интеграции банков, которая может быть нестабильной или поддерживаться не для всех банков;

– интерфейс и настройки могут быть сложны для пользователей, не привыкших к автоматической системе;

– бесплатная версия ограничена по функциональности – полноценный доступ возможен по подписке;

– нельзя гибко настраивать алгоритмы категоризации под индивидуальные сценарии.

Таким образом, Zenmoney предлагает продвинутую автоматизацию, но ограничен зависимостью от банковской синхронизации и не полностью закрывает задачи пользовательской настройки категорий и бюджетов.

Money Manager – простое приложение для ручного ввода финансовых операций и контроля основных финансовых потоков (рисунок 1.3). Часто используется как основное решение для пользователей, желающих фиксировать операции без сложной аналитики [9].

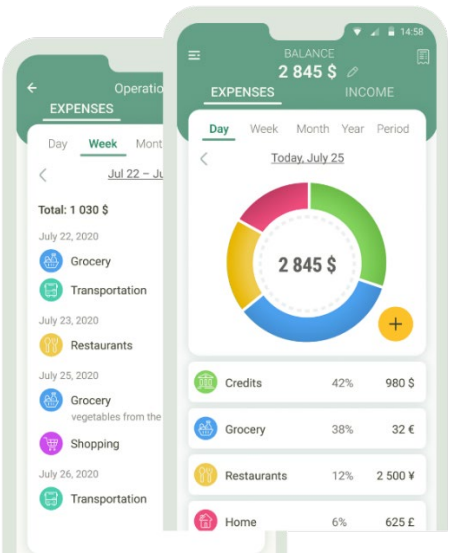


Рисунок 1.3 – Приложение Money Manager

Можно выделить следующие функциональные возможности: ввод доходов и расходов вручную, разовое распределение по категориям, просмотр статистики за выбранные периоды, отчёты в виде таблиц и диаграмм, экспорт и импорт данных в стандартных форматах (CSV, Excel).

Выделяют следующие преимущества:

– интуитивно понятный интерфейс;

– подходит для офлайн-использования без привязки к банковским данным;

– возможность самостоятельного управления категориями;

– низкий порог освоения – подходит для новичков.

И следующие недостатки:

– отсутствуют продвинутые инструменты планирования бюджета;

– нельзя автоматизировать импорт банковских операций;

– нет прогнозирования финансовых показателей;

– ограниченный объём аналитики – только базовые отчёты.

Money Manager удобен для базового учёта, но не предоставляет механизмов для углублённого анализа или автоматической проверки исполнения бюджета.

YNAB – зарубежная система, построенная на методологии «каждой единице денег назначена цель». Система ориентирована на активное бюджетирование, повышение финансовой дисциплины и планирование расходов заранее [10].

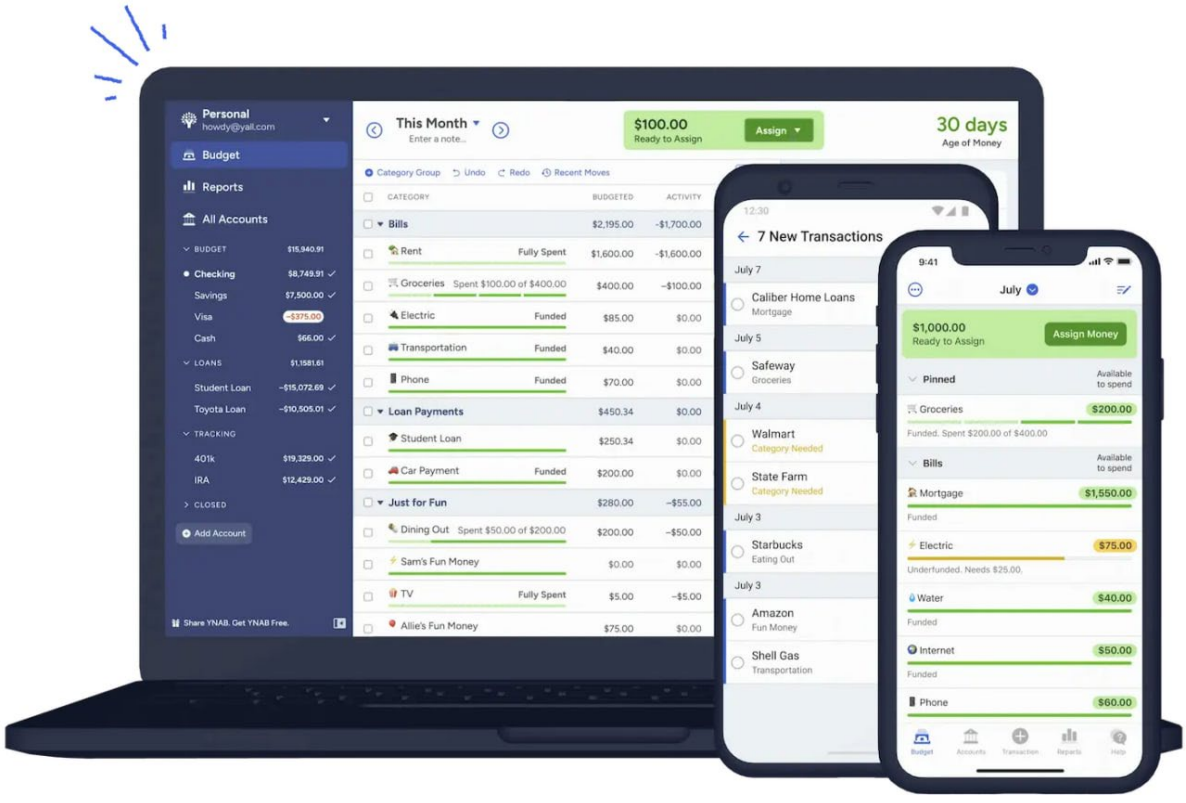


Рисунок 1.4 – Система YNAB

Функциональными возможностями являются: распределение всех доходов по категориям ещё до расходования, поддержка правил бюджетирования (например, «каждый доллар должен работать»), аналитика исполнения бюджета по периодам, прогнозирование финансового состояния на основе исторических данных, синхронизация между устройствами, веб- и мобильный доступ.

Выделяют такие преимущества:

– методология, способствующая рациональному распределению средств;

– многоуровневое планирование бюджета;

– поддержка обучающих материалов и занятий.

Недостатками являются:

– высокая стоимость подписки по сравнению с аналогами;

– англоязычный интерфейс и специфическая терминология;

– сложность освоения без предварительной подготовки;

– ограниченные возможности интеграции с банковскими счетами в некоторых регионах.

YNAB подходит для пользователей, стремящихся к строгому финансовому планированию, но его высокая стоимость и сложность делают его менее доступным для широкой аудитории.

Анализ аналогов показывает, что современные решения частично покрывают задачи управления личными финансами, но ни одно из них не обеспечивает полноценной гибкой автоматической категоризации операций, адаптируемой под пользователя; развитой системы планирования бюджета с предупреждениями и прогнозированием; баланса между простотой интерфейса и глубокой аналитикой; автоматической проверки логики исполнения бюджета при изменении данных несколькими пользователями.

Существующие приложения, которые либо слишком упрощены, либо ориентированы на узкий круг задач, либо требуют высокой стоимости подписки и сложны в использовании. Это подтверждает актуальность разработки собственной платформы, сочетающей удобство, гибкость, автоматизацию и продвинутую аналитику.

2 МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОГРАММНОМУ СРЕДСТВУ

2.1 Анализ и формализация бизнес-процессов предметной области

Для анализа и формализации бизнес-процессов разрабатываемой платформы управления личными финансами с модулем категоризации расходов и планирования бюджета используется нотация BPMN 2.0, которая позволяет наглядно представить последовательность действий, роли участников и информационные потоки между ними. Разработанная BPMN-диаграмма представляет собой формализованное описание взаимодействия пользователя с системой при управлении финансовыми операциями, начиная от ввода данных о расходах или доходах и заканчивая формированием аналитических отчетов и уведомлений о состоянии бюджета [11].

BPMN-диаграмма данного бизнес-процесса изображена на рисунке В.1, приложения В.

Процесс начинается с инициирования пользователем операции, которая фиксируется событием «Добавление операции». Пользователь вводит данные операции через задачу User Task «Ввести данные операции», после чего данные отправляются в систему (User Task / Send Task «Отправить данные»), что фиксируется Intermediate Throw Message Event.

В пуле «Платформа управления финансами», lane Backend, система получает сообщение от пользователя (Intermediate Catch Message Event) и выполняет проверку корректности данных (Service Task «Проверить корректность данных»). В зависимости от результата проверки используется Exclusive Gateway «Данные корректны?»

В случае некорректных данных формируется сообщение об ошибке (Service Task «Сформировать сообщение об ошибке») и отправляется пользователю (Intermediate Throw Message Event), после чего пользователь исправляет данные (User Task «Исправить данные») и повторно отправляет их в систему. В случае корректных данных операция сохраняется (Service Task «Сохранить операцию»).

Далее система проверяет тип операции через Gateway «Тип операции = расход?». Если операция является доходом, система обновляет баланс (Service Task «Обновить баланс»). Если операция является расходом, процесс передается lane Сервис бюджета, где проверяется лимит бюджета (Service Task «Проверить лимит бюджета»). В зависимости от результата проверки (Gateway «Лимит превышен?») формируется предупреждение о превышении лимита (Service Task «Сформировать предупреждение») и затем обновляется баланс (Service Task «Обновить баланс»).

После обновления баланса управление передается lane Аналитика, где выполняются задачи по пересчету показателей (Service Task «Пересчитать показатели») и обновлению отчетов (Service Task «Обновить отчёты»). По

завершении анализа формируется уведомление пользователю (Intermediate Throw Message Event), которое возвращается в пул «Пользователь».

В пуле «Пользователь» пользователь получает уведомление (Intermediate Catch Message Event «Получить уведомление») и просматривает отчет (User Task «Просмотреть отчёт»), после чего процесс завершается (End Event «Процесс завершён»).

Разработанная BPMN-диаграмма демонстрирует чёткое разделение ролей между пользователем и системой, автоматизацию проверки корректности данных, обновления баланса и аналитической обработки, обработку ошибок и уведомлений, что обеспечивает устойчивость бизнес-процесса к некорректным данным, интеграцию различных модулей платформы (backend, сервис бюджета, аналитика) с визуальным отображением потоков данных и сообщений.

Таким образом, формализованная BPMN-модель является основой для проектирования архитектуры программного продукта и служит руководством для реализации всех ключевых бизнес-функций платформы, обеспечивая наглядное отображение логики взаимодействия пользователя с системой на каждом этапе финансовой операции.

Анализ и формализация бизнес-процессов предметной области в нотации IDEF0. Методология IDEF0 – это метод функционального моделирования и графическая нотация, предназначенная для построения функциональной модели предметной области [12]. Применение IDEF0 позволяет наглядно представить взаимодействие между основными процессами системы управления личными финансами, включая модуль категоризации расходов и планирования бюджета. Методология помогает определить входные и выходные данные, управляющие воздействия и механизмы для каждого процесса, а также выявить ключевые направления для автоматизации финансовых операций. Использование этой нотации способствует повышению эффективности проектирования платформы, улучшает качество анализа финансовых процессов пользователя и снижает риски ошибок при реализации системы.

На рисунке 2.1 представлена контекстная диаграмма процесса «Управлять личными финансами пользователя». Входными потоками данных для данного процесса являются данные о доходах, расходах и транзакциях. Управляющими воздействиями выступают «Правила категоризации», «Ограничения бюджета», «Финансовые цели пользователя», «Настройки пользователя». Механизмами, обеспечивающими выполнение процесса, являются пользователь, приложение, алгоритмы анализа и база данных. Выходными данными являются «Финансовые отчеты», «Категоризированные расходы», «Бюджетный план», «Аналитика и рекомендации».

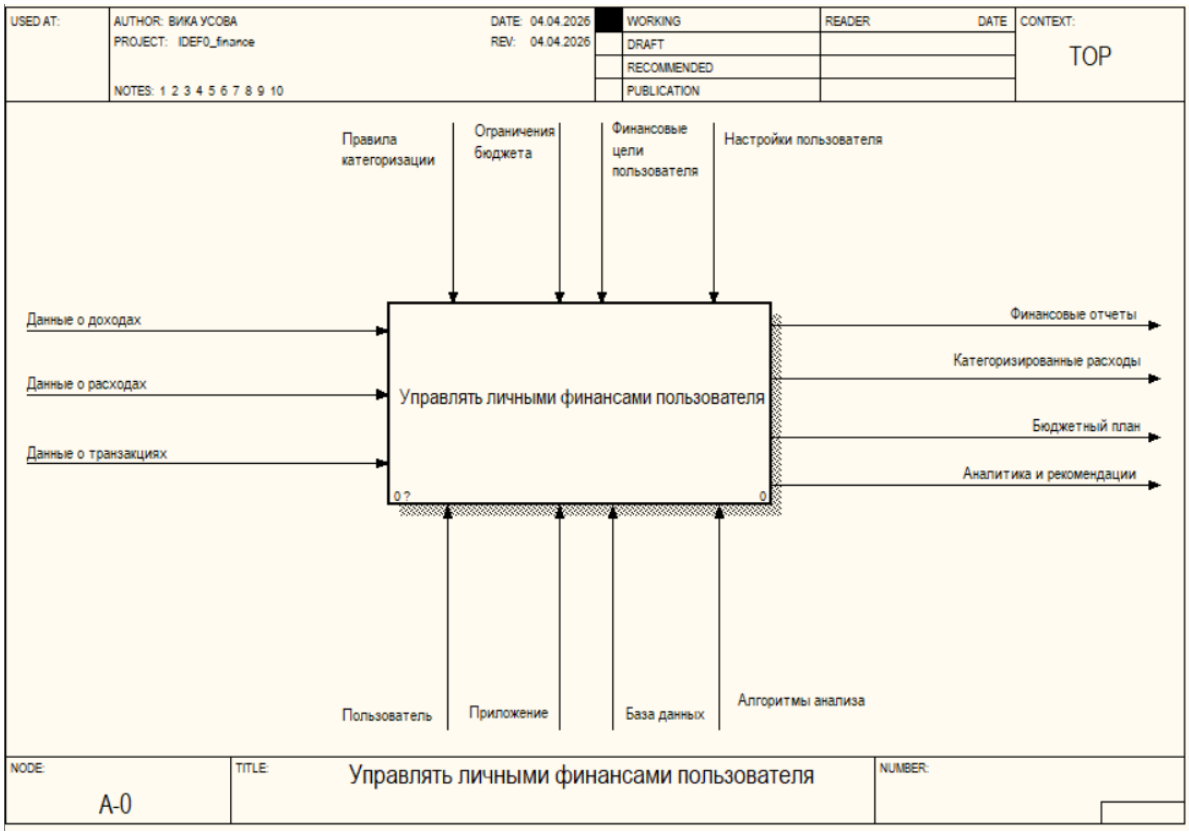


Рисунок 2.1 – Контекстная диаграмма процесса «Управлять личными финансами пользователя»

На рисунке 2.2 представлена декомпозиция процесса «Управлять личными финансами пользователя». Данный процесс включает пять подпроцессов. Вводить финансовые данные – осуществлять добавление информации о доходах, расходах и транзакциях, фиксировать её в системе и передавать для дальнейшей обработки. Категоризировать расходы – на основе правил категоризации распределять расходы по категориям, проверять корректность и готовить данные для анализа. Анализировать финансовые данные – формировать аналитические отчёты и рекомендации для пользователя, выявлять тенденции расходов и доходов. Планировать бюджет – на основе аналитики и финансовых целей пользователя составлять бюджетный план, учитывать ограничения и рекомендации системы. Формировать отчёты – собирать категоризированные расходы, бюджетный план и аналитику в готовые финансовые отчёты для пользователя.

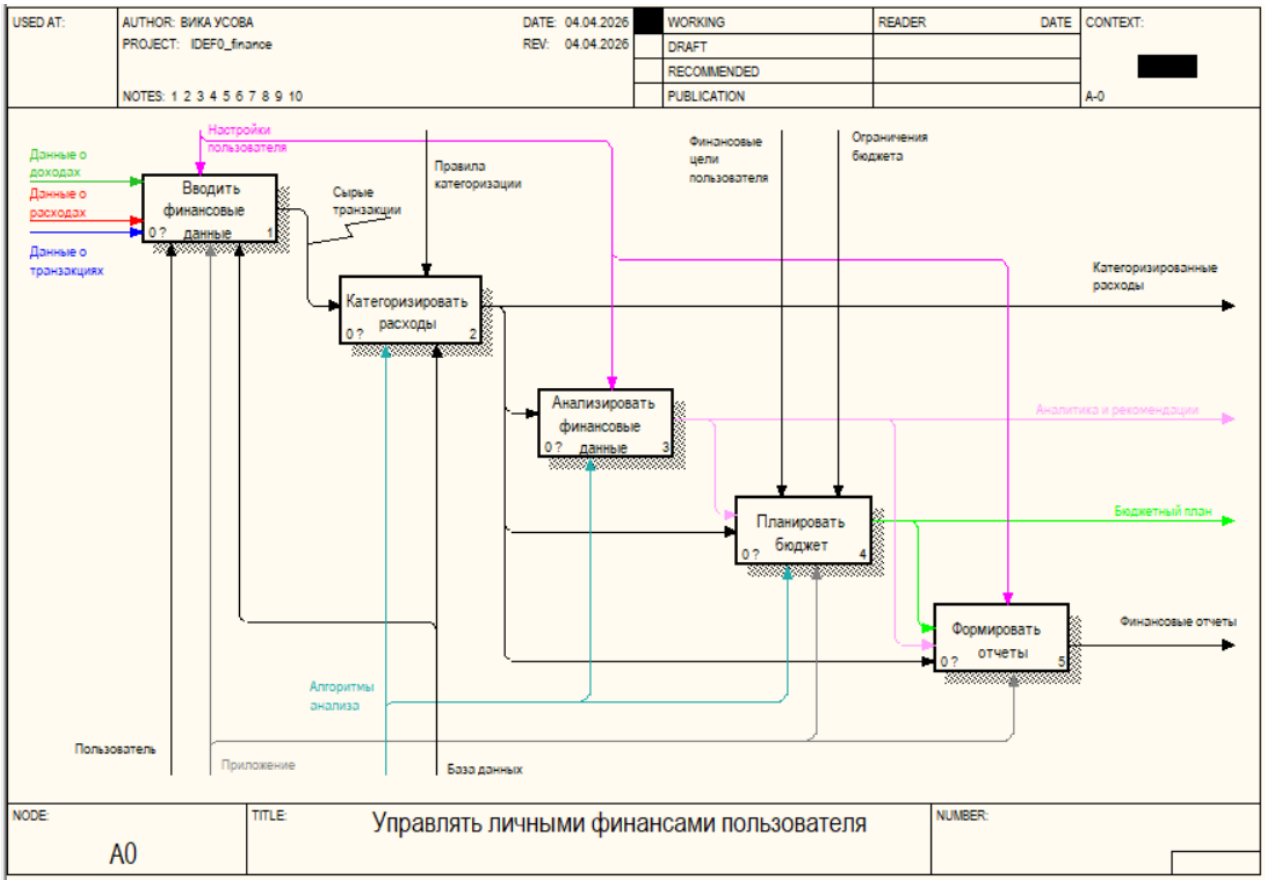


Рисунок 2.2 – Декомпозиция процесса «Управлять личными финансами пользователя»

Декомпозиция процесса «Вводить финансовые данные» представлена на рисунке 2.3 и включает 3 этапа. Получать финансовые данные – осуществить сбор сведений о доходах, расходах и банковских транзакциях из внешних и внутренних источников. Проверять финансовые данные – провести верификацию введённой информации на соответствие заданным настройкам пользователя для обеспечения корректности учёта. Сохранять финансовые данные – зафиксировать проверенные записи в базе данных системы для их дальнейшей обработки и формирования массива сырых транзакций.

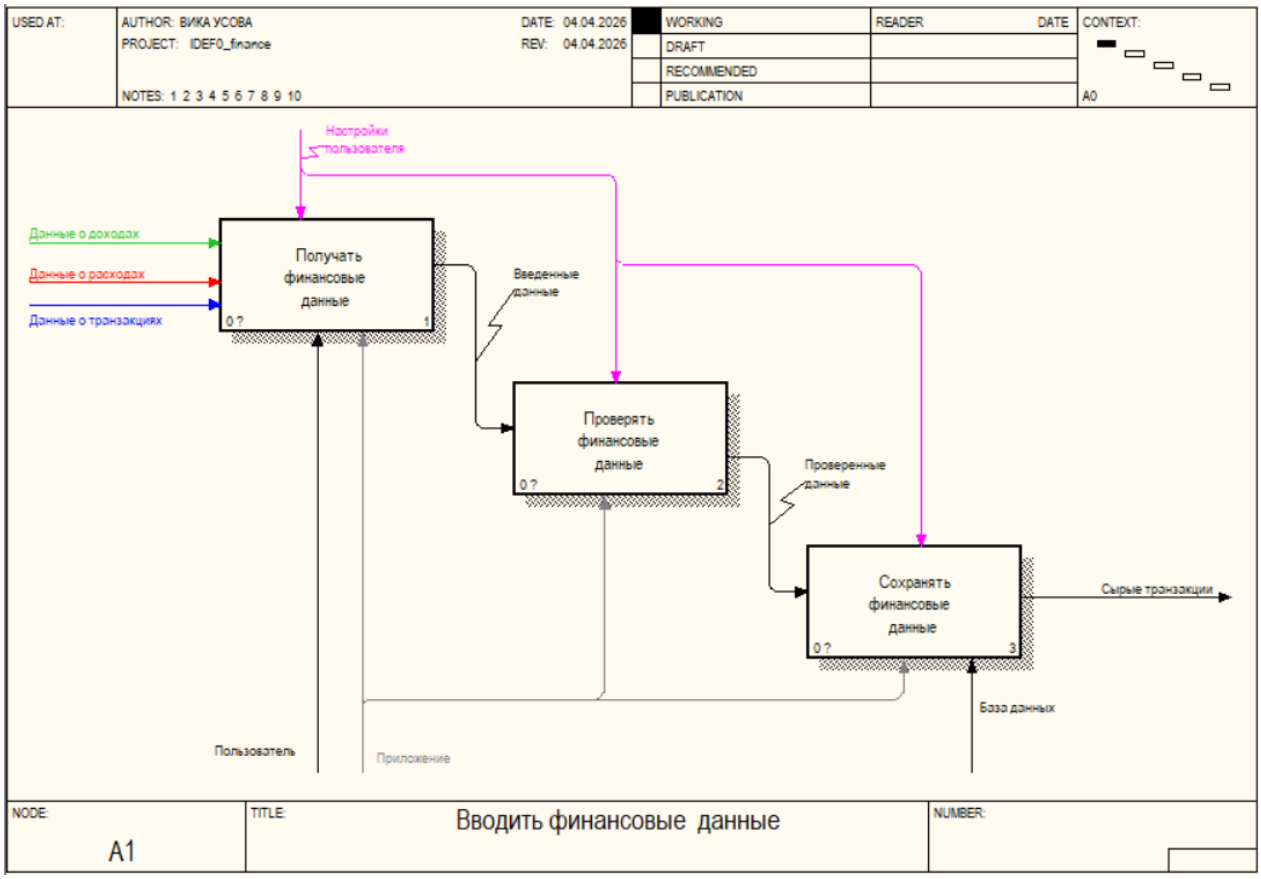


Рисунок 2.3 – Декомпозиция процесса «Вводить финансовые данные»

Декомпозиция третьего уровня процесса «Получать финансовые данные» представлена на рисунке 2.4 и включает также 3 этапа. Получить данные о доходах – осуществить сбор и первичную обработку информации о поступающих финансовых средствах (зарплата, переводы и др.) на основе введённых пользователем параметров. Получить данные о расходах – зафиксировать сведения о затратах, сопоставляя их с накопленной информацией о доходах для корректного отображения текущего финансового состояния. Получить данные о транзакциях – агрегировать все типы финансовых операций (доходы и расходы) в единый стандартизированный поток «Введенные данные» для их передачи на этап валидации.

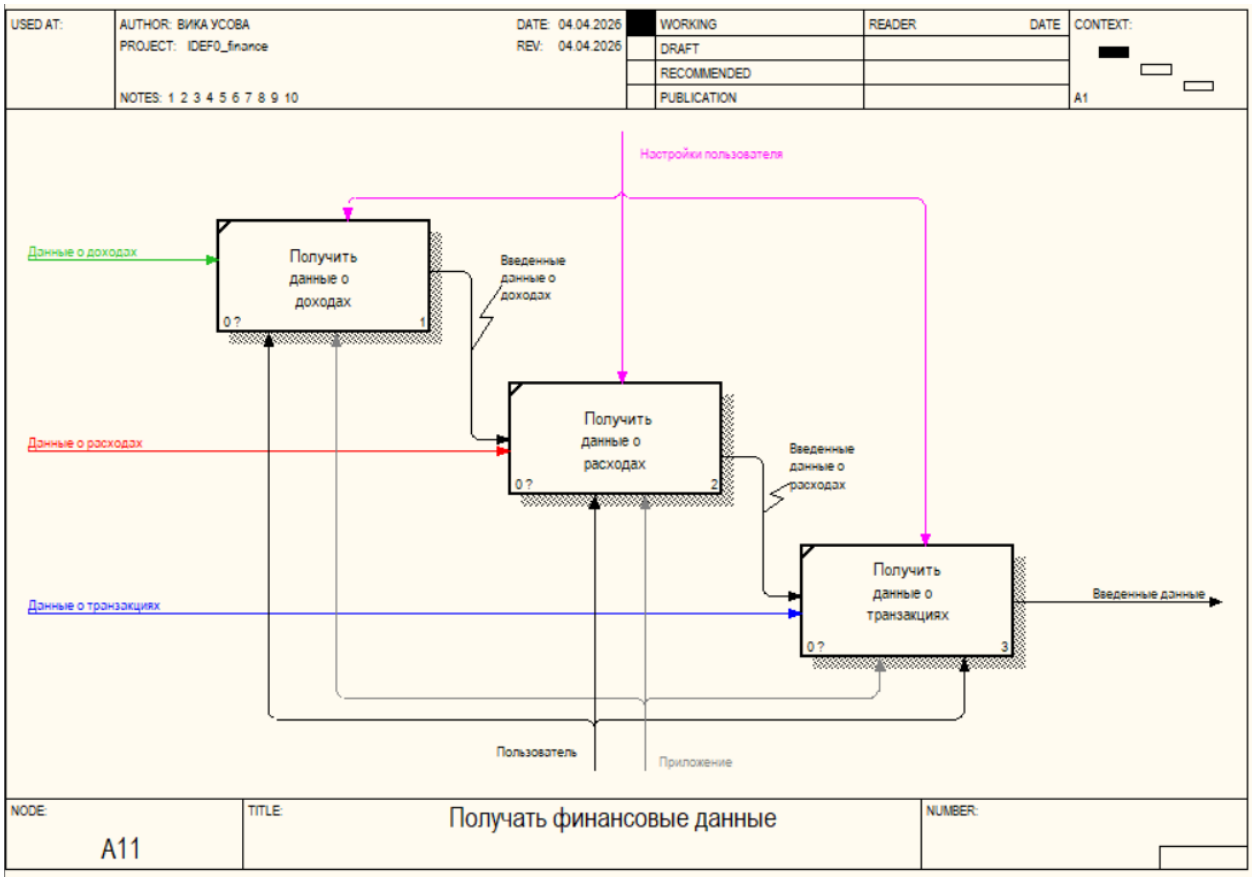


Рисунок 2.4 – Декомпозиция процесса «Получать финансовые данные»

Декомпозиция третьего уровня процесса «Проверять финансовые данные» представлена на рисунке 2.5 и включает следующие этапы. Проверить формат данных – выполнить технический контроль входящей информации (соответствие типов полей, масок ввода и корректность синтаксиса) для получения данных с корректным форматом. Проверить полноту данных – убедиться в наличии всех обязательных атрибутов транзакции (дата, сумма, источник), формируя на выходе массив полных данных для дальнейшего анализа. Проверить корректность значений – осуществить финальную сверку данных на предмет логических ошибок и соответствия лимитам, установленным в настройках пользователя, для формирования итогового потока проверенных данных.

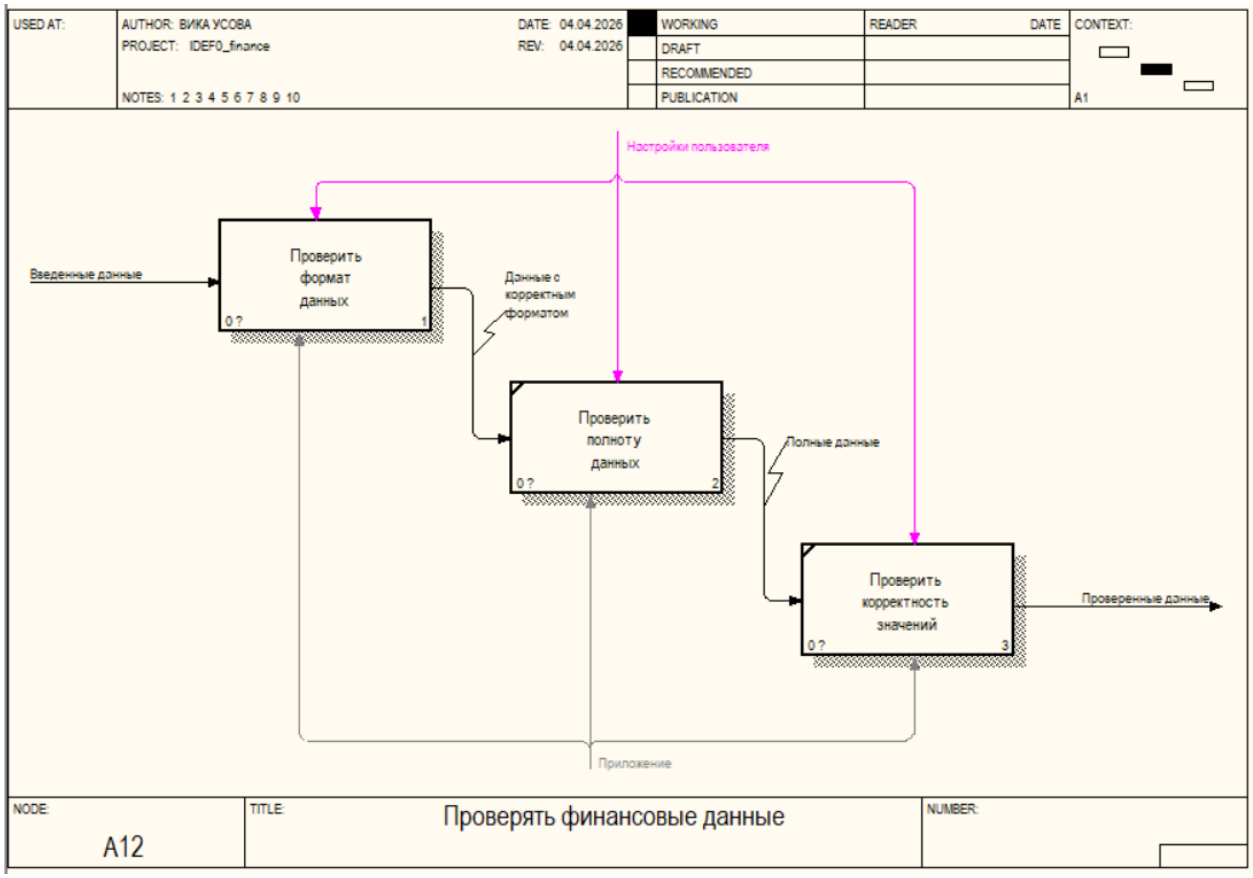


Рисунок 2.5 – Декомпозиция процесса «Проверять финансовые данные»

На рисунке 2.6 представлена декомпозиция четвертого уровня процесса «Проверить формат данных», она включает четыре этапа. Проверить тип данных – выполнить контроль соответствия входных значений ожидаемым типам (например, числовое значение для суммы, формат даты для времени операции), формируя на выходе данные с корректным типом. Проверить структуру данных – верифицировать иерархию и вложенность полученных сведений на соответствие внутренним протоколам системы для получения структурированных данных. Преобразовать формат данных – привести проверенную и структурированную информацию к единому системному стандарту (например, перевод всех валют в базовую или приведение даты к формату ISO) для вывода данных с корректным форматом.

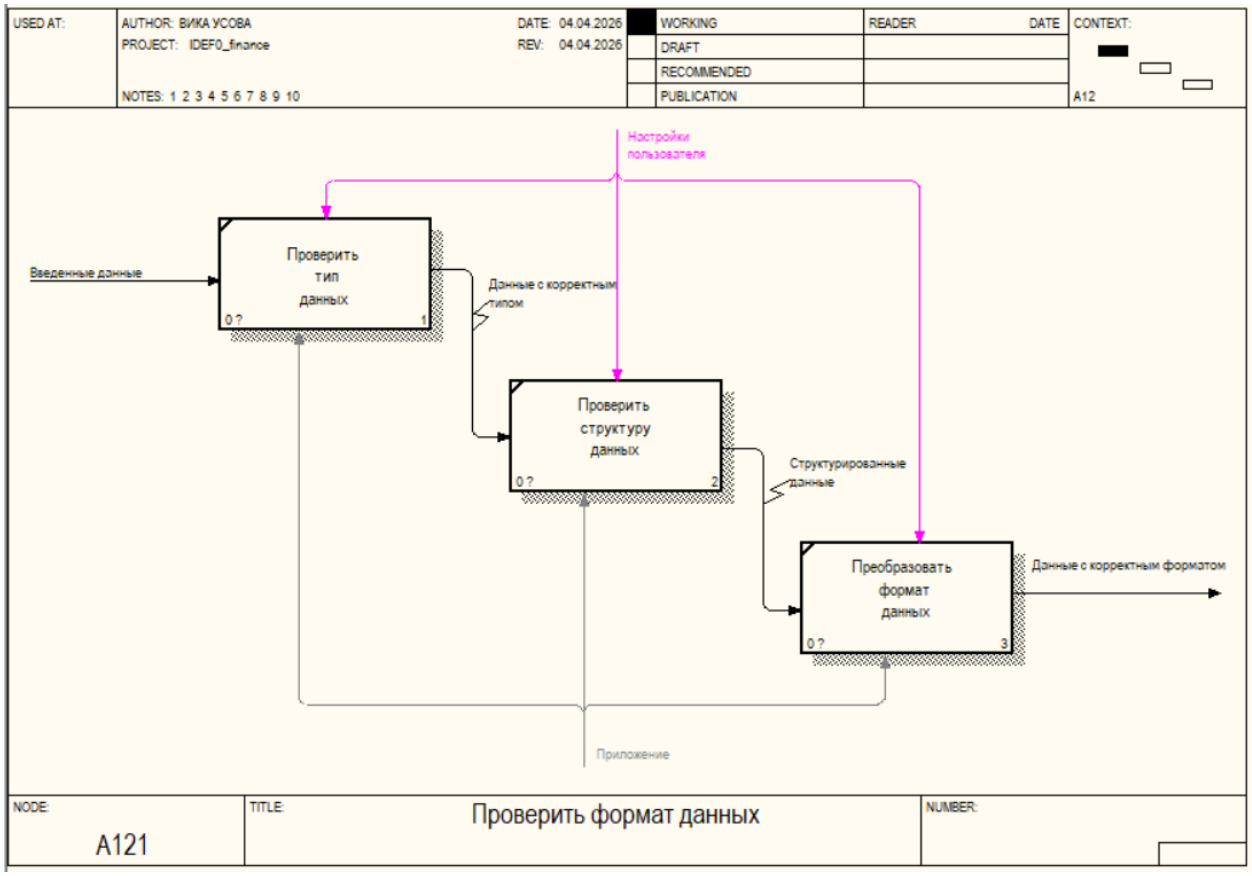


Рисунок 2.6 – Декомпозиция процесса «Проверять формат данных»

Декомпозиция третьего уровня процесса «Сохранять финансовые данные» представлена на рисунке 2.7. Подготовить данные к сохранению – выполнить финальную агрегацию проверенной информации и её упаковку в структуру, соответствующую схеме хранения, для формирования потока подготовленных данных. Записать данные в систему – осуществить физическую транзакцию записи подготовленных сведений в хранилище, обеспечивая целостность информации и формируя на выходе массив сырых транзакций для дальнейшего использования.

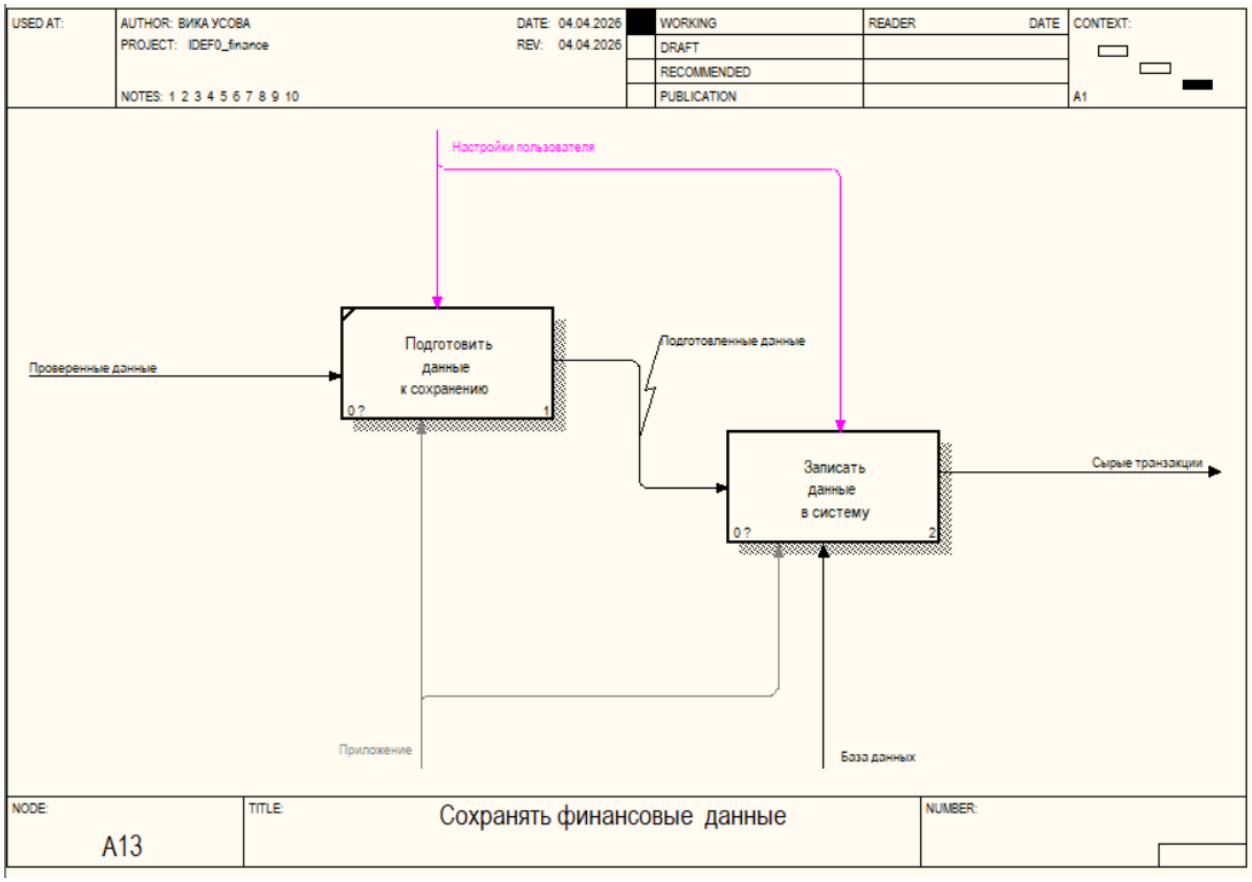


Рисунок 2.7 – Декомпозиция процесса «Сохранять финансовые данные»

Декомпозиция четвертого уровня процесса «Записать данные в систему» представлена на рисунке 2.8. Формировать запрос на запись – преобразовать подготовленные финансовые данные в программный запрос (например, SQL-команду), учитывая текущие настройки пользователя и параметры сессии. Выполнить запись в базу данных – осуществить непосредственное обращение к хранилищу через механизмы базы данных для физического размещения информации. Подтвердить сохранение данных – провести верификацию успешного завершения транзакции и обновить статус записей, формируя итоговый выход в виде массива сырых транзакций.

Блок «Подтвердить сохранение данных» важен для предотвращения потери данных в случае сбоя соединения с базой – система не выдаст «Сырые транзакции», пока не получит положительный ответ от СУБД.

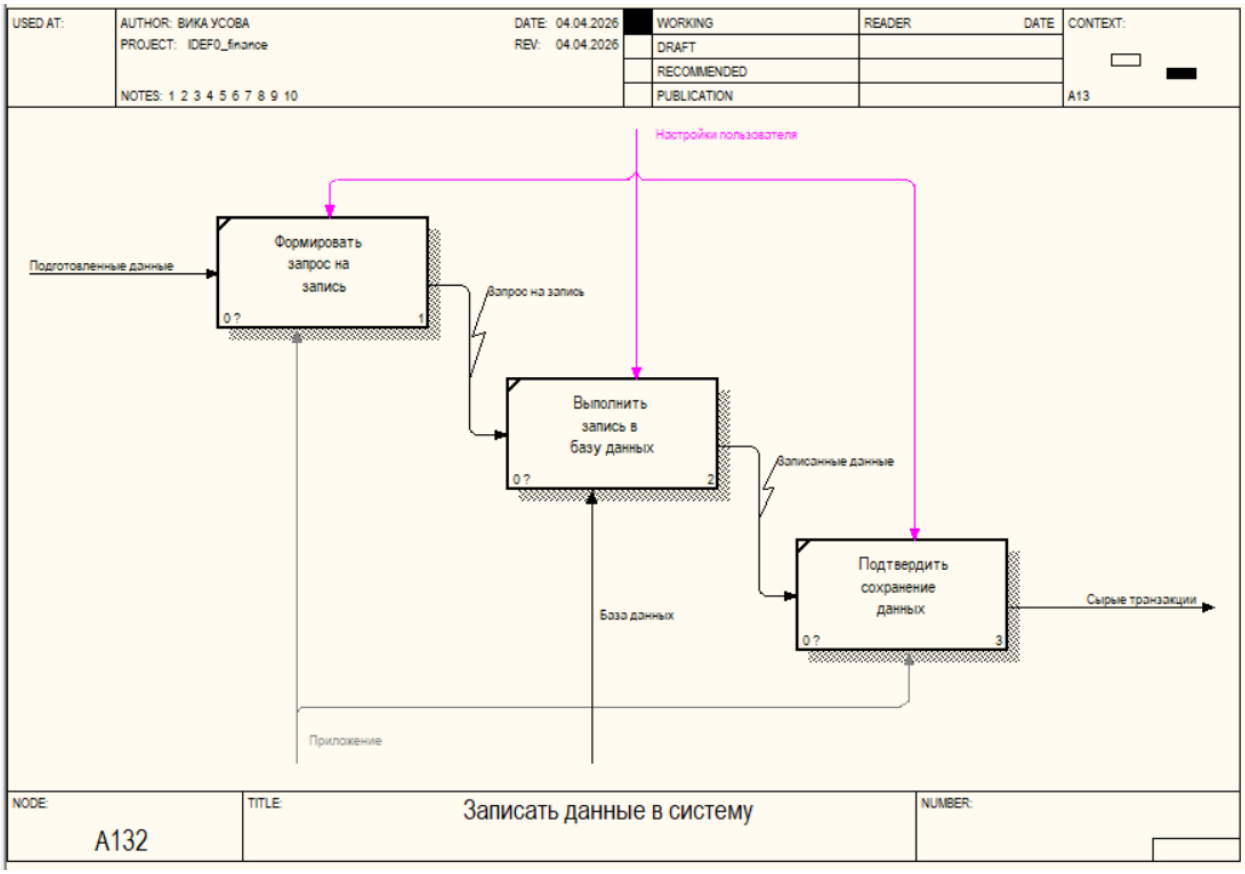


Рисунок 2.8 – Декомпозиция процесса «Записать данные в систему»

Декомпозиция процесса «Анализировать финансовые данные» представлена на рисунке 2.9. Группировать финансовые данные – выполнить агрегацию категоризированных расходов по временным периодам, типам транзакций или иным критериям, формируя массив сгруппированных данных. Рассчитывать финансовые показатели – произвести вычисление ключевых метрик (общий баланс, средний чек, процент выполнения бюджета, динамика накоплений), основываясь на сгруппированной информации. Формировать рекомендации – сопоставить рассчитанные показатели с финансовыми целями и лимитами пользователя для создания персонализированной аналитики и советов по оптимизации бюджета.

Этап «Группировать» необходим для того, чтобы аналитика была многомерной – пользователь может видеть свои траты не просто списком, а в разрезе недель, месяцев или конкретных категорий товаров.

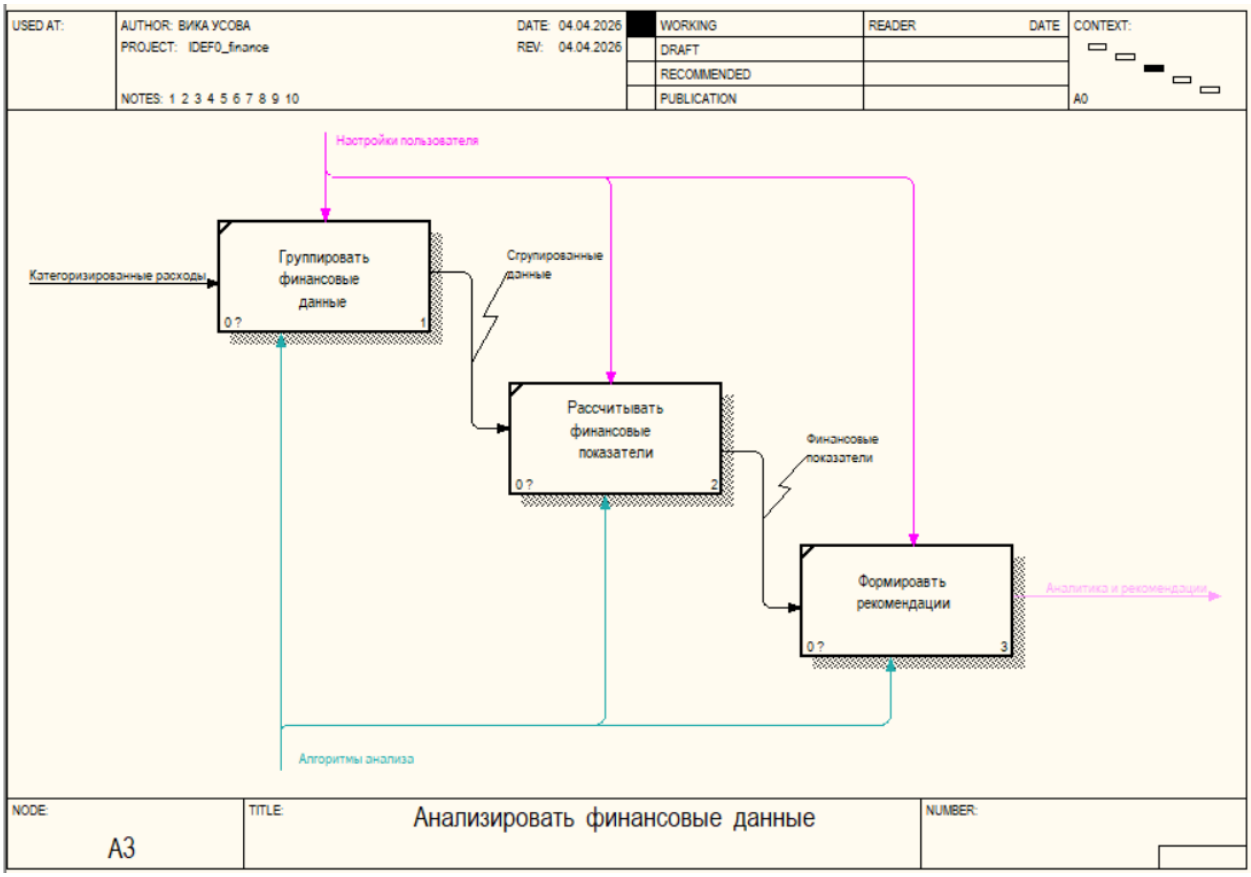


Рисунок 2.9 – Декомпозиция процесса «Анализировать финансовые данные»

На рисунке 2.10 представлена декомпозиция процесса «Планировать бюджет». Определять параметры бюджета – установить основные критерии планирования, основываясь на данных о прошлых расходах, текущих финансовых целях и рекомендациях аналитического модуля, формируя набор параметров бюджета. Рассчитывать бюджет – выполнить математическое распределение денежных средств по категориям на предстоящий период, используя заданные параметры и алгоритмы анализа для получения распределенного бюджета. Формировать бюджетный план – консолидировать расчетные показатели в итоговый документ с учетом установленных ограничений, формируя на выходе структурированный бюджетный план для пользователя.

Этап «Определять параметры» позволяет пользователю гибко настраивать план (например, выделить больше средств на «Образование» за счет сокращения категории «Развлечения»).

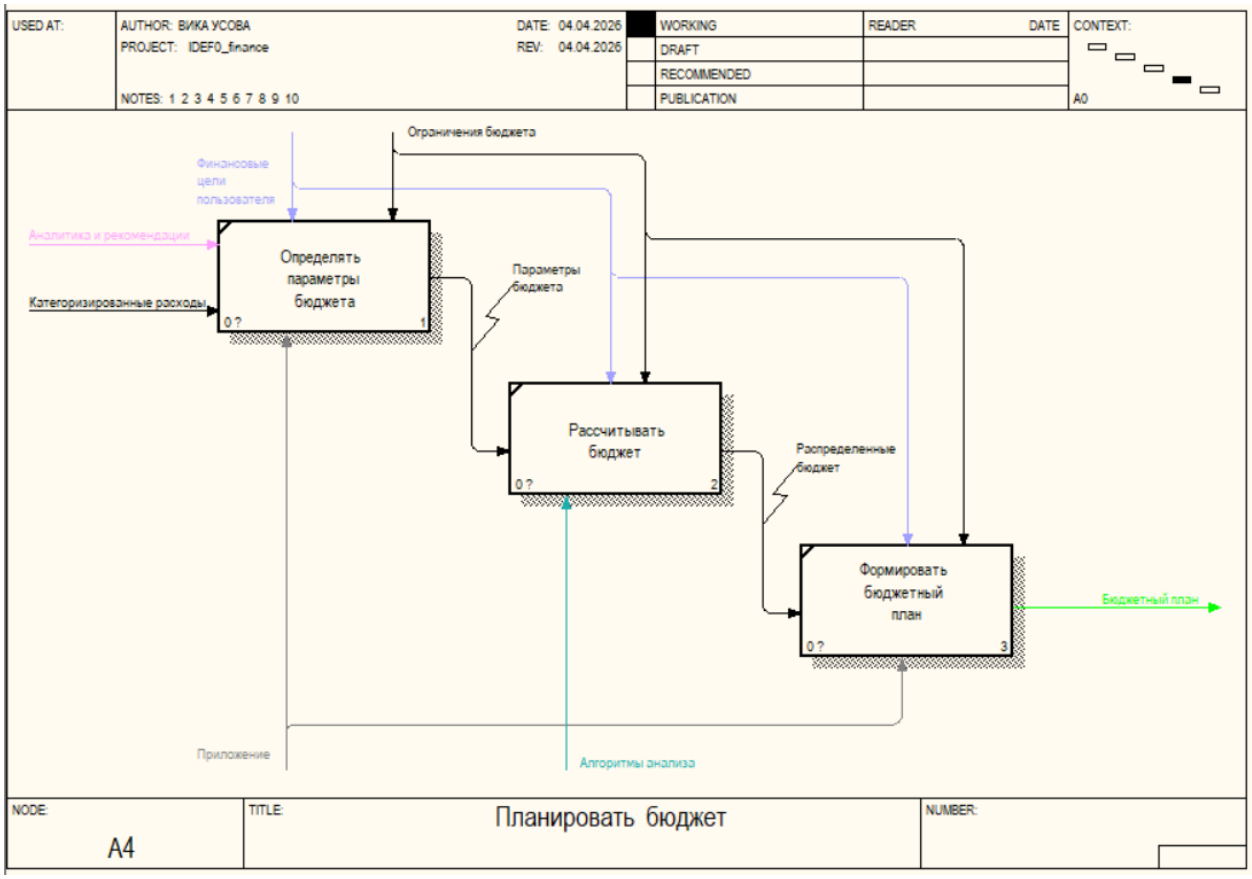


Рисунок 2.10 – Декомпозиция процесса «Планировать бюджет»

Декомпозиция процесса «Формировать отчеты» представлена на рисунке 2.11. Эта диаграмма описывает финальный этап жизненного цикла данных в системе – визуализацию накопленной информации в виде удобных для пользователя отчетов. Здесь все потоки (планы, аналитика, расходы) объединяются в единый результат. Подготовить данные для отчета – выполнить консолидацию информации из различных модулей (бюджетный план, аналитические рекомендации, история категоризированных расходов) и их структурирование для генерации отчетности. Сформировать отчет – произвести техническую сборку отчета в выбранном формате (графики, таблицы, PDF-документы), используя подготовленные данные и алгоритмы визуализации. Отображать отчет – осуществить вывод сформированной финансовой отчетности в пользовательский интерфейс платформы, обеспечивая наглядное представление итоговых данных.

Разделение на «Подготовку» и «Формирование» позволяет системе быстро переключаться между разными видами визуализации без повторного запроса к тяжелым аналитическим модулям, что ускоряет работу интерфейса.

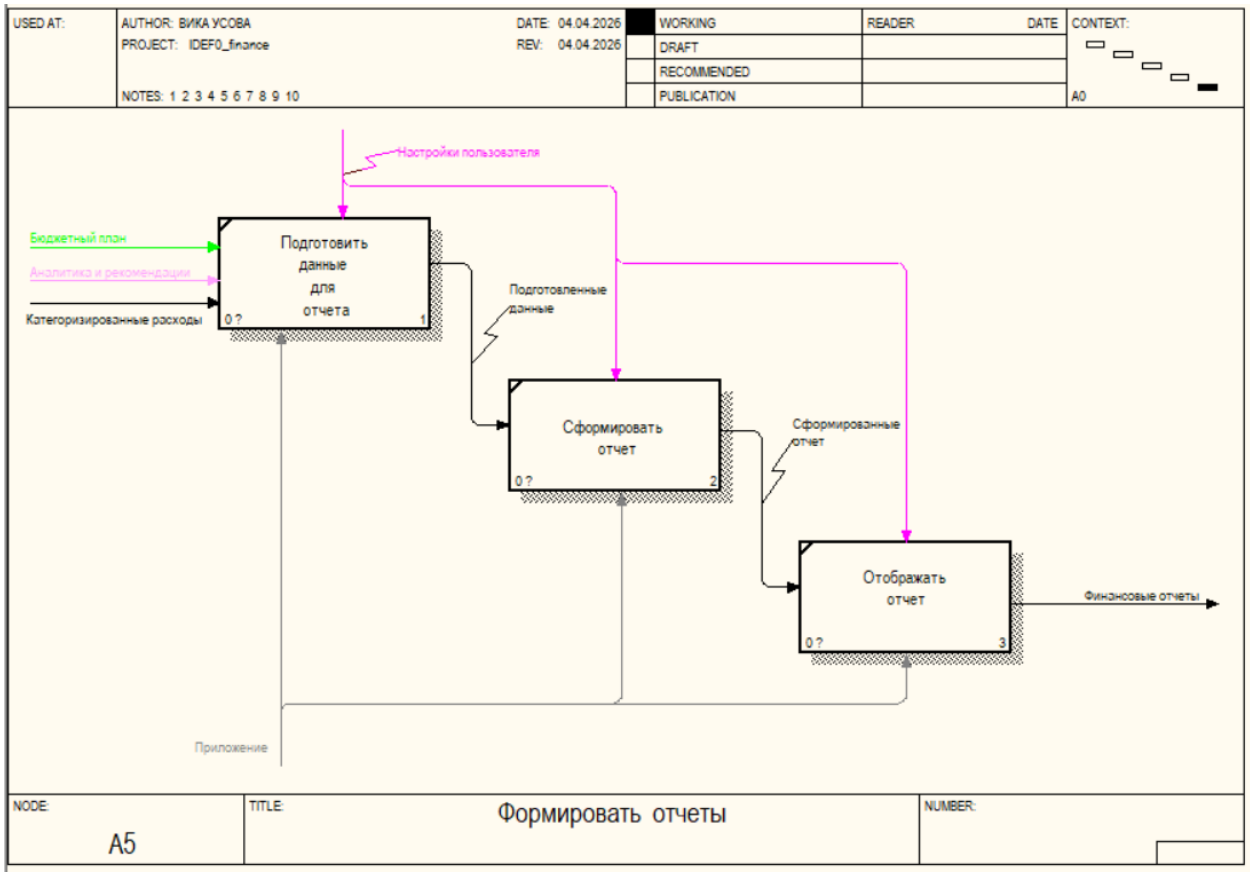


Рисунок 2.11 – Декомпозиция процесса «Формировать отчеты»

Моделирование бизнес-процессов с использованием нотации IDEF0 позволило системно описать архитектуру платформы для управления личными финансами, определить функциональные зависимости и установить четкие границы автоматизируемых модулей. Контекстная диаграмма отразила общую структуру взаимодействий между первичными финансовыми потоками, механизмами обработки и управляющими воздействиями со стороны пользователя.

Декомпозиция процессов, проведенная до четвертого уровня, наглядно продемонстрировала взаимосвязь между операциями.

Особое внимание было уделено модулю автоматической валидации и сохранения данных, что позволило детально проработать алгоритмы обеспечения точности финансового учета и исключить попадание некорректной информации в систему.

Разработанная модель стала фундаментом для последующего проектирования структуры базы данных и программной реализации всех функциональных блоков платформы.

2.2 Разработка спецификации требований к программному средству

Для анализа функциональных возможностей платформы для управления личными финансами с модулем категоризации расходов и планирования

бюджета была спроектирована диаграмма вариантов использования (Use Case diagram) в нотации UML. Эта диаграмма позволяет наглядно структурировать требования к программному обеспечению, распределить зоны ответственности между участниками и определить границы взаимодействия каждого типа пользователя с системой. Диаграмма отражает перечень доступных функций и логические отношения между ними, что дает полное представление о сценариях работы платформы [13].

Серверная часть программного обеспечения будет обеспечивать поддержку 20 ключевых сценариев использования, которые охватывают все основные процессы управления личными финансами, включая учет доходов и расходов, планирование бюджета, анализ финансовой динамики, управление целями и обучением финансовой грамотности. Эти сценарии должны обеспечить эффективную и безопасную работу платформы.

Система поддерживает две ключевые роли пользователей. Клиент – авторизованный пользователь, который получает полный доступ к функционалу платформы, включая управление личными финансами, анализ доходов и расходов, настройку уведомлений, работу с целями и планируемыми расходами. Администратор – технический специалист, обеспечивающий стабильность работы платформы, управление пользователями, контроль доступа к информационным ресурсам и настройку параметров обучения финансовой грамотности.

Для более детального разбора функциональных возможностей каждой роли на рисунке 2.12 представлена диаграмма вариантов использования в нотации UML.



Рисунок 2.12 – Диаграмма вариантов использования

Диаграмму начнем рассматривать со стороны клиента. Пользователю доступны следующие ключевые действия.

Авторизация/регистрация – процедура создания учетной записи и входа в систему. Включает валидацию контактных данных, проверку надежности пароля и согласие с политикой обработки персональной информации. После регистрации клиент получает доступ к персонализированным функциям платформы.

Управление профилем – редактирование личных данных (ФИО, контакты), настройка предпочтений отображения интерфейса, настройка валюты и лимитов расходов, а также возможность подключения внешних финансовых сервисов.

Смена пароля – безопасная процедура обновления учетных данных с проверкой текущего пароля и соблюдением требований к сложности нового.

Управление данными о денежных расходах и доходах – добавление, редактирование и удаление записей о финансовых операциях с возможностью указания даты, суммы, категории и комментария. Система обеспечивает проверку корректности данных и автоматическую классификацию расходов.

Управление категориями расходов и доходов – создание, редактирование и удаление категорий, возможность группировать подкатегории, настройка цветовой маркировки для наглядного анализа бюджета.

Управление контактами – создание и редактирование контактов для совместного планирования бюджета, возможность делиться расходами и доходами с другими пользователями или группами.

Управление целями и планируемыми расходами – постановка финансовых целей (например, накопления на поездку), установка лимитов расходов по категориям, контроль выполнения целей через графики и уведомления о прогрессе.

Анализ динамики доходов, расходов и баланса – визуализация финансовых данных через графики, диаграммы и отчеты. Пользователь может отслеживать тренды по категориям, сравнивать текущий период с предыдущими и получать рекомендации по оптимизации расходов.

Управление вкладами и отложенными доходами – учет накоплений, депозитов, инвестиционных инструментов и прогнозируемых доходов, с расчетом процентов, графиков начисления и отображением общей финансовой картины.

Управление информационными ресурсами и обучение финансовой грамотности – доступ к обучающим материалам (статьи, видеоуроки, тесты), отслеживание прогресса обучения, выполнение заданий и получение обратной связи от платформы.

Управление общими целями и расходами – совместное планирование бюджета с другими пользователями, распределение расходов, контроль достижения общих целей и синхронизация изменений в реальном времени.

Настройка уведомлений и публичного доступа – настройка push-уведомлений и электронной почты о новых расходах, доходах, достижении

целей, а также возможность сделать часть данных публичными или доступными определенной группе пользователей.

Поиск, фильтрация и сортировка данных – возможность быстро находить записи по ключевым параметрам (дата, категория, сумма, контакты), фильтровать операции по категориям и сортировать их для удобного анализа.

Администратор взаимодействует с платформой через специализированный интерфейс, обеспечивая безопасность, контроль и поддержку работы системы. Основные действия:

Авторизация и смена пароля – безопасный вход в административный интерфейс с двухфакторной аутентификацией, обновление учетных данных.

Управление пользователями – создание, редактирование и удаление учетных записей клиентов, настройка уровней доступа, блокировка подозрительных пользователей, восстановление доступа по запросам и ведение журналов активности.

Управление информационными ресурсами и обучением финансовой грамотности – добавление, обновление и удаление обучающих материалов, настройка тестов, отслеживание результатов пользователей, генерация статистики об успеваемости и вовлеченности.

Поиск, фильтрация и сортировка данных – инструменты для анализа активности пользователей, контроля статистики по расходам и доходам, поиска данных по категориям, контактам и финансовым операциям для аудита и поддержки клиентов.

Настройка платформы и уведомлений – настройка глобальных параметров системы, включая уведомления, шаблоны сообщений, правила публичного доступа и контроль безопасности данных.

Аналитика и отчеты – формирование отчетов по активности пользователей, популярности категорий расходов, выполнению целей и эффективности обучающих программ.

Представленная диаграмма вариантов использования демонстрирует продуманную структуру платформы и высокую степень автоматизации функционала. Четкое разделение ролей клиента и администратора обеспечивает безопасное и эффективное взаимодействие с системой.

Особого внимания заслуживают интеллектуальные модули анализа финансовой динамики и планирования бюджета, которые позволяют пользователю принимать обоснованные решения, прогнозировать расходы и доходы, а также ставить и контролировать финансовые цели.

Модульная структура функций администратора обеспечивает масштабируемость и поддерживаемость платформы, позволяя оперативно управлять пользователями, информационными ресурсами и настройками обучения финансовой грамотности.

В целом, спроектированная диаграмма вариантов использования полностью отражает требования к платформе для управления личными финансами, охватывает все ключевые бизнес-процессы и создает надежную

основу для дальнейшего проектирования архитектуры и разработки программного обеспечения.

2.3 Образ предлагаемого решения

В современных условиях цифровизации управление личными финансами требует системного подхода, обеспечивающего прозрачность, контроль и автоматизацию процессов. Для решения задач учета доходов и расходов, планирования бюджета, анализа финансовой динамики и формирования финансовых целей предлагается разработка комплексной платформы для управления личными финансами с модулем категоризации расходов и планирования бюджета. Основной идеей платформы является объединение всех ключевых функций в едином цифровом пространстве, обеспечивающем простоту взаимодействия пользователя с системой и гибкость настройки под индивидуальные потребности.

Центральным элементом архитектуры является серверный модуль управления финансами, который координирует работу всех подсистем платформы. В его состав входят модули учета финансовых операций, категоризации расходов и доходов, планирования бюджета, управления целями и совместными финансами, а также аналитики и формирования отчетов. Каждый модуль выполняет специализированные функции, при этом взаимодействие между ними позволяет реализовать комплексный подход к управлению средствами, обеспечивая актуальность и точность данных, а также автоматизацию вычислений и прогнозирования.

Модуль учета операций отвечает за регистрацию всех финансовых транзакций, их валидацию и первичную обработку. Он фиксирует сумму, дату, категорию и комментарий к каждой операции, что позволяет формировать полную картину доходов и расходов пользователя. Информация из этого модуля поступает в модуль категоризации, который обеспечивает группировку расходов по категориям и подкатегориям. Это упрощает анализ финансовых потоков и позволяет пользователю видеть структуру своих расходов, выявлять дисбаланс между обязательными и необязательными тратами, а также оценивать возможности для оптимизации бюджета.

Модуль планирования бюджета и управления целями интегрирован с системой уведомлений и аналитики. Он позволяет задавать лимиты расходов по категориям, планировать накопления и контролировать выполнение финансовых целей. Система автоматически сопоставляет фактические траты с запланированными значениями и уведомляет пользователя о превышениях или достижении целей. Взаимодействие с модулем аналитики и отчетов обеспечивает генерацию графиков, диаграмм и статистических сводок, что позволяет пользователю видеть динамику доходов, расходов и баланса, анализировать тенденции и принимать обоснованные решения о корректировке бюджета.

Особое внимание уделено работе с совместными финансами и обучению финансовой грамотности. Модуль совместного планирования позволяет пользователям обмениваться информацией о расходах и доходах, ставить общие цели и отслеживать прогресс, обеспечивая синхронизацию данных в реальном времени. Модуль финансового обучения предоставляет доступ к образовательным материалам, тестам и интерактивным курсам, позволяя пользователям развивать навыки финансового планирования и принимать более обоснованные решения.

Администратор взаимодействует с системой через отдельный интерфейс, позволяющий управлять пользователями, контролировать доступ к информационным ресурсам, настраивать обучение и уведомления, а также проводить аудит активности. Это обеспечивает безопасность платформы, поддержание корректности данных и возможность масштабирования функционала без влияния на работу конечных пользователей.

Все данные платформы хранятся в централизованной базе данных, которая обеспечивает сохранность информации о пользователях, финансовых операциях, категориях и целях, контактах, образовательных материалах и истории активности. Это позволяет каждому модулю получать доступ к актуальной информации и поддерживать согласованность данных, а также реализовывать многоуровневую аналитику и прогнозирование на основе исторических данных. Архитектура системы поддерживает многопользовательский режим, интеграцию с внешними финансовыми сервисами, защиту персональных данных и резервное копирование информации.

Таким образом, предлагаемое решение представляет собой клиент-серверную платформу, в которой клиентский интерфейс обеспечивает удобное взаимодействие пользователя с функционалом системы, а серверный модуль координирует работу всех подсистем, обеспечивая целостность и актуальность данных. Интеграция модулей учета, аналитики, планирования и обучения позволяет создать единое пространство для управления личными финансами, обеспечивая баланс между удобством использования, глубиной аналитики и гибкостью настройки.

На рисунке 2.13 представлена функциональная архитектура решения, отражающая взаимосвязь клиентского интерфейса, серверного модуля и базы данных, а также основные функциональные подсистемы, обеспечивающие работу платформы [14].



Рисунок 2.13 – Функциональная архитектура решения

Для реализации всех заявленных функций платформа будет базироваться на архитектуре клиент-серверного веб-приложения. Такое решение обеспечит пользователям стабильный доступ к финансовым инструментам с любых устройств через браузер, сохраняя при этом высокую производительность системы.

Для надежной работы с данными выбрана реляционная СУБД, управление которой будет осуществляться через специализированные инструменты миграции. Это позволит гибко масштабировать систему и изменять структуру данных без прерывания рабочих процессов. Высокая скорость отклика интерфейса будет достигнута за счет внедрения системы кэширования, которая снизит нагрузку на базу данных при обработке сложных аналитических запросов.

Особое внимание в технической реализации уделено безопасности и интеграционным возможностям. Взаимодействие с внешними сервисами через REST API позволит расширять функционал платформы, а многоуровневая защита, включающая шифрование трафика и защищенное хранение паролей, гарантирует конфиденциальность персональной информации. Таким образом, выбранный технологический стек создает надежный фундамент для построения удобной, быстрой и защищенной системы управления личными финансами.

3 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Архитектурные решения и технологии реализации программного средства

В рамках курсового проекта разрабатывается программное средство, представляющее собой платформу для управления личными финансами с модулем автоматической категоризации расходов и планирования бюджета. Основной целью системы является автоматизация учета денежных потоков, интеллектуальный анализ совершенных транзакций и предоставление пользователю обоснованных рекомендаций для эффективного распределения личных средств и достижения финансовых целей.

Разрабатываемая платформа реализована с использованием клиент-серверной архитектуры, которая позволяет эффективно разделить функциональность между различными компонентами системы. Такой подход обеспечивает необходимую масштабируемость при обработке больших массивов транзакций, гибкость при интеграции модуля категоризации и удобство сопровождения программного кода.

В архитектуре информационной системы выделяются две ключевые части. Серверная часть выступает в роли поставщика бизнес-логики и хранилища данных. Она отвечает за безопасное содержание финансовых записей, выполнение алгоритмов классификации расходов и расчет бюджетных лимитов. Клиентская часть выполняет роль интерфейса взаимодействия с пользователем. Она обеспечивает визуализацию аналитических отчетов, отображение графиков распределения бюджета и предоставляет инструменты для оперативного ввода данных о доходах и расходах.

Такое разделение позволяет перенести тяжелые вычислительные процессы (например, работу с базой данных и автоматическую категоризацию) на сервер, обеспечивая при этом высокую скорость отклика и удобство работы пользователя в интерфейсе приложения.

Ключевой принцип клиент-серверной архитектуры платформы заключается в разделении системы на три основных уровня:

– уровень представления (интерфейс пользователя);

– уровень бизнес-логики (обработка финансовых операций и категоризация);

– уровень управления данными (хранение транзакций и настроек бюджета).

Таким образом, архитектура разрабатываемого программного средства включает следующие компоненты:

– клиентское приложение (Frontend);

– серверное приложение (Backend);

– система хранения данных (база данных PostgreSQL и кэширование).

Клиентская часть реализована в виде одностраничного веб-приложения (SPA). В качестве основного технологического стека используются:

– язык программирования TypeScript;

– библиотека React;

– инструмент сборки проекта Vite.

Применение SPA-подхода позволяет обеспечить высокую скорость взаимодействия с пользователем за счет динамической подгрузки финансовых данных и мгновенного обновления графиков без полной перезагрузки страницы.

Клиентская часть отвечает за:

– отображение интерактивного интерфейса пользователя;

– оперативный ввод и первичную валидацию данных о доходах и расходах;

– отправку запросов к серверу через протокол HTTPS;

– визуализацию категоризированных расходов в виде диаграмм и таблиц;

– отображение текущего состояния исполнения бюджета;

– вывод персональных финансовых рекомендаций и аналитики.

Для управления состоянием приложения используется централизованный механизм (React Context), что позволяет синхронизировать данные о балансе и транзакциях между различными экранами платформы. Взаимодействие с серверной частью осуществляется через REST API с использованием формата данных JSON, что обеспечивает легкость интеграции и высокую скорость обмена информацией.

Серверная часть платформы реализована на языке программирования Java с использованием фреймворка Spring Boot. Сервер представляет собой REST API, обеспечивающий высокопроизводительную обработку запросов от клиентского приложения и выполнение бизнес-логики системы.

В рамках архитектуры используется подход модульного монолита с элементами чистой архитектуры, что позволяет разделить систему на логические слои:

– контроллеры (Controller) – прием и первичная обработка HTTP-запросов от фронтенда;

– сервисы (Service) – реализация ключевой бизнес-логики (алгоритмы категоризации, расчет бюджета);

– репозитории (Repository) – абстракция для работы с базой данных через Spring Data JPA;

– модели данных (Entity, DTO) – описание структуры таблиц и объектов передачи данных.

Серверная часть выполняет следующие функции:

– обработка и сохранение финансовых транзакций (доходов и расходов);

– автоматическая категоризация расходов на основе заданных правил и алгоритмов;

– хранение истории транзакций и метаданных операций;

– расчет исполнения бюджета и отслеживание лимитов в реальном времени;

– формирование персональной аналитики и финансовых рекомендаций;

– аутентификация и безопасная авторизация пользователей.

Взаимодействие между слоями осуществляется с использованием принципов инверсии и внедрения зависимостей (Dependency Injection), реализованных средствами Spring Framework.

В качестве системы управления базами данных используется реляционная СУБД PostgreSQL. База данных содержит взаимосвязанные таблицы, приведенные к третьей нормальной форме. Доступ к данным осуществляется с помощью Spring Data JPA, что обеспечивает типобезопасность и удобство разработки. Для повышения производительности при получении частых аналитических срезов используется механизм кэширования Redis.

Связь между клиентской и серверной частями происходит по протоколу HTTPS с использованием RESTful подхода. Все данные передаются в формате JSON. Основные принципы взаимодействия включают:

– использование стандартных HTTP-методов (GET для получения отчетов, POST для создания транзакций, PUT/PATCH для корректировки бюджета);

– корректная обработка статус-кодов (200 OK, 201 Created, 400 Bad Request и др.);

– поддержка фильтрации транзакций по датам, категориям и суммам.

Для защиты персональных финансовых данных реализована система на базе Spring Security и JWT (JSON Web Token). Это позволяет безопасно передавать состояние авторизации между клиентом и сервером. Основные меры безопасности включают:

– хеширование паролей (BCrypt);

– ролевая модель доступа (User/Admin);

– защита API от несанкционированного доступа;

– строгая валидация входящих финансовых данных на стороне сервера.

Диаграмма развертывания, изображенная на рисунке 3.1, использовалась для описания конфигурации узлов внутри системы [15].

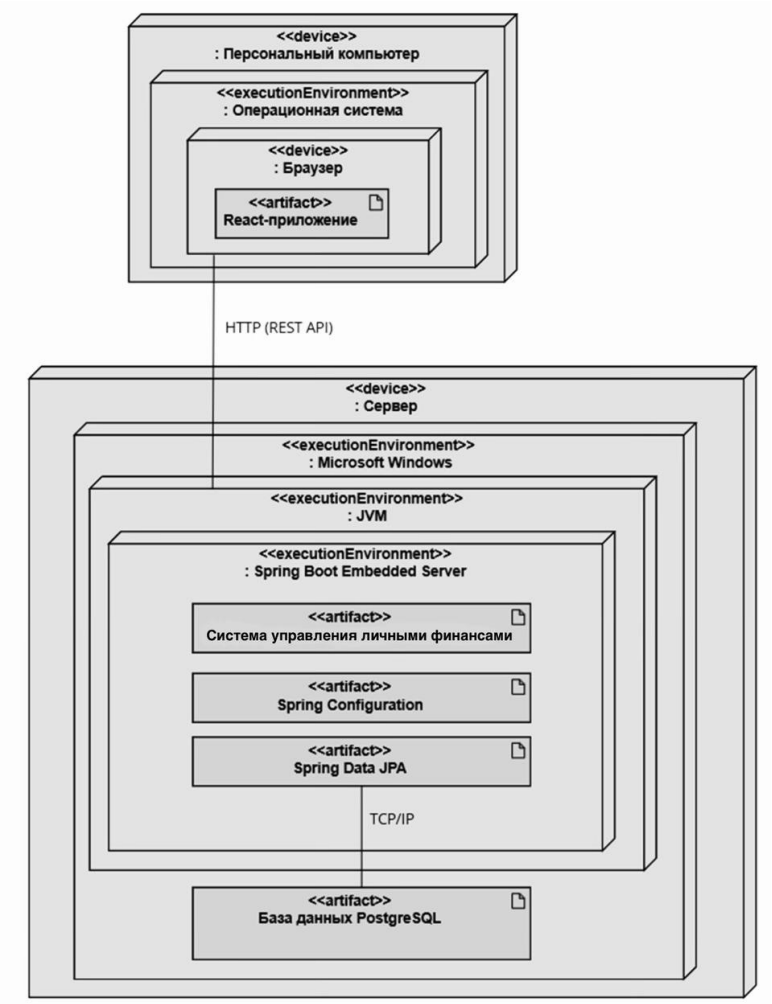


Рисунок 3.1 – Диаграмма развертывания

Представленная на рисунке 3.1 диаграмма развертывания демонстрирует физическую архитектуру платформы и взаимодействие между ключевыми узлами системы: «персональный компьютер», «сервер» и «база данных PostgreSQL».

Узел «персональный компьютер» представляет собой клиентское устройство пользователя. В среде выполнения «операционная система» через «браузер» развернут артефакт «React-приложение». Данный компонент является фронтенд-частью системы, которая отвечает за визуализацию интерфейса управления финансами. Взаимодействие с серверной частью осуществляется по протоколу HTTP (REST API).

Узел «сервер» функционирует под управлением операционной системы Microsoft Windows. Среда выполнения представлена виртуальной машиной Java (JVM), внутри которой запущен «Spring Boot Embedded Server». В данной среде развернуты ключевые программные артефакты:

– «Система управления личными финансами» – основной модуль, содержащий бизнес-логику категоризации и планирования;

– «Spring Configuration» – компоненты настройки программной среды;

– «Spring Data JPA» – программный слой для обеспечения объектно-реляционного отображения и взаимодействия с хранилищем данных.

Узел «База данных PostgreSQL» выделен как отдельный артефакт, работающий в связке с сервером приложений по протоколу TCP/IP. Данный узел отвечает за надежное хранение структурированной информации: профилей пользователей, истории транзакций, настроек категорий и параметров бюджетных планов.

Выбранная схема развертывания обеспечивает логическое и физическое разделение уровней представления, обработки и хранения данных, что гарантирует стабильность работы платформы и безопасность обработки финансовых сведений пользователя.

Для детального анализа внутренней архитектуры платформы управления личными финансами была разработана диаграмма компонентов, представленная на рисунке 3.2. Данная диаграмма визуализирует компонентную структуру системы, разделение на логические пакеты и используемые библиотеки [16].

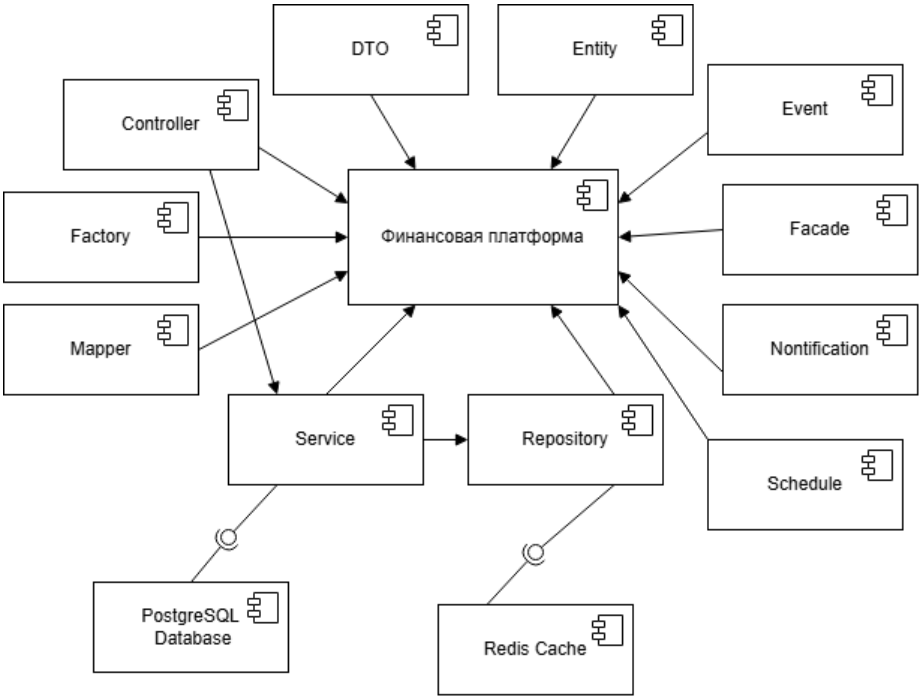


Рисунок 3.2 – Диаграмма компонентов

Представленная на рисунке 3.2 диаграмма компонентов отражает внутреннюю архитектуру серверной части платформы управления личными финансами, визуализируя ключевые функциональные модули и логику их взаимодействия. Центральным элементом системы является компонент «Финансовая платформа», который объединяет в себе бизнес-логику и координирует работу всех подсистем.

Архитектура системы построена на разделении ответственности между компонентами. Слой представления и передачи данных, в нем компоненты

Controller, DTO и Mapper обеспечивают обработку входящих API-запросов и преобразование данных между внутренними сущностями и внешними объектами обмена.

Слой бизнес-логики включает компоненты Service, Facade, Factory и Event. Данные блоки отвечают за реализацию алгоритмов автоматической категоризации транзакций, расчет лимитов бюджета и обработку внутренних событий системы.

Слой вспомогательных сервисов, в нем компоненты Notification, Schedule и Entity обеспечивают работу системы уведомлений, выполнение регламентных задач по расписанию и описание структуры данных.

Слой доступа к данным, в котором компонент Repository организует взаимодействие с PostgreSQL Database для обеспечения долгосрочного хранения информации, а также связь с Redis Cache для оптимизации доступа к часто запрашиваемым аналитическим данным.

Применение современного технологического стека, включающего Java 21, Spring Boot 3 и React, позволяет создать отказоустойчивое и легко масштабируемое решение. Использование профессиональных инструментов разработки, таких как IntelliJ IDEA и СУБД PostgreSQL, в сочетании с четким разделением системы на компоненты, способствует эффективному проектированию, поддержке и дальнейшему расширению функциональных возможностей платформы.

Таким образом, выбранные архитектурные подходы и технологический стек полностью соответствуют поставленным задачам. Реализованные решения позволяют создать современное, отказоустойчивое и эффективное программное средство для автоматизированного управления личными финансами, обладающее развитыми интеллектуальными функциями категоризации транзакций и предиктивного планирования бюджета. Использование многоуровневой структуры и клиент-серверного взаимодействия обеспечивает системе необходимую гибкость для дальнейшего масштабирования и внедрения новых аналитических инструментов.

3.2 Проектирование и разработка пользовательского интерфейса

Проектирование пользовательского интерфейса (UI) является критическим этапом формирования визуальной и интерактивной среды, через которую пользователь взаимодействует с внутренней бизнес-логикой системы управления финансами. Основной акцент при разработке был сделан на принципах UX (User Experience): интуитивности навигации при работе со счетами, минимизации количества действий (кликов) для фиксации расхода и консистентности графических элементов аналитики [17].

Для создания высокоточных прототипов использовался графический редактор Figma, что позволило детально проработать композиционные схемы дашбордов, цветовую палитру и типографику до начала этапа активной

разработки. Техническая реализация фронтенд-части выполнена в виде SPA (Single Page Application) на базе библиотеки React 18 и языка TypeScript, что обеспечивает строгую типизацию финансовых данных и высокую производительность интерфейса за счет механизмов Virtual DOM.

Архитектура клиентской части базируется на следующих решениях. Компонентный подход – каждый элемент интерфейса представляет собой независимый изолированный модуль. Управление глобальным состоянием – синхронизация данных между различными экранами реализована с помощью современных инструментов управления состоянием (Redux Toolkit / Zustand). Сетевое взаимодействие – обмен данными с серверным API организован через библиотеку Axios с применением перехватчиков (interceptors) для безопасной передачи JWT-токенов.

Для того, чтобы продемонстрировать реализованный пользовательский интерфейс, рассмотрим меню регистрации и входа, а также главное меню. Меню входа представлено на рисунке 3.3.

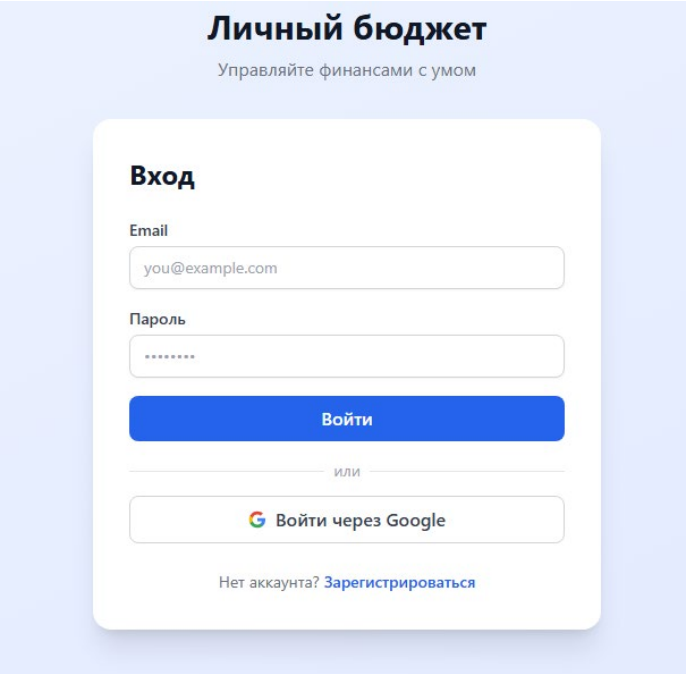


Рисунок 3.3 – Меню входа

На рисунке 3.3 представлена экранная форма «Вход», которая является единой точкой доступа пользователя к личной финансовой панели. Дизайн формы выполнен в минималистичном стиле с акцентом на удобство и безопасность использования. Основными элементами экранной формы являются блок идентификации, который содержит текстовые поля для ввода адреса электронной почты (email) и пароля. Поле ввода пароля поддерживает маскирование символов для обеспечения конфиденциальности.

Кнопка основного действия: ярко-синяя кнопка «Войти» инициирует процесс валидации данных на стороне сервера и переход к основной функциональности платформы.

Альтернативный способ входа, в котором реализована интеграция с внешним сервисом аутентификации Google, что позволяет сократить время авторизации за счет использования существующего защищенного аккаунта.

Навигационный блок, в котором под основной формой расположена гиперссылка «Зарегистрироваться», ведущая на окно создания нового профиля для пользователей, не имеющих учетной записи в системе.

Интерфейс спроектирован с учетом адаптивности, обеспечивая корректное отображение формы как на стационарных компьютерах, так и на мобильных устройствах.

На рисунке 3.4 представлено окно регистрации.

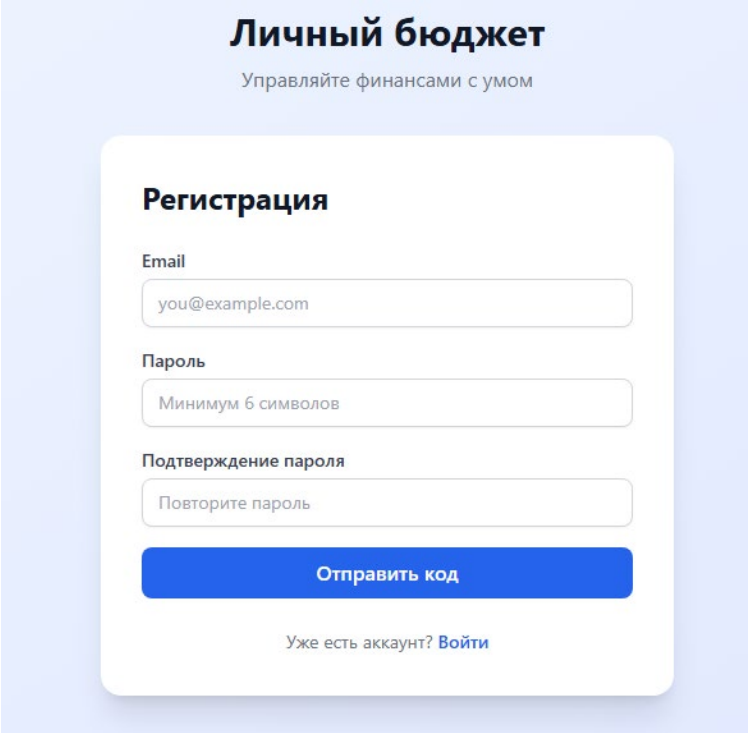


Рисунок 3.4 – Меню регистрации

На рисунке 3.4 представлена экранная форма «Регистрация», предназначенная для создания новой учетной записи в системе. Интерфейс выполнен в единой стилистике с формой входа, что обеспечивает визуальную целостность приложения.

Основными элементами экранной формы являются поля ввода данных:

– email: текстовое поле с валидацией формата адреса электронной почты;

– пароль: поле с подсказкой о минимальной длине (6 символов), обеспечивающее соблюдение базовых требований безопасности;

– подтверждение пароля: дополнительное поле для проверки корректности введенного пароля и предотвращения ошибок при регистрации.

Кнопка подтверждения – функциональная кнопка «Отправить код» инициирует процесс двухфакторной проверки (например, через email) для верификации владения указанным почтовым ящиком.

Вспомогательная навигация – в нижней части формы расположена активная ссылка «Войти», позволяющая пользователю быстро вернуться к окну авторизации, если учетная запись уже была создана ранее.

Реализация данной формы на стороне клиента включает в себя первичную проверку совпадения паролей и заполнения всех обязательных полей, что снижает количество ошибочных запросов к серверу и повышает общую отзывчивость интерфейса.

На рисунке 3.5 представлено главное окно.

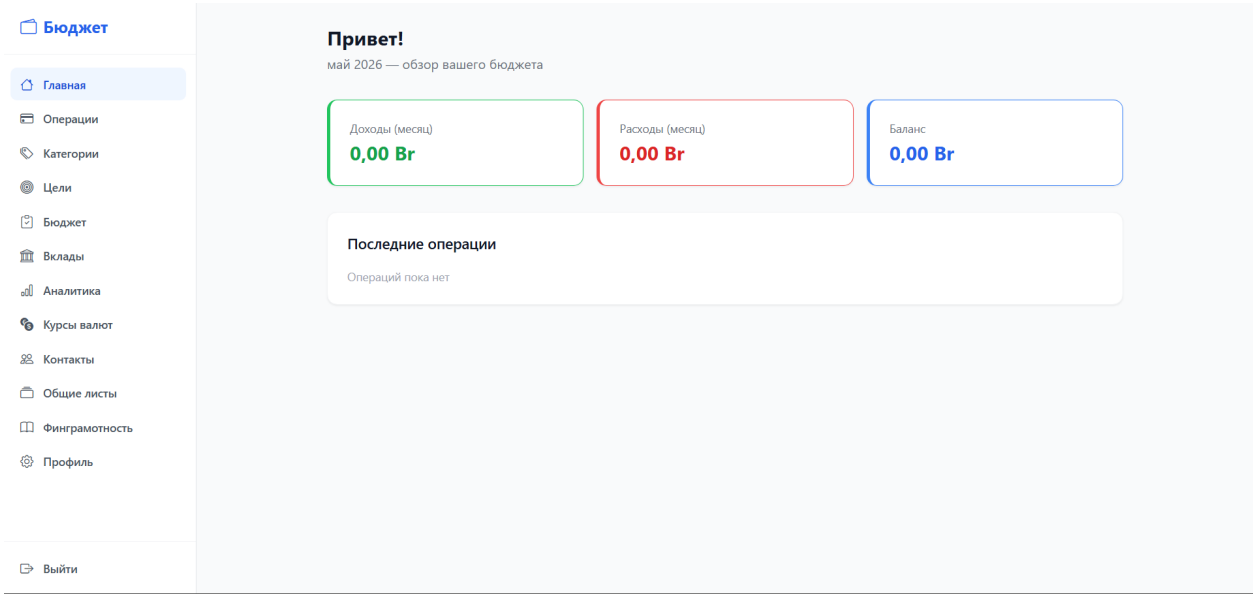


Рисунок 3.5 – Главное меню

На рисунке 3.5 представлена главная страница платформы, которая служит основным центром визуализации финансового состояния пользователя. Интерфейс спроектирован по принципу модульности, что позволяет быстро оценить ключевые показатели бюджета.

Основные функциональные зоны интерфейса – боковая панель навигации, которая обеспечивает быстрый переход между ключевыми разделами системы: «Операции», «Категории», «Цели», «Бюджет», «Аналитика» и др. Нижняя часть панели содержит функциональную кнопку «Выйти» для корректного завершения сессии.

Информационные виджеты (карточки показателей): доходы отображают суммарный приток средств за текущий период (выделен зеленым цветом). Расходы демонстрируют общую сумму затрат (выделен красным цветом). Баланс показывает текущий остаток денежных средств на счетах пользователя

(выделен синим цветом). Все показатели отображаются в национальной валюте (Br).

Секция «Последние операции» – центральный блок страницы, предназначенный для динамического вывода списка последних транзакций. При отсутствии записей система выводит информационное сообщение «Операций пока нет», что подтверждает корректную инициализацию интерфейса.

Заголовок и контекстная информация, в них содержится персонализированное приветствие и указание текущего отчетного периода (апрель 2026 г.), что позволяет пользователю ориентироваться в актуальности представленных данных.

Интерфейс реализован с использованием библиотеки Tailwind CSS, что обеспечивает «чистый» современный вид и адаптивность элементов под различные разрешения экранов. Применение иконок рядом с пунктами меню повышает интуитивность навигации и снижает когнитивную нагрузку на пользователя.

Таким образом, использование технологической связки React и TypeScript позволило создать высокопроизводительное, масштабируемое и функциональное веб-приложение, которое полностью отвечает современным требованиям в области учета личных финансов, автоматизированной категоризации транзакций и эффективного планирования бюджета. Выбранный стек обеспечил надежность обработки данных и удобство пользовательского интерфейса, необходимое для работы с персональной финансовой аналитикой.

3.3 Разработка модели данных

Одним из определяющих факторов качества спроектированной модели данных является её соответствие принципам нормализации, которые направлены на минимизацию избыточности и предотвращение аномалий. Процесс нормализации строится по кумулятивному принципу: каждая последующая нормальная форма (НФ) базируется на требованиях предыдущих. Так, достижение моделью 3НФ автоматически гарантирует соблюдение правил 1НФ и 2НФ.

Первая нормальная форма требует обеспечить атомарность данных, при которой каждое поле таблицы содержит только одно неделимое значение.

Вторая нормальная форма, помимо соблюдения условий 1НФ, предписывает исключение частичных зависимостей. Все неключевые атрибуты должны полностью зависеть от всего составного первичного ключа, а не от его отдельных частей.

Третья нормальная форма, основываясь на требованиях первых двух форм, накладывает запрет на транзитивные зависимости. Это означает, что любой неключевой атрибут должен зависеть исключительно от первичного ключа, а не от других неключевых полей.

Такой иерархический подход к структурированию данных позволяет создать логически строгую архитектуру базы данных, обеспечивающую целостность информации и удобство её дальнейшего обслуживания [18].

На рисунке 3.6 представлена информационная модель базы данных.

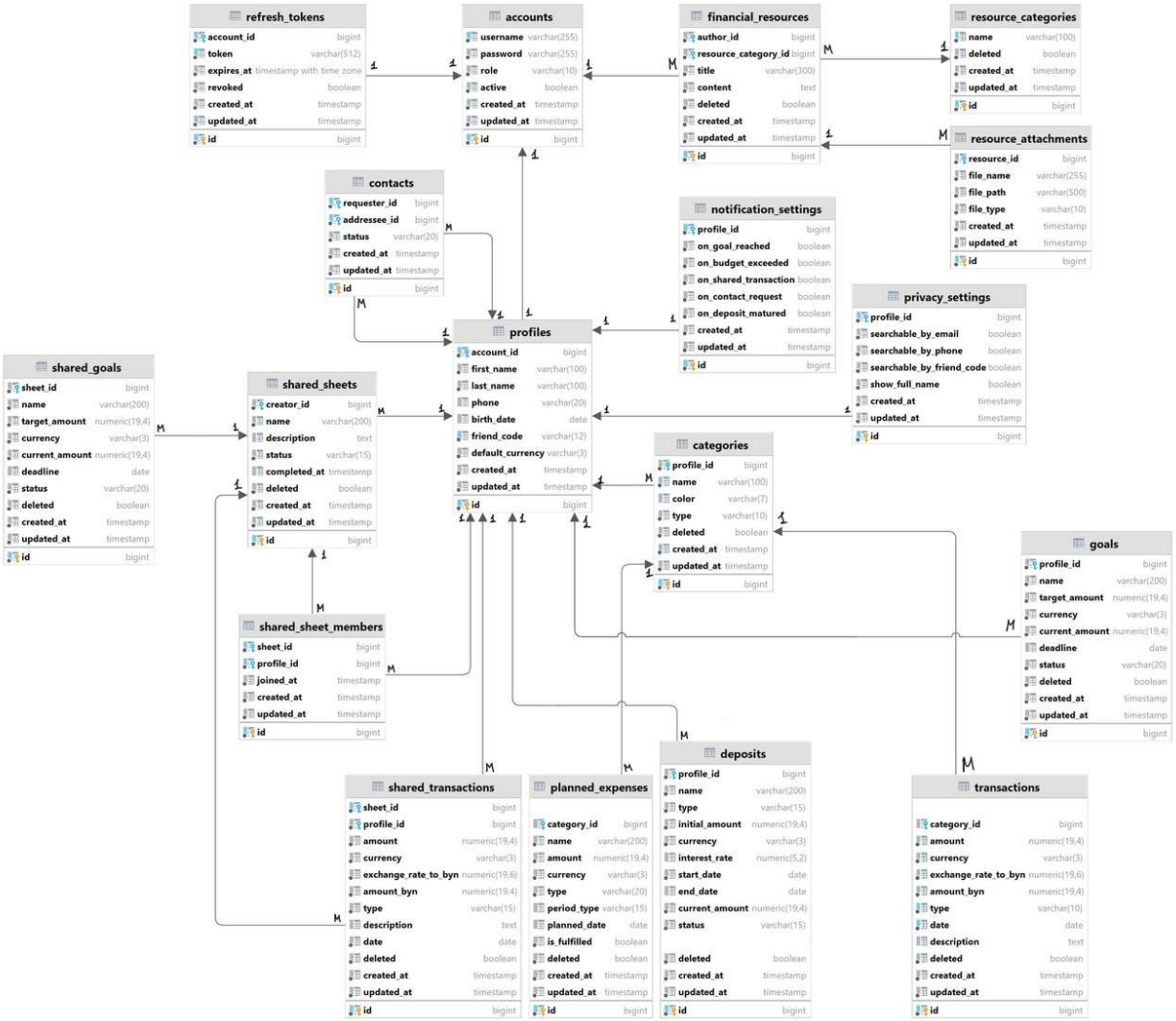


Рисунок 3.6 – Информационная модель базы данных

Данная диаграмма представляет собой информационную модель платформы управления личными финансами, разработанную с учетом принципов нормализации реляционных баз данных. Чтобы продемонстрировать ее соответствие третьей нормальной форме (3НФ), был проведен анализ структуры таблиц и связей между ними. В представленной модели каждая таблица обладает уникальным первичным ключом (id). Поскольку все поля содержат неделимые значения (атомарность), модель удовлетворяет требованиям первой нормальной формы (1НФ), а отсутствие частичных зависимостей неключевых атрибутов от первичного ключа подтверждает соответствие второй нормальной форме (2НФ).

Ниже приведено детальное описание таблиц, входящих в состав базы данных. Accounts: содержит учетные данные пользователей (username, password), их роль в системе и статус активности. Все атрибуты напрямую зависят от первичного ключа id.

Profiles: центральная таблица профилей пользователей, включающая персональные данные (first_name, last_name, phone, birth_date) и уникальный friend_code. Поля атомарны, транзитивные зависимости отсутствуют (3НФ).

Refresh_tokens: техническая таблица для управления сессиями, хранящая токены и время их истечения. Все поля зависят исключительно от первичного ключа.

Categories: справочник категорий расходов и доходов, привязанный к профилю пользователя. Содержит наименование, цвет и тип категории.

Transactions: основная таблица финансовых операций, фиксирующая сумму, валюту, дату и описание транзакции. Включает внешние ключи для связи с профилем и категорией.

Planned_expenses: таблица планируемых расходов, позволяющая пользователю устанавливать бюджетные лимиты на определенные периоды.

Deposits: содержит информацию о банковских вкладах пользователя, включая процентную ставку, сроки начала и окончания депозита.

Goals: таблица финансовых целей, где фиксируется целевая сумма, текущий прогресс и крайний срок (deadline).

Financial_resources: хранилище полезных статей или обучающих материалов по финансовой грамотности, разделенных по категориям в таблице resource_categories.

Resource_attachments: содержит сведения о файлах, прикрепленных к финансовым ресурсам (пути к файлам, типы и расширения).

Notification_settings: индивидуальные настройки уведомлений для профиля (превышение бюджета, достижение цели и т.д.), что исключает избыточность в основной таблице профилей.

Privacy_settings: управляет параметрами видимости профиля для других пользователей системы.

Contacts: реализует социальную механику, храня связи между пользователями (запросы в друзья и их статусы).

Shared_sheets и shared_sheet_members: обеспечивают функционал совместного управления бюджетом, разделяя доступ к общим спискам транзакций между несколькими участниками.

Shared_goals и shared_transactions: специализированные таблицы для учета коллективных финансовых целей и операций в рамках общих счетов.

Таким образом, все таблицы в представленной информационной модели разработаны в соответствии с требованиями третьей нормальной формы (3НФ). Это обеспечивает высокую эффективность хранения данных, минимизацию дублирования информации и поддержание строгой целостности базы данных. Выбранная структура является надежным фундаментом для

реализации сложной бизнес-логики финансовой платформы, включая аналитику и многопользовательский доступ.

Далее будет описана диаграмма классов, которые используются в системе. Данная диаграмма классов детально представляет различные виды взаимодействия классов в системе. На рисунке Г.1. приложения Г представлена общая диаграмма классов, реализующая логику управления личными финансами [19].

На представленной диаграмме изображены основные классы сущностей, их интерфейсы-строители (Builders) и перечисления, которые формируют каркас приложения.

Группа классов управления профилем и безопасностью. Класс Profile является центральной сущностью и содержит персональные данные пользователя:

– firstName, lastName – имя и фамилия;

– phone, birthDate – контактные данные и дата рождения;

– friendCode – уникальный идентификатор для социальных взаимодействий;

– defaultCurrency – предпочтительная валюта учета.

Класс Account: отвечает за аутентификацию:

– username, password – учетные данные;

– role – роль пользователя в системе;

– active – статус учетной записи.

Класс PrivacySettings: управляет приватностью профиля (видимость по email, телефону или коду). Класс NotificationSettings: содержит логические флаги для системных уведомлений (превышение бюджета, достижение цели, новости).

Группа классов финансового учета. Класс Transaction описывает финансовую операцию:

– amount – сумма транзакции;

– currency – валюта;

– date – дата совершения;

– type – тип (доход/расход).

Данные класс связан с классом Category, который определяет наименование и визуальное оформление (цвет) категории трат.

Класс PlannedExpense служит для планирования бюджета:

– name, amount – название и лимит расхода;

– periodType – периодичность (неделя, месяц);

– isFulfilled – флаг выполнения плана.

Класс Goal описывает финансовые цели:

– targetAmount, currentAmount – целевая и накопленная суммы;

– deadline – срок достижения цели.

Класс Deposit содержит данные о банковских вкладах:

– interestRate – процентная ставка;

– startDate, endDate – период действия вклада.

Группа классов совместного доступа и ресурсов. Класс SharedSheet реализует логику общих счетов:

– name, description – описание общего кошелька;

– status – текущее состояние доступа.

Класс SharedSheetMember – связующая сущность, хранящая данные об участниках общего счета и дате их присоединения (joinedAt).

Класс FinancialResource представляет базу знаний: title, content – заголовок и текст статьи по финансовой грамотности.

Данный класс связан с классом ResourceAttachment, хранящим метаданные файлов (fileName, filePath).

Реализация паттерна Builder. Для каждой крупной сущности в системе реализован паттерн Builder, что отражено наличием соответствующих интерфейсов и классов-реализаций (например, ProfileBuilder, TransactionBuilder, GoalBuilderImpl и т.д.). Интерфейсы строителей определяют методы для пошаговой инициализации объектов (например, setName(), setAmount()). Метод build() гарантирует создание валидного объекта только после заполнения всех необходимых полей.

Данный подход позволяет избежать перегруженных конструкторов и обеспечивает чистоту кода при создании сложных финансовых моделей.

Перечисления (Enums). В системе используются перечисления для типизации данных:

– transactionType – типы операций (INCOME, EXPENSE);

– currency – поддерживаемые валюты (BYN, USD, EUR);

– periodType – периоды планирования (WEEKLY, MONTHLY, YEARLY).

Таким образом, разработанная архитектура объединяет строгую структуру данных и гибкую логику управления. Использование паттерна Builder и четкое разделение на модули (профиль, транзакции, совместный доступ) позволяет легко масштабировать систему. Интеграция настроек приватности и уведомлений делает сервис надежным и персонализированным, создавая качественный фундамент для полноценной работы платформы управления личным бюджетом.

3.4 Модели представления программного средства на основе UML

Диаграмма последовательности «Авторизация пользователя с подтверждением email» представляет собой детальную визуализацию процесса двухфакторной аутентификации в системе покупки и продажи бизнеса. Этот инструмент наглядно демонстрирует взаимодействие между ключевыми компонентами системы в рамках выполнения конкретного сценария использования, показывая временной порядок обмена сообщениями между участниками процесса [20]. Диаграмма изображена на рисунке 3.7.

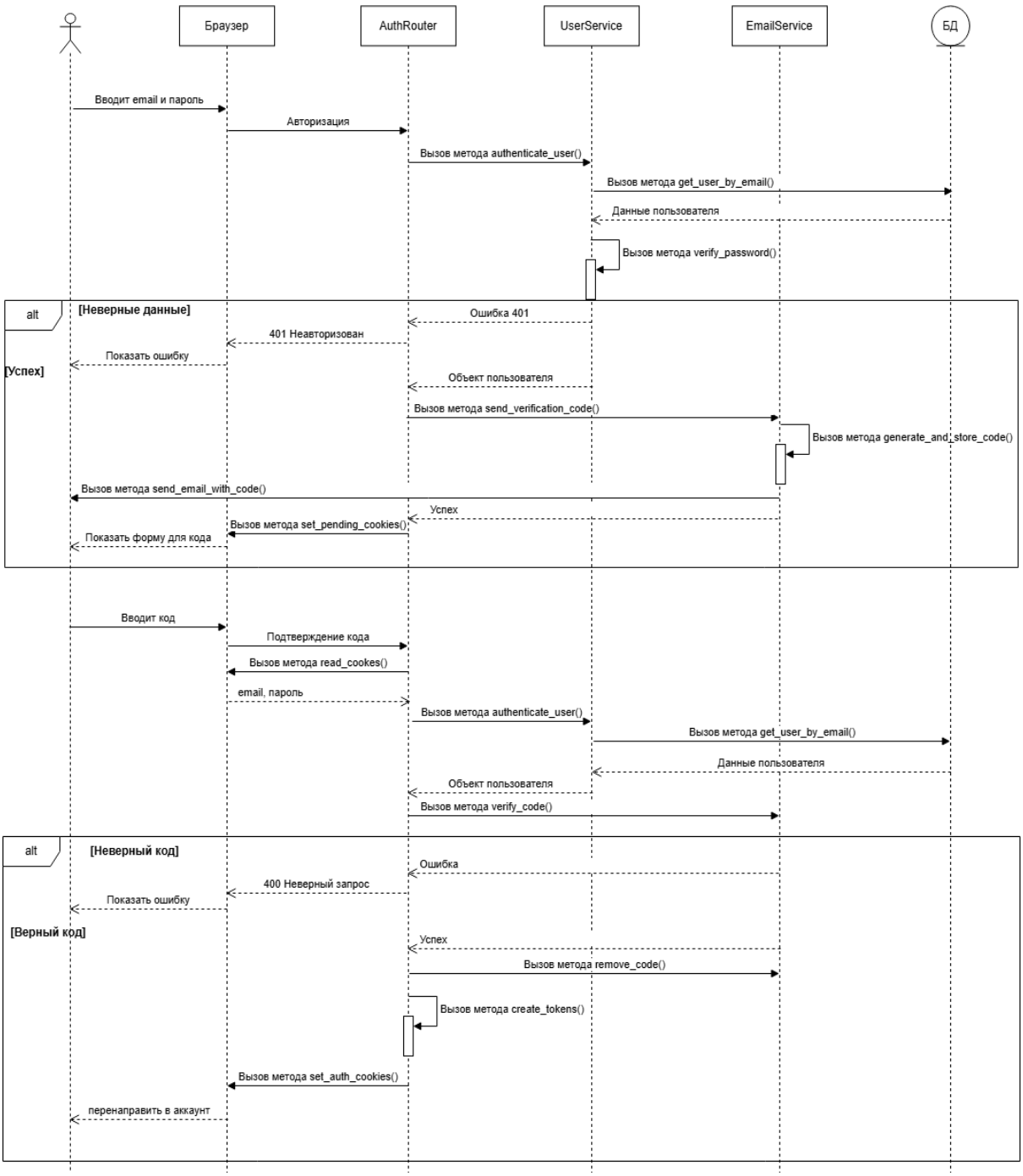


Рисунок 3.7 – Диаграмма последовательности процесса авторизации пользователя с подтверждением email

В диаграмме участвуют шесть основных компонентов системы. Пользователь инициирует процесс взаимодействия через веб-браузер, который выступает в роли клиентской части приложения. Серверная часть представлена маршрутизатором аутентификации AuthRouter, обрабатывающим API-запросы, сервисом пользователей UserService, отвечающим за бизнес-логику работы с учетными данными, сервисом электронной почты EmailService, обеспечивающим отправку кодов подтверждения, и базой данных, хранящей информацию о пользователях.

Процесс аутентификации разделен на две последовательные фазы. Первая фаза начинается с ввода пользователем своих учетных данных в интерфейсе браузера. Браузер передает эти данные маршрутизатору аутентификации через POST-запрос к endpoint /auth/login. Маршрутизатор делегирует проверку учетных данных сервису пользователей, который выполняет запрос к базе данных для поиска пользователя по email и последующей проверки корректности пароля. При неудачной аутентификации система возвращает пользователю сообщение об ошибке. При успешной проверке сервис пользователей возвращает объект пользователя, после чего маршрутизатор инициирует процесс верификации через электронную почту.

Сервис электронной почты генерирует уникальный код подтверждения, сохраняет его во временном хранилище с привязкой к email адресу и отправляет сообщение с этим кодом пользователю. Одновременно с этим маршрутизатор устанавливает в браузере временные cookies, содержащие учетные данные пользователя, которые будут использованы на следующем этапе аутентификации. Браузер отображает пользователю интерфейс для ввода полученного кода подтверждения.

Вторая фаза процесса начинается с ввода пользователем кода из электронного письма. Браузер передает этот код вместе с временными cookies маршрутизатору аутентификации через POST-запрос к endpoint /auth/verify. Маршрутизатор извлекает учетные данные из cookies и повторно выполняет аутентификацию пользователя для подтверждения его идентичности. После успешной повторной аутентификации система проверяет соответствие введенного кода сохраненному в временном хранилище.

При несовпадении кодов система уведомляет пользователя об ошибке и предоставляет возможность повторного ввода. При успешной проверке кода маршрутизатор удаляет использованный код из временного хранилища и генерирует JWT токены доступа. Система устанавливает эти токены в cookies браузера, одновременно удаляя временные cookies, и перенаправляет пользователя в личный кабинет с сообщением об успешной авторизации.

Представленная диаграмма последовательности демонстрирует реализацию безопасного процесса аутентификации с дополнительным уровнем защиты через подтверждение электронной почты. Такой подход обеспечивает баланс между безопасностью и удобством пользователя, предотвращая несанкционированный доступ к учетным записям даже в случае компрометации основных учетных данных. Временные механизмы хранения состояния между этапами аутентификации позволяют сохранять пользовательский опыт без ущерба для безопасности системы.

Архитектура взаимодействия компонентов, показанная на диаграмме, обеспечивает четкое разделение ответственности между различными модулями системы. Каждый компонент выполняет строго определенный набор функций, что способствует поддержке и масштабируемости решения.

Процесс оптимизирован для работы в веб-среде, где безопасность передачи и хранения учетных данных имеет критическое значение, особенно в контексте финансовых операций, связанных с покупкой и продажей бизнеса.

Для наглядного представления того, как происходит процесс авторизации пользователя в системе, используется диаграмма состояний (state chart diagram) в нотации UML, изображенная на рисунке 3.8 [21].

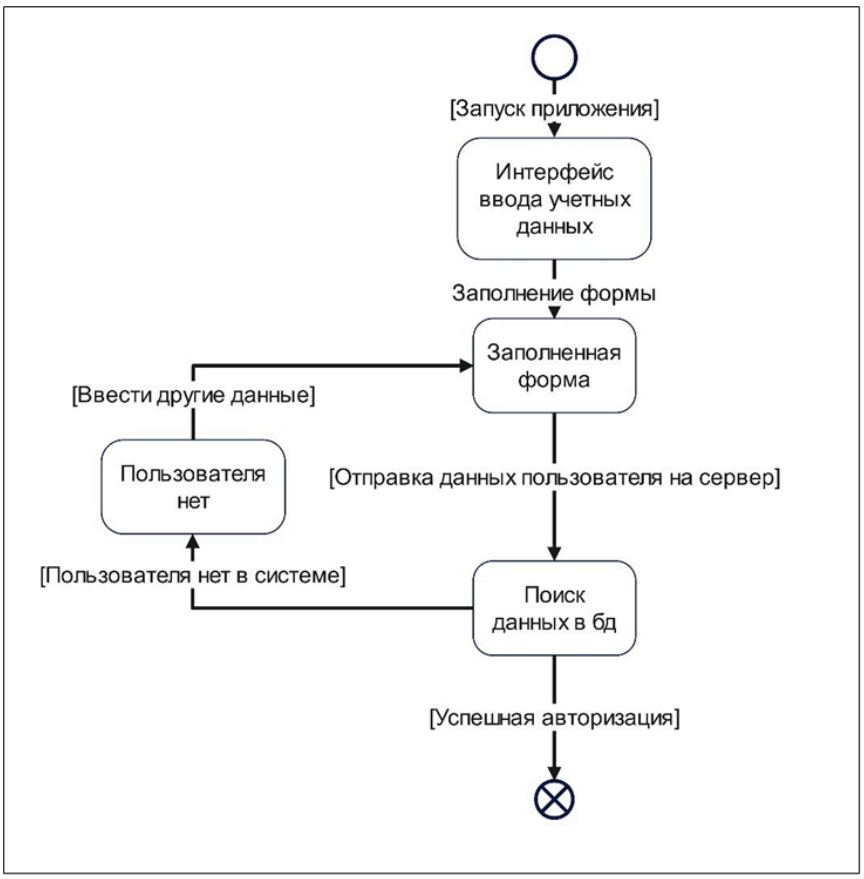


Рисунок 3.8 – Диаграмма состояний процесса авторизации

Данная диаграмма состояний в нотации UML наглядно отображает процесс авторизации пользователя в системе. Процесс начинается с состояния «Запуск приложения», откуда система немедленно переходит к состоянию «Интерфейс ввода учетных данных», где пользователю предоставляется форма для аутентификации.

При событии «Заполнение формы» система переходит в состояние «Заполненная форма», что означает этап готовности данных к отправке. Отсюда пользователь может вернуться к вводу других данных, что направляет систему обратно в интерфейс ввода, или подтвердить информацию, инициировав событие «Отправка данных пользователя на сервер». Это действие переводит систему в критическое состояние «Поиск данных в БД», в ходе которого проверяются учетные данные.

Результат этой проверки определяет дальнейший путь системы. Если пользователь не найден, система переходит в состояние «Пользователя нет в

системе», после чего процесс автоматически возвращается к интерфейсу ввода для повторной попытки. В случае успешного подтверждения учетных данных система завершает свой workflow переходом в конечное состояние «Успешная авторизация», предоставляя пользователю доступ к защищенным функциям системы. Таким образом, диаграмма четко демонстрирует не только линейный путь успешного входа, но и циклическую обработку ошибок, обеспечивая отказоустойчивость и удобство взаимодействия.

На рисунке 3.9 представлена диаграмма деятельности процесса «Создание совместного счета», которая отражает логику взаимодействия пользователя с системой при организации коллективного управления бюджетом. Данная схема демонстрирует пошаговый алгоритм от инициации процесса до активации общего финансового пространства [22].

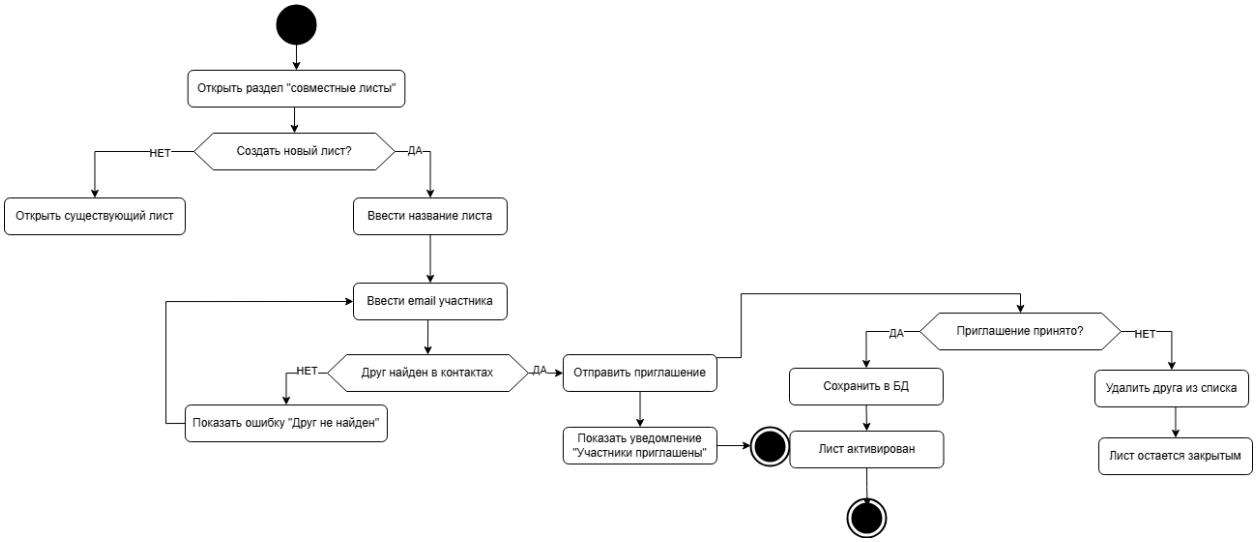


Рисунок 3.9 – Диаграмма деятельности процесса совместное управление бюджетом

Процесс начинается с действия «Открыть раздел «совместные листы». На этом этапе система переходит в состояние выбора, где пользователю предлагается либо создать новый финансовый инструмент, либо продолжить работу с уже существующим.

Далее видим ветвление «Создать новый лист?» Если пользователь выбирает работу с имеющимся счетом, активируется действие «Открыть существующий лист», и процесс завершается. В случае принятия решения о создании нового счета, система переходит к этапу «Ввести название листа».

Далее добавление участников. После задания идентификатора счета пользователю необходимо выполнить действие «Ввести email участника» для формирования круга доверенных лиц.

Затем система выполняет действие «Проверка наличия друга в контактах», которое является ключевым этапом валидации. Результат этой проверки определяет дальнейшее направление потока.

Если друг не найден, то процесс переходит к действию «Показать ошибку «Друг не найден», после чего возвращается на этап повторного ввода email. Данный итеративный цикл продолжается до тех пор, пока не будет введен корректный адрес.

Если друг найден, система выполняет действие «Отправить приглашение», отображает уведомление «Участники приглашены» и переходит в состояние ожидания обратной связи от приглашенного пользователя.

Заключительный этап процесса зависит от решения приглашенной стороны – «Приглашение принято?». При получении положительного ответа, система выполняет действие «Сохранить в БД», после чего происходит «Лист активирован». Поток выполнения достигает узла завершения (ActivityFinal), открывая доступ к совместному управлению транзакциями. В случае отказа, выполняется действие «Удалить друга из списка», и система фиксирует состояние «Лист остается закрытым», завершая процесс без активации совместного доступа.

Таким образом, представленная диаграмма деятельности визуализирует защищенный и последовательный подход к созданию совместных финансовых ресурсов, включающий обязательные этапы верификации участников и подтверждения доступа, что гарантирует конфиденциальность и целостность личных данных пользователей.

3.5 Описание организации бизнес-логики программного средств

Алгоритм представляет собой четкую последовательность шагов, определяющих процесс преобразования исходных данных в конечный результат. Применение алгоритмов позволяет стандартизировать выполнение задач, обеспечить предсказуемость работы системы и повысить ее общую надежность [23].

На рисунке 3.10 представлена блок-схема алгоритма, лежащего в основе работы серверной части веб-платформы для управления личными финансами. Данный алгоритм описывает основной цикл жизни сервера, обеспечивающий его непрерывную работу и взаимодействие с клиентами.

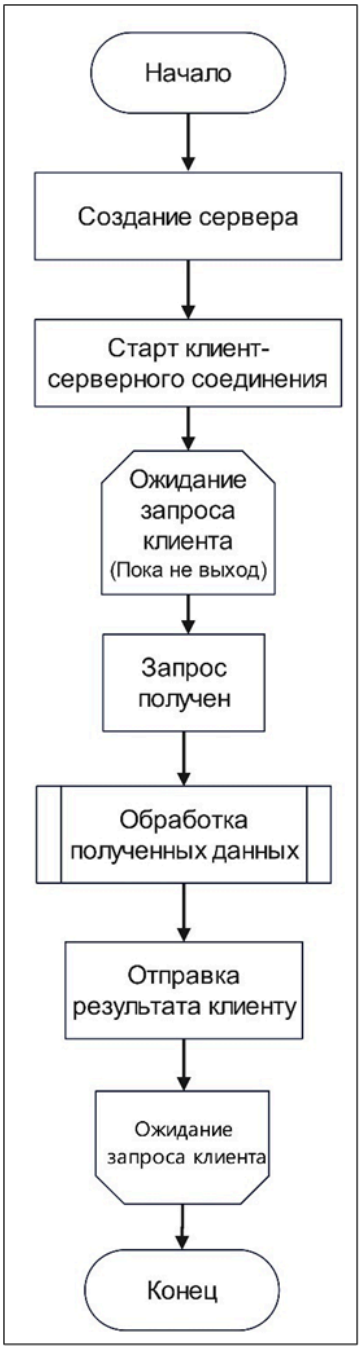


Рисунок 3.10 – Схема алгоритма работы серверной части приложения

Процесс начинается с инициализации и запуска сервера. После успешного старта система устанавливает клиент-серверное соединение, подготавливая канал для обмена данными. Затем сервер переходит в состояние постоянного ожидания входящих запросов от клиентских приложений. Этот цикл продолжается до тех пор, пока не будет получена команда на завершение работы.

Как только запрос от клиента поступает, сервер приступает к его обработке. На этом этапе выполняются операции, необходимые для формирования ответа: валидация данных, взаимодействие с базой данных, применение бизнес-логики. После обработки подготовленный результат

отправляется обратно клиенту. По завершении отправки сервер возвращается в состояние ожидания следующего запроса, обеспечивая тем самым непрерывность своего функционирования.

На рисунке 3.11 представлена схема алгоритма процесса «Регистрация пользователя», который является неотъемлемой частью функционала, доступного новому пользователю при взаимодействии с системой. Этот алгоритм тщательно описывает пошаговую последовательность валидации и создания новой учетной записи.

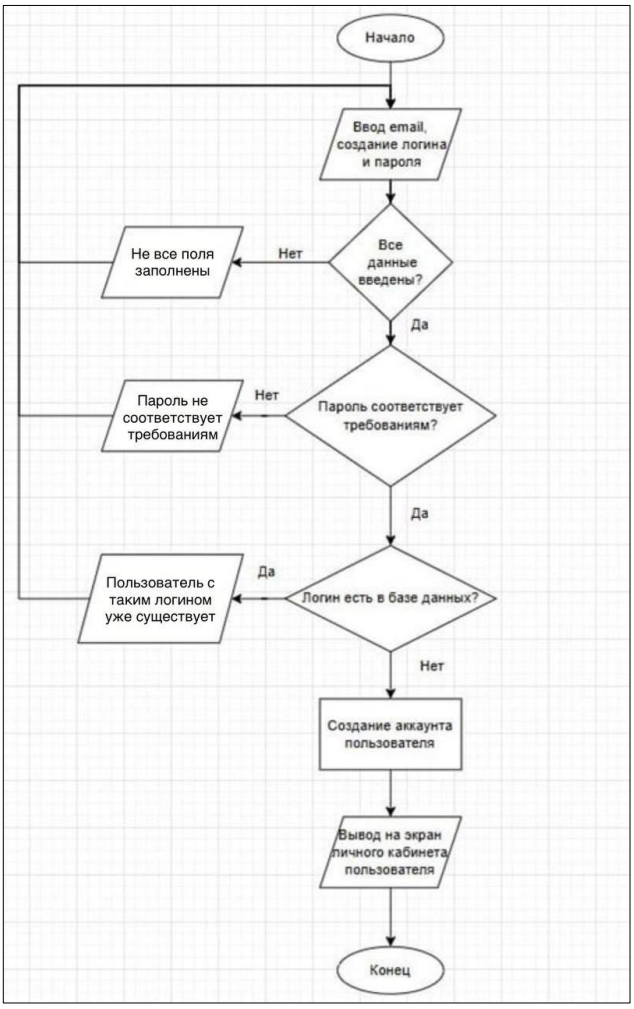


Рисунок 3.11 – Схема алгоритма процесса «Регистрация пользователя»

В начале процесса пользователь вводит необходимые для регистрации данные: адрес электронной почты и пароль. Система немедленно проверяет, все ли обязательные поля были заполнены. Если какое-либо поле осталось пустым, процесс прерывается, и пользователю предлагается завершить ввод.

Если все данные введены, система переходит к проверке сложности пароля на соответствие заданным требованиям безопасности (например, минимальная длина). В случае несоответствия пароля требованиям пользователь получает уведомление и должен создать новый пароль.

После успешной проверки пароля система запрашивает базу данных, чтобы убедиться, что пользователь с таким логином или email еще не зарегистрирован. Если логин уже существует, регистрация прерывается, и пользователю предлагается выбрать другой логин. Если логин уникален, система создает новый аккаунт в базе данных, после чего пользователь автоматически аутентифицируется и перенаправляется в свой личный кабинет, где может начать работу с платформой.

На рисунке 3.12 представлена схема алгоритма процесса «Авторизация пользователя», который является ключевым механизмом контроля доступа к персональным данным и функциям платформы управления личными финансами. Данный алгоритм описывает строгую последовательность действий по проверке подлинности пользователя и установлению защищенной сессии.

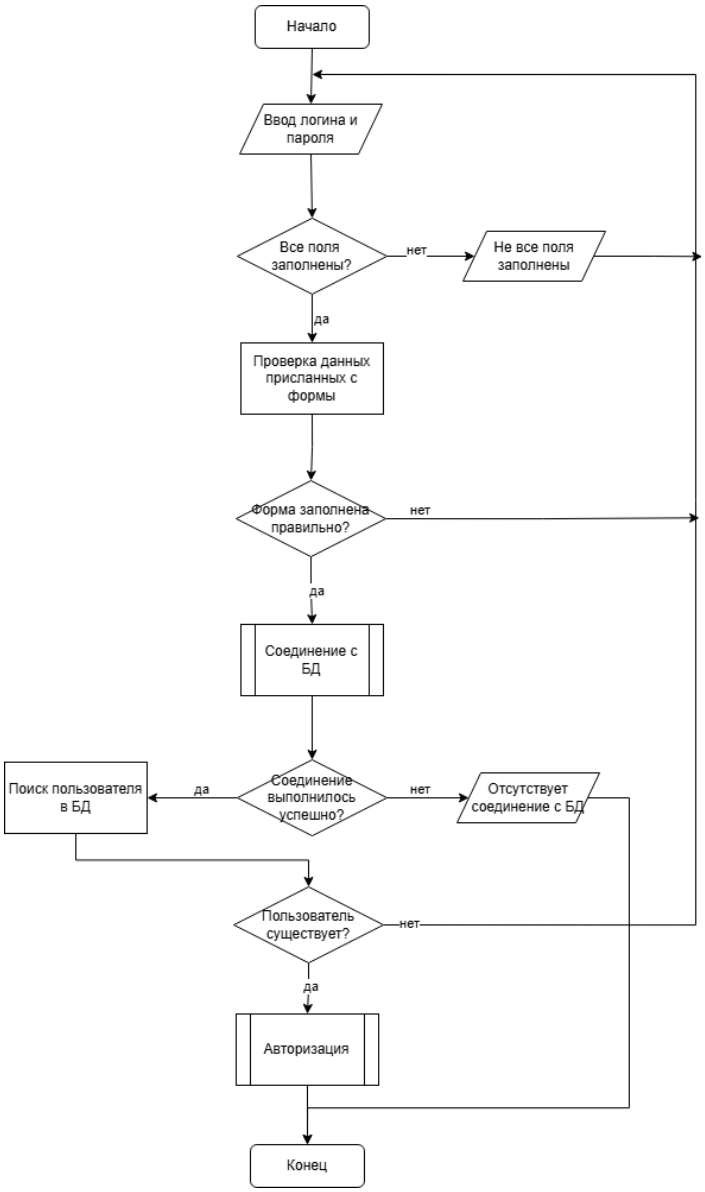


Рисунок 3.12 – Схема алгоритма процесса «Авторизация пользователя»

Процесс берет свое начало с формы ввода, где пользователь указывает свои учетные данные: логин (email) и пароль. Первичный этап алгоритма заключается в проверке заполнения всех обязательных полей. Если обнаружены пустые значения, система прерывает выполнение и возвращает пользователя к этапу ввода данных.

После успешной первичной валидации формы система переходит к этапу взаимодействия с базой данных. Соединение с БД – алгоритм инициирует запрос к хранилищу данных accounts. В случае возникновения технических неполадок или отсутствия связи, пользователю выводится соответствующее системное уведомление, и процесс завершается.

Далее происходит поиск пользователя. При наличии стабильного соединения система выполняет поиск записи, соответствующей введенному логину. Если учетная запись не найдена, алгоритм идентифицирует попытку входа как неудачную и завершает процесс.

Затем верификация данных. В случае нахождения профиля система приступает к проверке пароля. На данном этапе хешированное значение из БД сопоставляется с введенным паролем с помощью алгоритма BCrypt.

Заключительный этап алгоритма – непосредственная авторизация. Если все проверки пройдены успешно, сервер генерирует пару токенов и устанавливает соответствующие HTTP-записи (Cookies) в браузере клиента. После этого пользователь получает статус аутентифицированного субъекта и перенаправляется в личный кабинет для работы с финансовой аналитикой.

Такая структура алгоритма гарантирует, что доступ к управлению бюджетом получат только легитимные пользователи, обеспечивая высокий уровень безопасности системы.

Таким образом, представленные алгоритмы формируют логическую основу функционирования платформы для управления личными финансами. Алгоритм работы сервера обеспечивает стабильное взаимодействие между клиентом и серверной частью, гарантируя бесперебойную обработку запросов. Алгоритмы регистрации и авторизации отвечают за безопасное создание учетных записей и проверку подлинности пользователей с использованием современных механизмов защиты. Совокупность этих алгоритмов создает надежный фундамент для автоматизации ключевых процессов учета доходов, расходов и планирования бюджета, обеспечивая высокую точность и безопасность финансовых данных.

3.6 Спецификация к разработанному API и механизмы обеспечения информационной безопасности

Обеспечение информационной безопасности является критически важным аспектом при разработке системы управления личными финансами, поскольку приложение обрабатывает конфиденциальные данные пользователей. В рамках разработанного веб-приложения защита информации

реализуется на нескольких уровнях архитектуры – от организации взаимодействия клиентской и серверной частей до механизмов аутентификации, авторизации и управления доступом.

Архитектурной основой системы является REST API, обеспечивающий четкое разделение клиентской и серверной логики. Взаимодействие осуществляется по протоколу HTTP/HTTPS с использованием формата JSON. Дополнительно в системе реализовано версионирование API, позволяющее поддерживать обратную совместимость и обеспечивать эволюционное развитие сервиса без нарушения работы клиентских приложений. Версия API указывается в URI, что дает возможность параллельного существования нескольких версий интерфейса.

Серверная часть программного средства реализована на платформе Spring Boot с использованием Spring Security. Важную роль в обеспечении безопасности играют фильтры безопасности, которые перехватывают входящие HTTP-запросы и выполняют их предварительную обработку. Это позволяет на раннем этапе проверять корректность данных, наличие токенов доступа и права пользователя, предотвращая попадание некорректных или вредоносных запросов в бизнес-логику финансового приложения.

Ключевым элементом защиты учетных данных является безопасное хранение паролей. В системе реализован механизм хеширования паролей с использованием алгоритма BCrypt. Каждый пароль перед сохранением в таблице accounts преобразуется в хеш с добавлением уникальной соли, что исключает возможность восстановления исходного значения и существенно повышает устойчивость к атакам методом перебора. Дополнительно внедрена проверка сложности пароля, включающая требования к длине.

Для управления аутентификацией и авторизацией пользователей используется комбинированный подход, основанный на JSON Web Token (JWT) и OAuth 2.0. Основным механизмом является использование access-токенов, которые выдаются пользователю после успешной аутентификации и используются для доступа к защищенным ресурсам, таким как транзакции или цели. Access-токен имеет ограниченный срок действия, что снижает риск его компрометации.

Дополнительно реализована возможность аутентификации через сторонний сервис с использованием протокола OAuth 2.0, в частности, поддерживается вход через Google-аккаунт. В данном случае пользователь проходит аутентификацию на стороне провайдера, после чего сервер получает подтвержденные данные через GoogleStrategy и связывает их с профилем пользователя (Profile). Это повышает удобство регистрации и снижает риски, связанные с хранением паролей.

Реализованы механизмы повышения устойчивости к компрометации токенов. Применяется стратегия ротации refresh-токенов, при которой при каждом обновлении сессии пользователю выдается новый токен, а старый в базе данных refresh_tokens становится недействительным. Это минимизирует риск повторного использования перехваченных данных и повышает общую

безопасность сессий. Также предусмотрен механизм принудительного завершения сессии (logout) с удалением токенов из системы.

Контроль доступа к ресурсам реализован на основе ролевой модели (RBAC). Пользователям в системе назначаются роли, определяющие доступные им операции. Проверка прав осуществляется с помощью RolesGuard как на уровне HTTP-запросов, так и на уровне методов бизнес-логики с использованием аннотаций, что обеспечивает надежное разграничение доступа к персональной финансовой аналитике.

Для защиты от распространенных атак в системе реализованы механизмы валидации входных данных через объекты передачи данных (DTO). Все параметры, поступающие от клиента, проходят проверку на соответствие заданным ограничениям, что позволяет предотвратить SQL-инъекции и некорректные запросы. Передача данных между клиентом и сервером осуществляется по защищенному протоколу HTTPS, обеспечивающему их шифрование.

Таким образом, реализованная система безопасности, включающая версионирование API, использование JWT (access и refresh токенов), хранение refresh-токенов в базе данных, а также интеграцию с OAuth 2.0, обеспечивает высокий уровень защиты программного средства. Примененные подходы гарантируют конфиденциальность финансовых данных пользователей, целостность информации и устойчивость системы к несанкционированному доступу.

4 ТЕСТИРОВАНИЕ И ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ К ПРОГРАММНОМУ СРЕДСТВУ

4.1 Функциональное тестирование программного средства

Функциональное тестирование является важным этапом разработки программного средства и направлено на проверку соответствия реализованного функционала установленным требованиям. Основной целью функционального тестирования является подтверждение корректности работы системы в рамках предусмотренных пользовательских сценариев.

В рамках данного курсового проекта функциональное тестирование проводилось для платформы управления личными финансами с модулем категоризации расходов и планирования бюджета. Тестирование охватывало ключевые варианты использования системы, включая регистрацию и аутентификацию пользователей, управление транзакциями и категориями, планирование бюджета и финансовых целей, работу с депозитами, аналитику, совместные листы, контакты и обучающие ресурсы.

Функциональное тестирование выполнялось на уровне серверной части приложения, реализованной с использованием фреймворка Spring Boot. Взаимодействие с системой осуществлялось посредством REST API с использованием протокола HTTP и формата данных JSON.

Для проведения тестирования использовались следующие инструменты и технологии:

- JUnit 5 – для организации и выполнения тестов;
- Spring Boot Test – для настройки тестового окружения;
- MockMvc – для имитации HTTP-запросов к REST API без запуска сервера;
- Mockito – для создания mock-объектов и изоляции тестируемых компонентов.

В процессе тестирования решались следующие задачи:

- проверка корректности обработки HTTP-запросов;
- валидация входных данных;
- проверка бизнес-логики приложения;
- контроль корректности HTTP-статусов ответов;
- проверка механизмов аутентификации и авторизации;
- проверка корректности мягкого удаления (soft delete) сущностей;
- проверка разграничения прав доступа между ролями USER и ADMIN.

В модуле аутентификации и авторизации проверялась корректность регистрации пользователей, выполнения входа в систему, генерации и обработки JWT-токенов, а также разграничение прав доступа. Особое внимание уделялось проверке отказа в доступе при выполнении запросов без действующего токена авторизации.

В модуле транзакций тестировалась корректность создания доходов и расходов с привязкой к категории, фильтрация по типу, категории, валюте и

периоду дат, а также мягкое удаление записей и проверка права доступа владельца к транзакции.

В модуле категорий проверялась возможность создания, обновления и удаления пользовательских категорий с фильтрацией по типу (доход/расход), а также корректность возврата HTTP-статусов при авторизованных и неавторизованных запросах.

В модуле планирования бюджета (плановых расходов) тестировалась логика создания бюджетных лимитов по категориям и периодам, корректность обновления и удаления плановых расходов.

В модуле финансовых целей проверялась корректность создания и обновления целей, логика пополнения цели (метод contribute) с проверкой автоматического перевода статуса в COMPLETED при достижении целевой суммы, а также публикация события GoalReachedEvent. Дополнительно проверялась защита от доступа к чужим целям.

В модуле депозитов тестировалась логика создания депозитов и копилок, пополнение копилки (тип PIGGY_BANK) и фильтрация по статусу и типу.

В модуле аналитики проверялась корректность формирования сводных данных по доходам, расходам и балансу за период, а также динамика показателей в разрезе по месяцам и по категориям.

В модуле совместных листов тестировалась логика создания совместного бюджетного листа, приглашения участников из списка контактов, добавления транзакций и целей в лист, пополнения общей цели, а также получения аналитики по листу. Проверялось ограничение прав: только создатель может завершить или удалить лист.

В модуле контактов проверялась корректность поиска пользователей по email, телефону и коду приглашения, отправки, принятия и отклонения запросов в контакты, а также удаления контакта.

В модуле обучающих финансовых ресурсов тестировалось разграничение прав: пользователи могут просматривать ресурсы и добавлять их в избранное, тогда как создание, обновление, удаление ресурсов и управление вложениями доступны только администратору.

В модуле профиля проверялась корректность получения и обновления данных профиля, а также настройки приватности и уведомлений.

Тестирование выполнялось в автоматизированном режиме. Каждый тест представлял собой отдельный сценарий взаимодействия с API и включал вызов метода сервиса или отправку HTTP-запроса и проверку результата.

В ходе тестирования проверялись следующие параметры:

– соответствие HTTP-статусов ожидаемым (200, 201, 204, 401, 403, 404);

– корректность структуры и содержания JSON-ответов;

– соответствие бизнес-логики заданным требованиям;

– корректная обработка исключительных ситуаций.

Особое внимание уделялось тестированию негативных сценариев, включая:

– обращение к защищённым эндпоинтам без JWT-токена;
– попытку доступа к чужой сущности (проверка метода assertOwnership);
– попытку пополнить цель сверх целевой суммы;
– поиск несуществующей записи, приводящий к исключению EntityNotFoundException.

По результатам проведённого функционального тестирования установлено, что разработанное программное средство корректно реализует основные пользовательские сценарии и соответствует предъявленным функциональным требованиям. Выявленные в процессе тестирования недостатки были устранены, что позволило повысить надёжность и стабильность работы системы.

Таким образом, функциональное тестирование подтвердило работоспособность разработанной платформы управления личными финансами и её готовность к дальнейшей эксплуатации.

4.2 Модульное и интеграционное тестирование бизнес-логики программного средства

Модульное тестирование сервисного и фасадного слоёв выполнялось с применением аннотации @ExtendWith(MockitoExtension.class). Все зависимости тестируемых компонентов (репозитории, сервисы, маппер) заменялись mock-объектами посредством аннотаций @Mock и @InjectMocks. Изоляция тестов обеспечивается автоматическим сбросом состояния mock-объектов между тестами, что исключает взаимное влияние тестовых сценариев друг на друга. Для тестирования фасадного слоя, использующего статический метод SecurityUtils.currentAccountId(), применялся механизм MockedStatic с явной инициализацией в методе @BeforeEach и освобождением в методе @AfterEach.

Тест-кейсы модульного тестирования сервиса категорий (CategoryService) представлены в таблице 4.1.

Таблица 4.1 – «Тест-кейсы модульного тестирования CategoryService»

Идентификатор теста	Тестируемый метод	Входные данные / условие	Ожидаемый результат
TC-CS-01	findById()	Категория с указанным ID существует	Возвращается найденная сущность Category
TC-CS-02	findById()	Категория с указанным ID отсутствует	Выброс EntityNotFoundException
TC-CS-03	create()	Категория с таким именем и типом у профиля не существует	Категория сохраняется в репозитории, возвращается сохранённая сущность

Продолжение таблицы 4.1

Идентификатор теста	Тестируемый метод	Входные данные / условие	Ожидаемый результат
TC-CS-04	create()	Категория с таким именем и типом у профиля уже существует	Выброс DuplicateCategoryException; вызов save не выполняется
TC-CS-05	update()	Переданы новое имя и новый цвет	Поля name и color обновляются, вызов save выполняется
TC-CS-06	delete()	Существующая категория	Поле deleted устанавливается в true, вызов save выполняется
TC-CS-07	assertOwnership()	ID профиля совпадает с владельцем категории	Исключение не выбрасывается
TC-CS-08	assertOwnership()	ID профиля не совпадает с владельцем категории	Выброс CategoryAccessDeniedException

Тест-кейсы TC-CS-01 и TC-CS-02 проверяют корректность поиска категории по идентификатору, включая обработку отсутствующей записи. TC-CS-03 и TC-CS-04 покрывают логику создания категории: подтверждается сохранение при уникальном имени и отклонение дубликата без обращения к репозиторию. TC-CS-05 верифицирует обновление полей сущности. TC-CS-06 проверяет механизм мягкого удаления. TC-CS-07 и TC-CS-08 тестируют защиту от несанкционированного доступа к чужим категориям.

Тест-кейсы модульного тестирования фасада категорий (CategoryFacade) представлены в таблице 4.2.

Таблица 4.2 – «Тест-кейсы модульного тестирования CategoryFacade»

Идентификатор теста	Тестируемый метод	Входные данные / условие	Ожидаемый результат
TC-CF-01	getMyCategories()	Авторизованный пользователь, в базе одна категория	Возвращается страница с одним DTO, маппинг выполнен корректно
TC-CF-02	create()	Корректный запрос CreateCategoryRequest	Делегирование в CategoryService выполнено, возвращается CategoryResponse
TC-CF-03	update()	Передано новое имя, цвет не передан (null)	Имя обновляется переданным значением, вызов update выполнен
TC-CF-04	update()	Имя не передано (null), передан новый цвет	Имя берётся из существующей сущности, вызов update выполнен с прежним именем

Продолжение таблицы 4.2

Идентификатор теста	Тестируемый метод	Входные данные / условие	Ожидаемый результат
TC-CF-05	delete()	Корректный ID категории, принадлежащей текущему пользователю	Проверка владельца выполнена, вызов delete делегирован в CategoryService

TC-CF-03 и TC-CF-04 совместно проверяют логику частичного обновления (PATCH): при отсутствии какого-либо поля в запросе фасад подставляет текущее значение сущности, не допуская затирания данных. TC-CF-05 подтверждает, что перед удалением обязательно выполняется проверка прав владельца.

Тест-кейсы модульного тестирования сервиса транзакций (TransactionService) представлены в таблице 4.3.

Таблица 4.3 – «Тест-кейсы модульного тестирования TransactionService»

Идентификатор теста	Тестируемый метод	Входные данные / условие	Ожидаемый результат
TC-TS-01	findById()	Транзакция с указанным ID существует	Возвращается найденная сущность Transaction
TC-TS-02	findById()	Транзакция с указанным ID отсутствует	Выброс EntityNotFoundException
TC-TS-03	save()	Корректная сущность Transaction	Делегирование в репозиторий, возвращается сохранённая сущность
TC-TS-04	delete()	Существующая транзакция	Поле deleted устанавливается в true, вызов save выполняется
TC-TS-05	findByProfile()	Корректные параметры фильтрации и пагинации	Возвращается страница транзакций из репозитория
TC-TS-06	assertOwnership()	ID профиля совпадает с владельцем транзакции	Исключение не выбрасывается
TC-TS-07	assertOwnership()	ID профиля не совпадает с владельцем транзакции	Выброс TransactionAccessDeniedException

Тест-кейсы модульного тестирования сервиса финансовых целей (GoalService) представлены в таблице 4.4.

Таблица 4.4 – «Тест-кейсы модульного тестирования GoalService»

Идентификатор теста	Тестируемый метод	Входные данные / условие	Ожидаемый результат
TC-GS-01	findById()	Цель с указанным ID существует	Возвращается найденная сущность Goal
TC-GS-02	findById()	Цель с указанным ID отсутствует	Выброс EntityNotFoundException
TC-GS-03	save()	Корректная сущность Goal	Делегирование в репозиторий, возвращается сохранённая сущность
TC-GS-04	delete()	Существующая цель	Поле deleted устанавливается в true, вызов save выполняется
TC-GS-05	contribute()	Сумма пополнения меньше остатка до целевой суммы	Поле currentAmount увеличивается, статус остаётся ACTIVE, событие GoalReachedEvent не публикуется
TC-GS-06	contribute()	Сумма пополнения в точности достигает целевой суммы	Статус переводится в COMPLETED, публикуется событие GoalReachedEvent
TC-GS-07	contribute()	Сумма пополнения превышает целевую сумму	Статус переводится в COMPLETED, публикуется событие GoalReachedEvent
TC-GS-08	assertOwnership()	ID профиля совпадает с владельцем цели	Исключение не выбрасывается
TC-GS-09	assertOwnership()	ID профиля не совпадает с владельцем цели	Выброс GoalAccessDeniedException

TC-GS-05 – TC-GS-07 совместно покрывают все граничные сценарии логики пополнения цели: пополнение ниже лимита, точное достижение и превышение целевой суммы. Особую значимость представляет TC-GS-06: проверяется не только корректное изменение статуса, но и публикация доменного события GoalReachedEvent с захватом аргумента через ArgumentCaptor, что подтверждает передачу в событие именно той сущности, которая была изменена.

Интеграционное тестирование контроллера категорий (CategoryController) выполнялось с использованием @WebMvcTest в связке с MockMvc. Фасадный слой изолировался посредством @MockBean. Для проверки безопасности использовалась аннотация @WithMockUser. Тест-кейсы представлены в таблице 4.5.

Таблица 4.5 – «Тест-кейсы интеграционного тестирования CategoryController»

Идентификатор теста	Эндпоинт	HTTP-метод / входные данные	Ожидаемый HTTP-статус и результат
TC-CC-01	GET /api/v1/categories	Авторизованный пользователь	200 OK; тело содержит страницу с полями name и type
TC-CC-02	GET /api/v1/categories	Запрос без авторизации	401 Unauthorized
TC-CC-03	POST /api/v1/categories	Авторизованный пользователь, корректный запрос (name: «Salary», type: INCOME)	201 Created; тело содержит поля id и name с ожидаемыми значениями
TC-CC-04	DELETE /api/v1/categories/{id}	Авторизованный пользователь, существующий ID	204 No Content; вызов facade.delete выполнен один раз

TC-CC-02 подтверждает, что все эндпоинты модуля категорий защищены и недоступны без действующего токена авторизации. TC-CC-04 дополнительно верифицирует, что контроллер корректно делегирует вызов в фасадный слой без лишних преобразований.

По результатам проведённого тестирования все тест-кейсы завершились успешно. Дополнительно реализованы тесты для сервисов TransactionService и GoalService, покрывающие ключевые сценарии управления транзакциями и финансовыми целями, включая граничные условия пополнения и механизм публикации доменных событий.

Таким образом, модульное и интеграционное тестирование подтвердило корректность реализации бизнес-логики системы: механизмы управления категориями, транзакциями и финансовыми целями функционируют в соответствии с установленными требованиями и корректно обрабатывают как штатные сценарии, так и исключительные ситуации.

4.3 Руководство по развертыванию программного средства

Для развёртывания программного средства на целевом устройстве необходимо последовательно установить и настроить все компоненты системы: базу данных PostgreSQL, серверную и клиентскую части приложения.

Перед началом развёртывания необходимо убедиться, что на устройстве установлено следующее программное обеспечение:

– Java Development Kit (JDK) версии 17 или выше;

– Node.js версии 18 или выше и пакетный менеджер npm;

– React 18, TypeScript 5 и сборщик Vite 5: устанавливаются автоматически как зависимости проекта при выполнении команды npm install;

– PostgreSQL версии 15 или выше;

– IntelliJ IDEA (Community или Ultimate Edition).

После установки всего вышеперечисленного необходимо перейти к настройке базы данных. Для этого следует загрузить и установить PostgreSQL с официального сайта. В процессе установки задаётся пароль для суперпользователя postgres и порт подключения – по умолчанию 5432. После завершения установки необходимо открыть pgAdmin или psql и создать базу данных для приложения, выполнив команду: CREATE DATABASE postgres. Схема базы данных создаётся автоматически при первом запуске серверной части посредством механизма миграций Flyway, поэтому выполнять SQL-скрипты вручную не требуется.

Для запуска серверной части приложения необходимо открыть папку с проектом серверной части в IntelliJ IDEA. Перед запуском следует настроить параметры подключения к базе данных и почтовому серверу в файле application.yml, расположенном в корне проекта. К

Параметры JWT-токенов, срок действия access-токена и refresh-токена, а также настройки файлового хранилища и уровней логирования задаются в том же файле конфигурации. При необходимости использования OAuth2-аутентификации через Google следует указать корректные значения client-id и client-secret в разделе spring.security.oauth2.

После сохранения конфигурации необходимо запустить главный класс приложения ServerApplication через кнопку Run в IntelliJ IDEA. При первом запуске Flyway автоматически применит все миграции и создаст схему базы данных. Серверная часть будет доступна по адресу http://localhost:8080. Документация REST API в формате Swagger UI доступна по адресу http://localhost:8080/swagger-ui.html, а спецификация OpenAPI – по адресу http://localhost:8080/api-docs.

Для запуска клиентской части приложения необходимо открыть папку с проектом клиентской части в терминале и установить зависимости командой: npm install. Данная команда автоматически установит все необходимые библиотеки, указанные в файле package.json, в том числе React 18, React Router, Axios, TanStack Query, Recharts, Zustand, Tailwind CSS и Zod.

Взаимодействие клиентской части с серверной настроено через механизм proxy в файле vite.config.ts: все запросы по пути /api, /login и /oauth2/authorization автоматически перенаправляются на http://localhost:8080, что исключает необходимость дополнительной настройки CORS при локальном развёртывании.

После установки зависимостей клиентская часть запускается командой: npm run dev. Приложение будет доступно в браузере по адресу http://localhost:5173.

После запуска всех компонентов системы необходимо открыть браузер и перейти по адресу http://localhost:5173. Главная страница приложения представлена на рисунке 4.1.

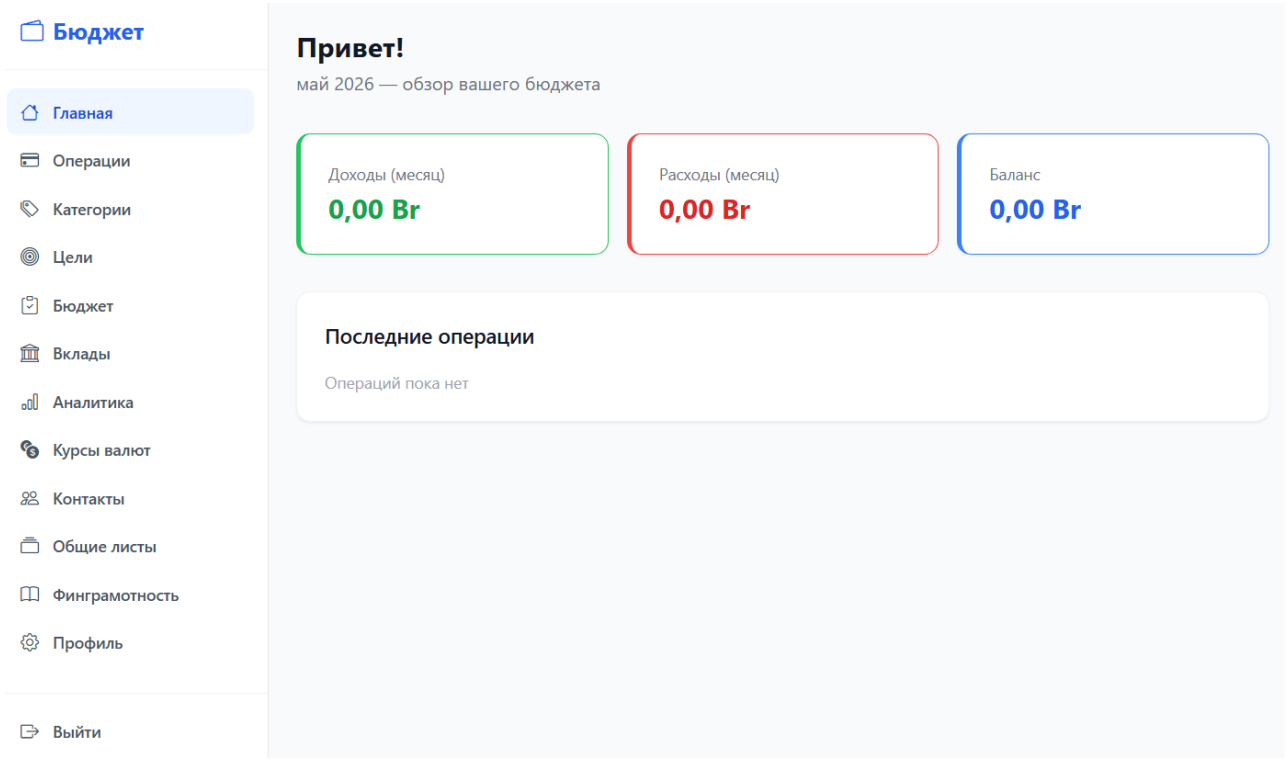


Рисунок 4.1 – Главная страница приложения

Корректная загрузка главной страницы приложения свидетельствует об успешном развёртывании программного средства. Взаимодействие клиентской части с серверной можно дополнительно проверить, открыв вкладку Network в инструментах разработчика браузера и убедившись, что запросы к API возвращают ожидаемые HTTP-статусы.

4.4 Руководство пользователя

После запуска приложения и перехода по адресу http://localhost:5173 пользователю открывается страница входа в систему, представленная на рисунке 4.2. Для входа в систему необходимо ввести адрес электронной почты и пароль, после чего нажать кнопку «Войти».

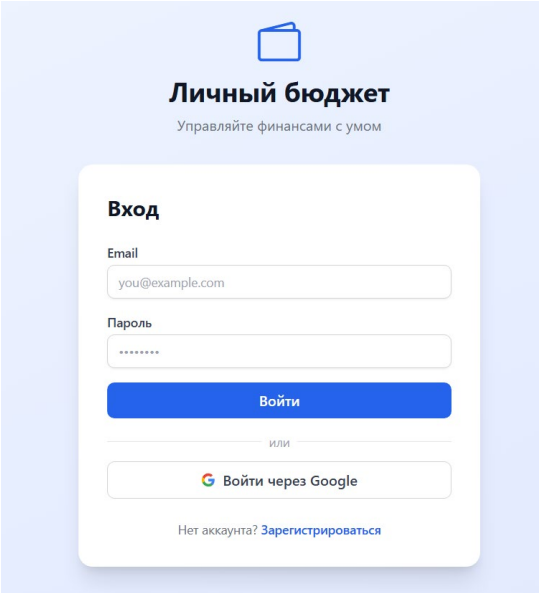


Рисунок 4.2 – Страница входа

Кроме того, предусмотрена возможность аутентификации через аккаунт Google – для этого следует нажать кнопку «Войти через Google», после чего откроется стандартная страница выбора аккаунта Google, представленная на рисунке 4.3.

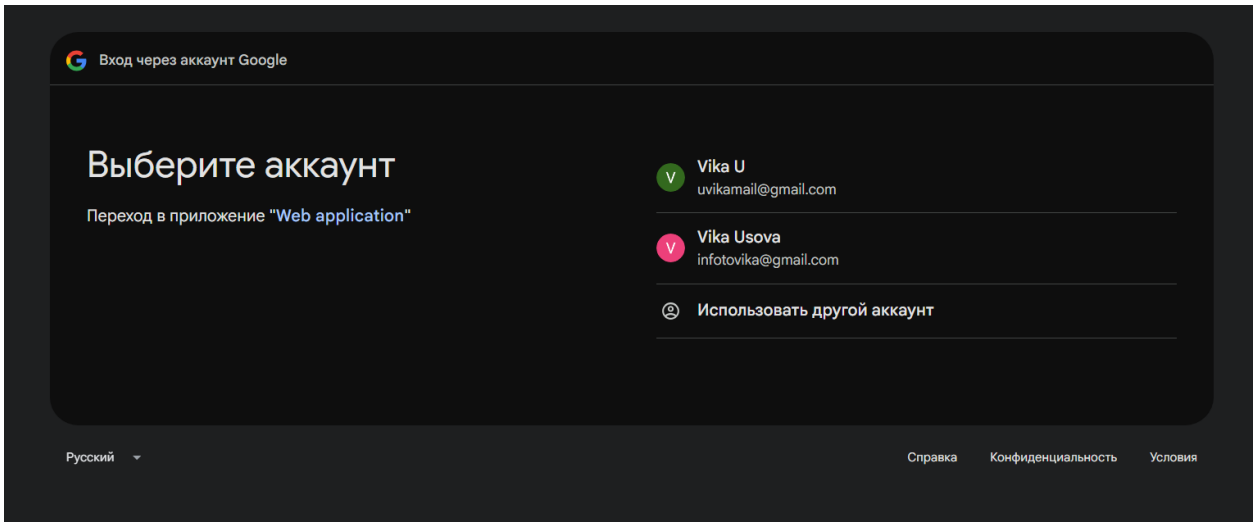


Рисунок 4.3 – Страница входа с помощью Google

Если у пользователя ещё нет аккаунта, необходимо перейти по ссылке «Зарегистрироваться». Страница регистрации представлена на рисунке 4.4.

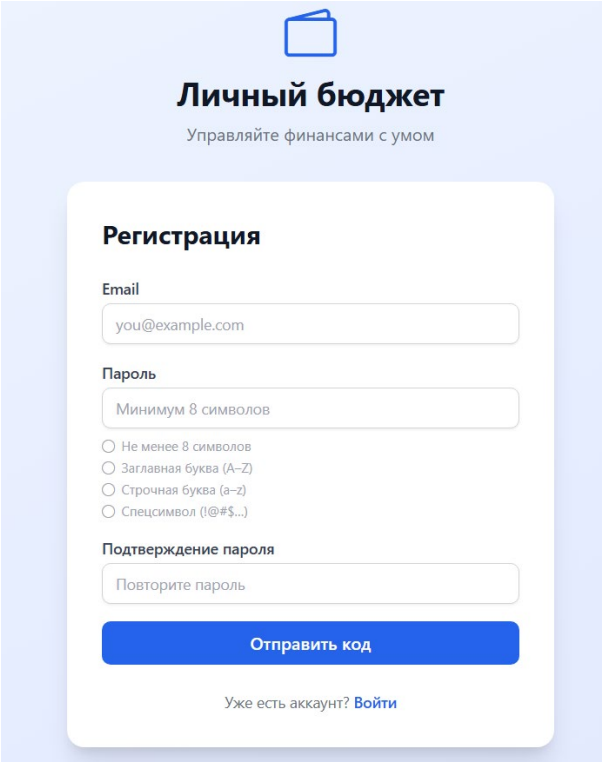


Рисунок 4.4 – Страница регистрации

Для создания аккаунта пользователю необходимо ввести адрес электронной почты, придумать пароль, соответствующий требованиям надёжности (не менее 8 символов, наличие заглавной буквы, строчной буквы и спецсимвола), и подтвердить его повторным вводом, после чего нажать кнопку «Отправить код». На указанный адрес электронной почты будет выслан код подтверждения для верификации аккаунта.

После успешного входа в систему пользователю становится доступна боковая навигационная панель, содержащая следующие разделы: Главная, Операции, Категории, Цели, Бюджет, Вклады, Аналитика, Курсы валют, Контакты, Общие листы, Финграмотность и Профиль.

Раздел «Категории» позволяет управлять пользовательскими категориями доходов и расходов. Список категорий можно фильтровать по типу, выбрав вкладку «Все», «Доходы» или «Расходы». Для создания новой категории необходимо нажать кнопку «Создать», после чего откроется модальное окно, представленное на рисунке 4.5. В форме создания указывается название категории, её тип (Доход или Расход), а также при желании – цвет для визуального выделения в интерфейсе.

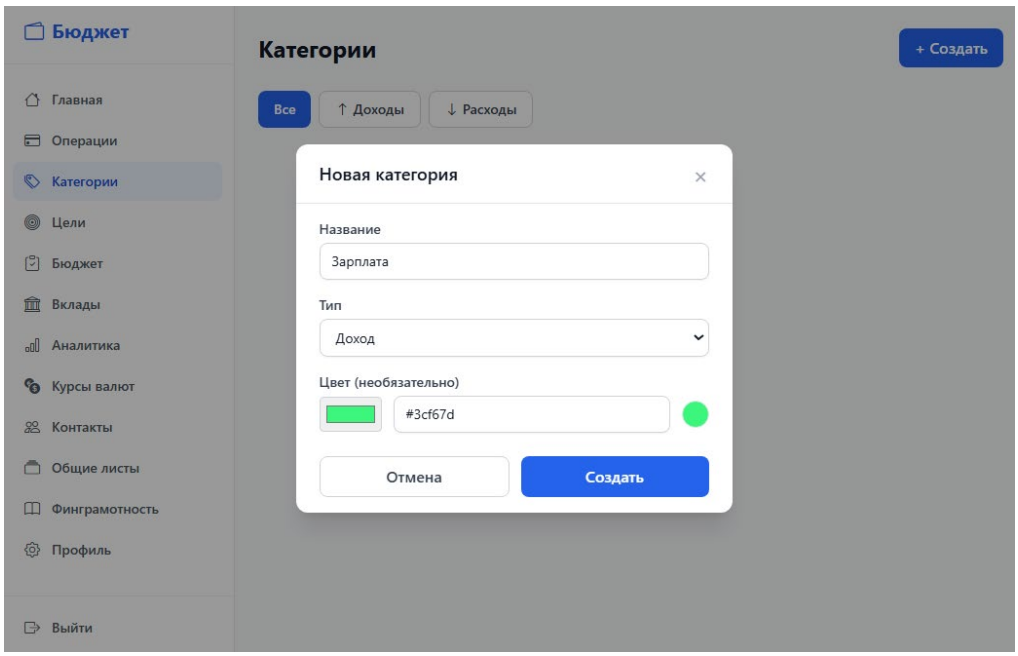


Рисунок 4.5 – Страница добавление категории

Раздел «Операции» предназначен для учёта финансовых транзакций пользователя. Список операций представлен на рисунке 4.6 и отображает для каждой записи дату, тип (Доход или Расход), категорию, описание, сумму в исходной валюте и эквивалент в белорусских рублях. Доступна фильтрация по типу операции, категории и периоду дат, а также сортировка по дате.

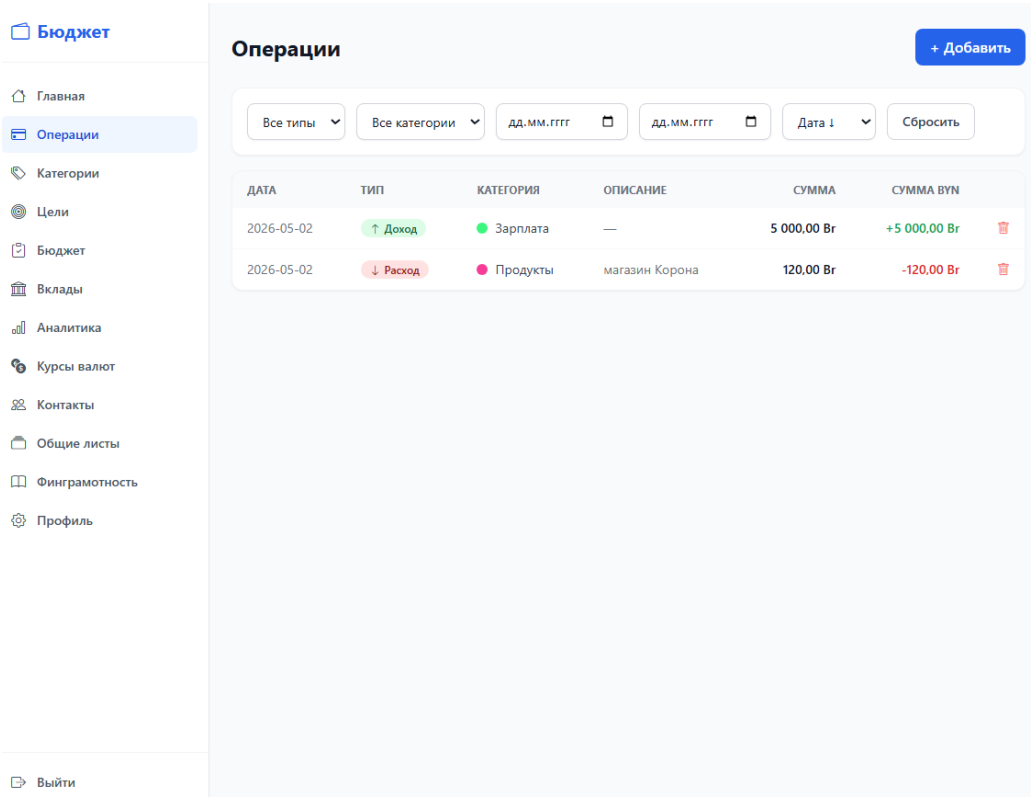


Рисунок 4.6 – Страница Операции

Для добавления новой операции необходимо нажать кнопку «Добавить», после чего откроется модальное окно, представленное на рисунке 4.7. В форме указывается тип операции, категория, сумма, валюта, дата и необязательное описание.

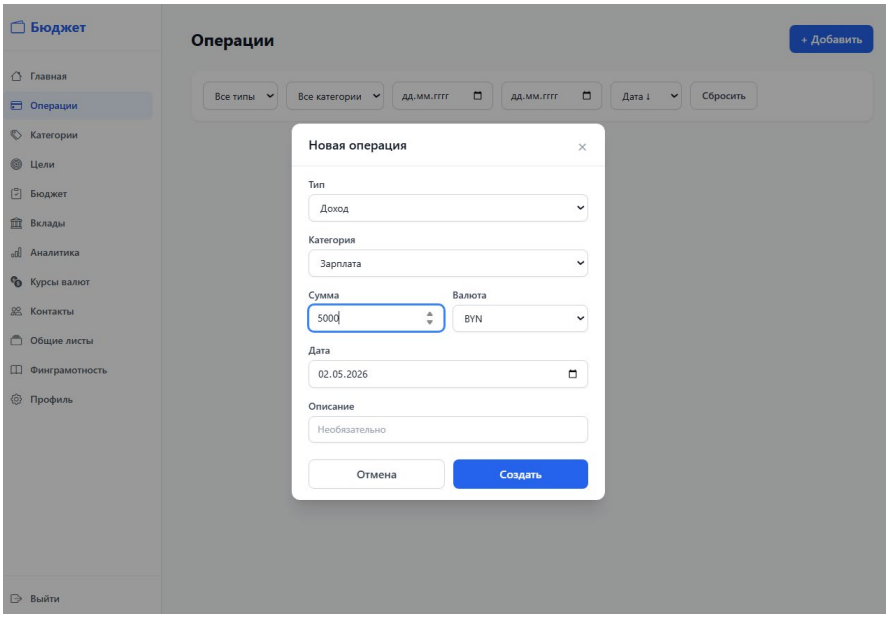


Рисунок 4.7 – Страница добавление операции

Раздел «Цели накопления» предназначен для постановки и отслеживания финансовых целей пользователя. Для создания новой цели необходимо нажать кнопку «Создать цель», после чего откроется модальное окно, представленное на рисунке 4.8.

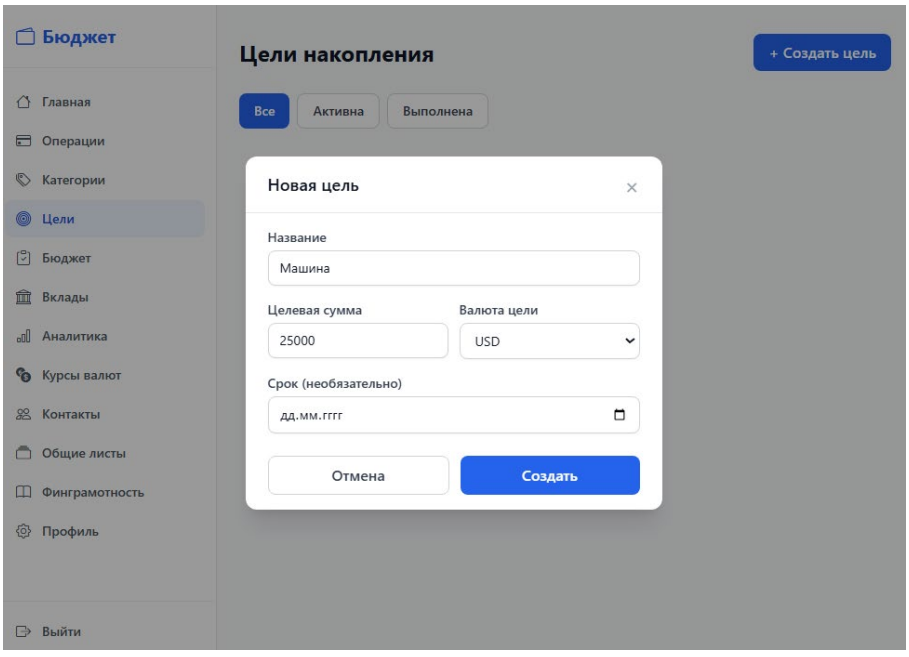


Рисунок 4.8 – Страница создание цели

В форме указывается название цели, целевая сумма, валюта и необязательный срок достижения. После создания цель отображается в списке в виде карточки с прогресс-баром, показывающим текущий процент накопления.

Для пополнения цели необходимо нажать кнопку «Пополнить» на карточке цели, после чего откроется модальное окно, представленное на рисунке 4.9.

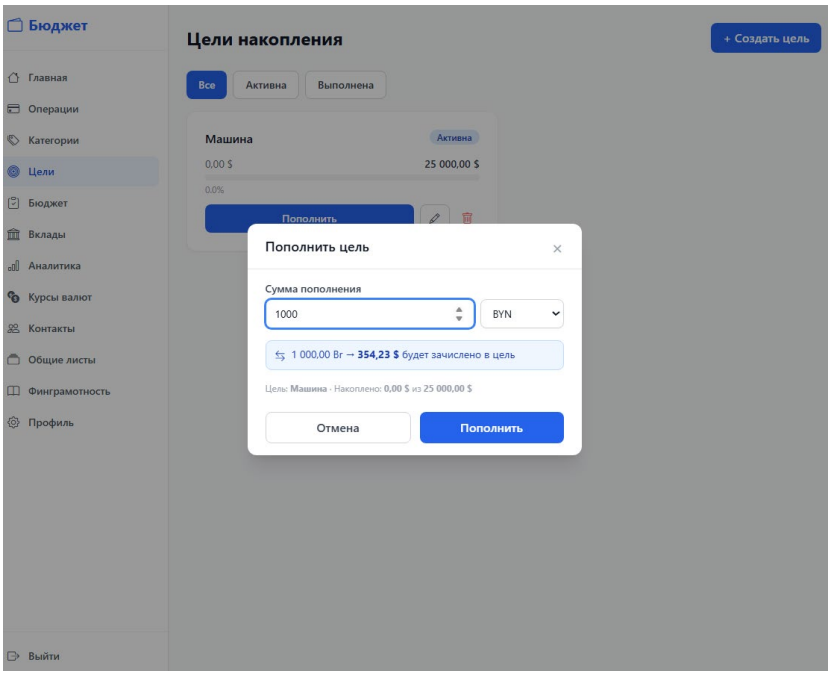


Рисунок 4.9 – Страница пополнение цели

В форме указывается сумма пополнения и валюта. Система автоматически рассчитывает конвертацию. При достижении целевой суммы статус цели автоматически меняется на «Выполнена», а пользователю приходит уведомление на электронную почту. Страница со списком целей, включающая активную и выполненную, представлена на рисунке 4.10.

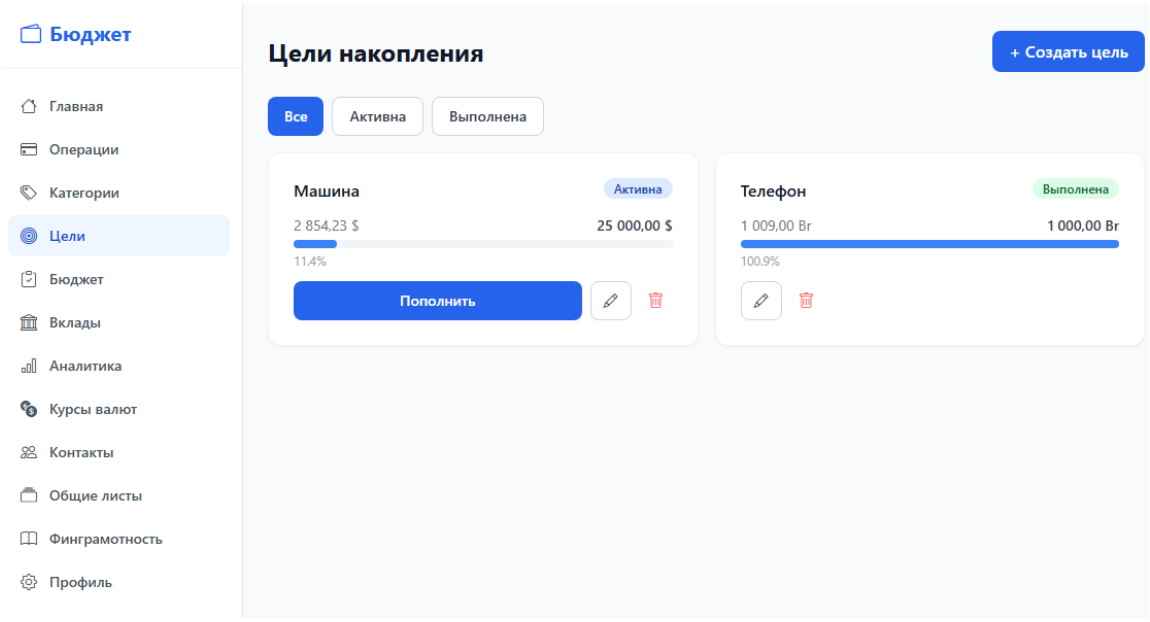


Рисунок 4.10 – Страница со списком целей

Пример письма-уведомления о достижении цели представлен на рисунке 4.11.

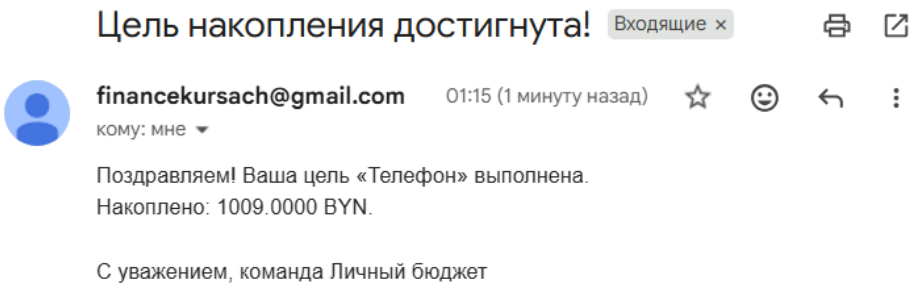


Рисунок 4.11 – Уведомление

Раздел «Бюджет и планы» предназначен для управления бюджетными лимитами по категориям расходов. Для создания новой записи необходимо нажать кнопку «Создать», после чего откроется модальное окно, представленное на рисунке 4.12. В форме указывается тип (лимит бюджета или плановый расход), название, категория, сумма, валюта и период (например, ежемесячно). При превышении установленного лимита система формирует уведомление на электронную почту пользователя.

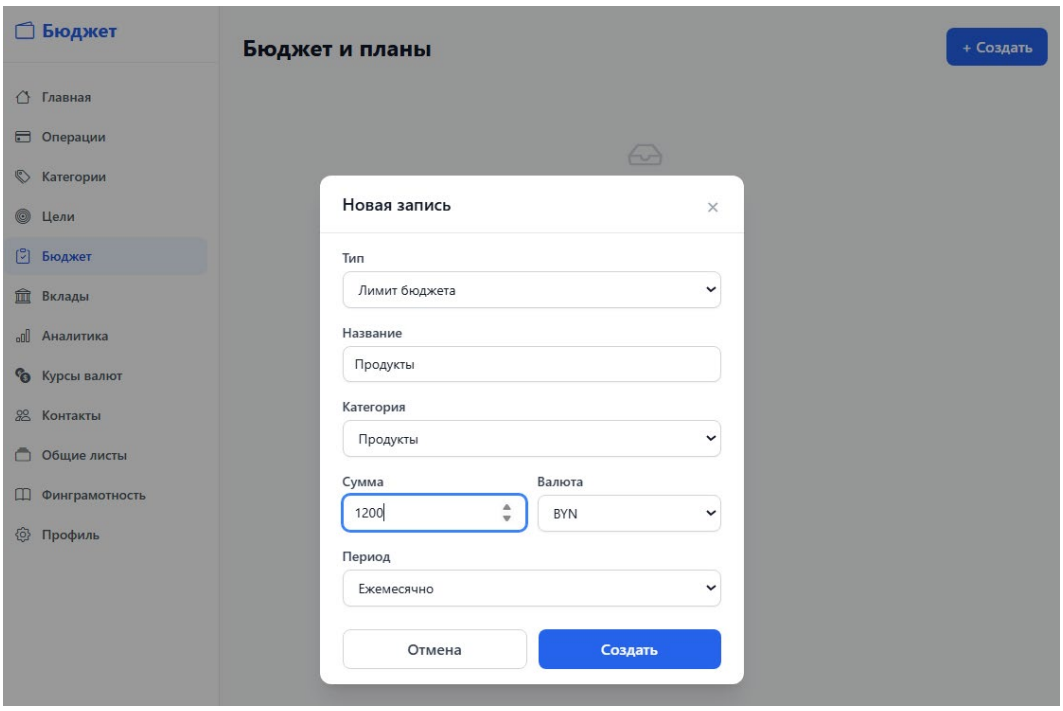


Рисунок 4.12 – Страница создание лимита

Раздел «Вклады и копилки» позволяет фиксировать банковские вклады и вести учёт накоплений. Для добавления нового вклада необходимо нажать кнопку «Создать» и заполнить форму, представленную на рисунке 4.13, указав тип (Вклад или Копилка), название, начальную сумму, валюту, дату открытия и закрытия, а также процентную ставку.

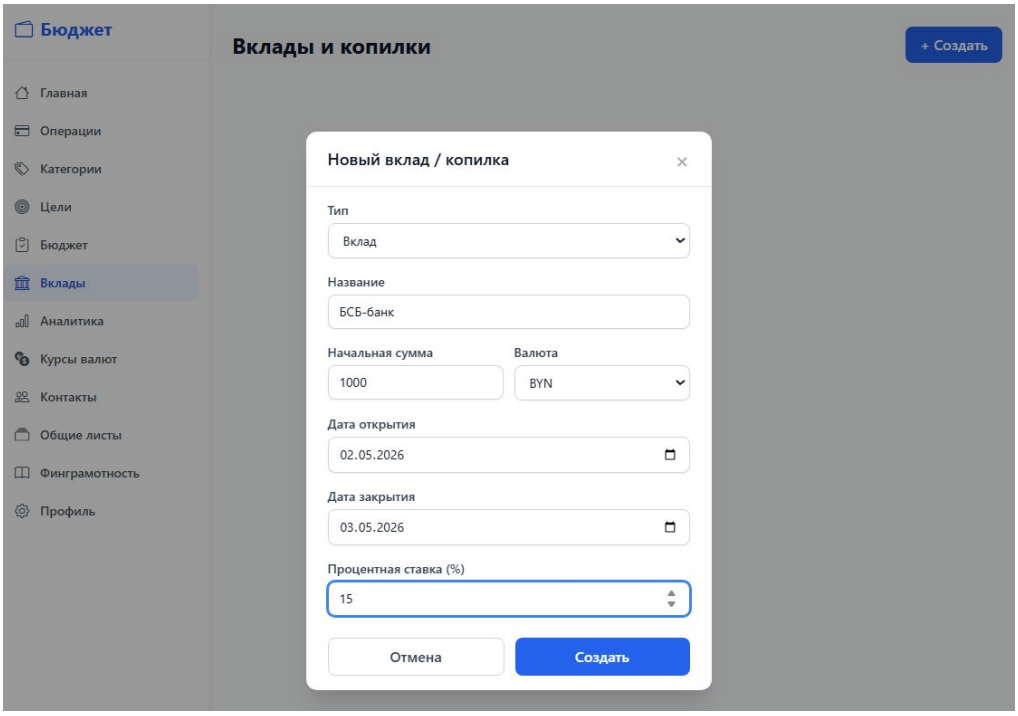


Рисунок 4.13 – Страница добавление вклада

После создания карточка вклада отображает текущую сумму, ставку, прогнозируемый доход и срок действия, как показано на рисунке 4.14.

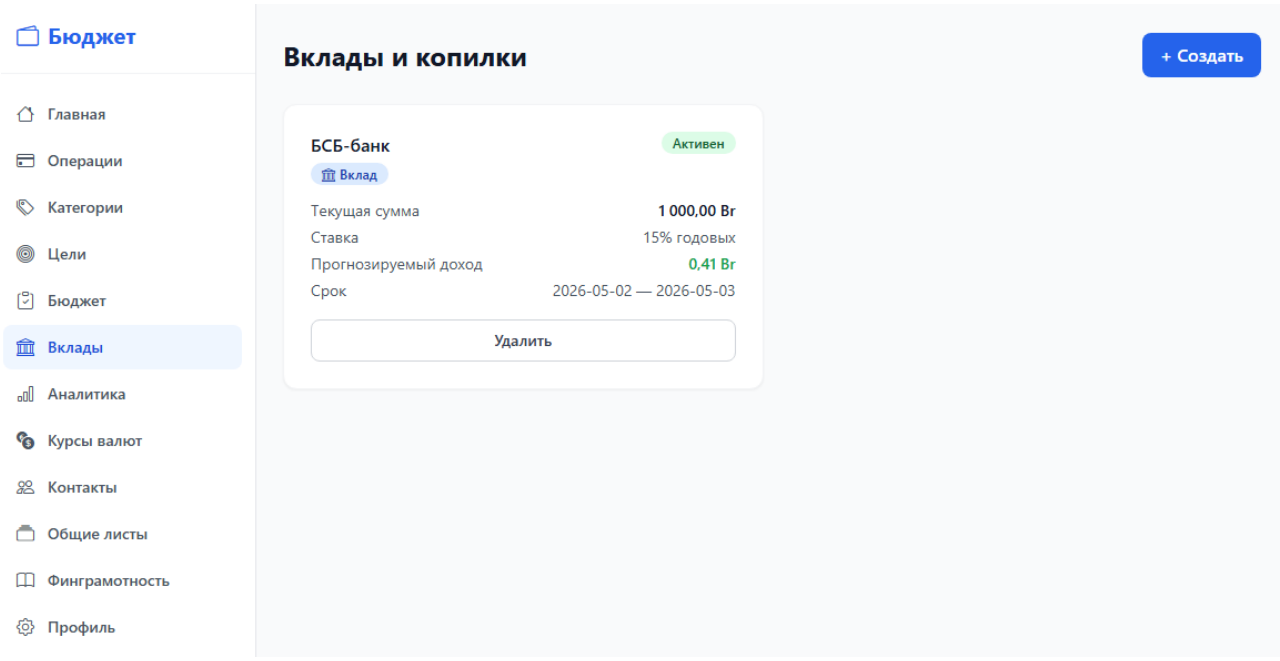


Рисунок 4.14 – Страница карточки вкладов

Раздел «Аналитика» предоставляет сводную информацию о финансовом состоянии пользователя за выбранный период, представленную на рисунке 4.15.

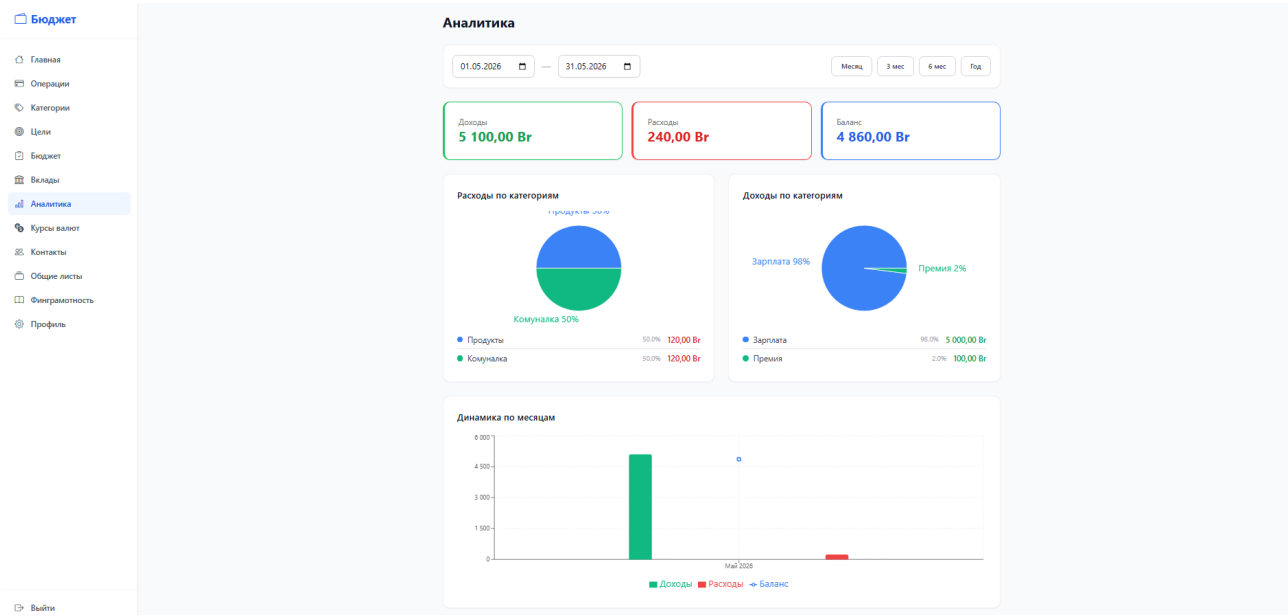


Рисунок 4.15 – Страница аналитики

В верхней части страницы отображаются три ключевых показателя: суммарные доходы, суммарные расходы и баланс. Ниже представлены круговые диаграммы расходов и доходов в разрезе по категориям с указанием доли и суммы каждой категории, а также столбчатый график динамики доходов, расходов и баланса по месяцам. Для удобства анализа предусмотрены быстрые кнопки выбора периода: месяц, 3 месяца, 6 месяцев, год.

Раздел «Курсы валют» отображает актуальные официальные курсы Национального банка Республики Беларусь по отношению к белорусскому рублю, представленные на рисунке 4.16.

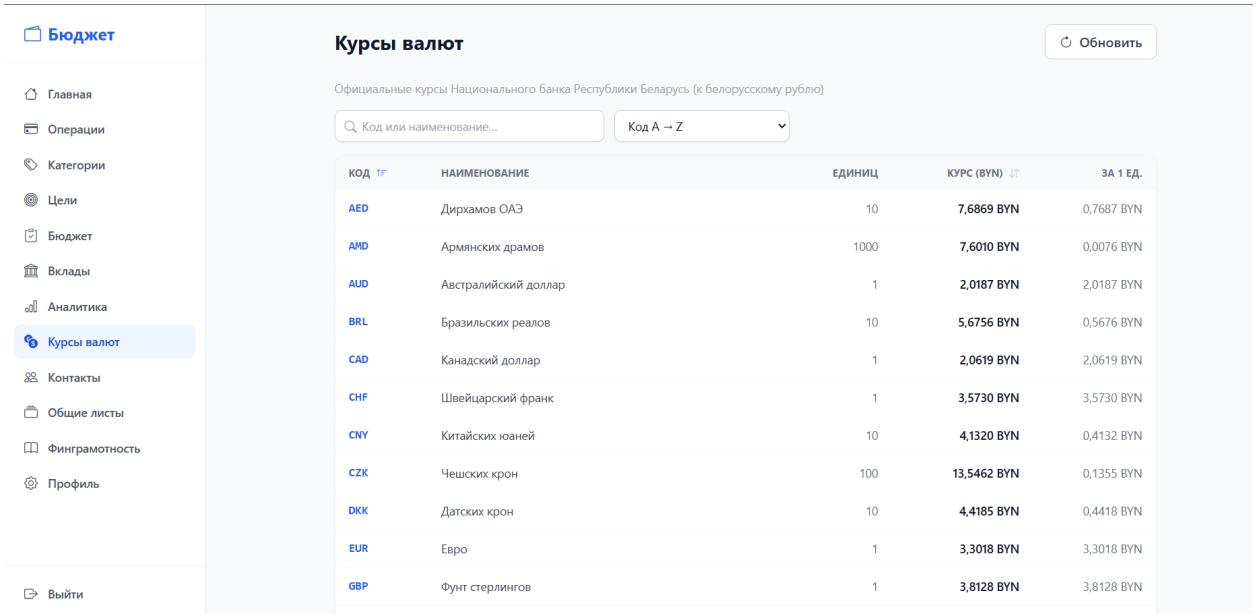


Рисунок 4.16 – Страница курсы валют

Для каждой валюты указывается код, наименование, количество единиц, курс в BYN и курс за одну единицу. Доступны поиск по коду или наименованию валюты и сортировка списка. Данные можно обновить вручную кнопкой «Обновить».

Раздел «Контакты» предназначен для управления списком контактов, используемых при работе с общими листами. Страница содержит три вкладки: «Мои контакты», «Заявки» и «Поиск». На вкладке «Поиск», представленной на рисунке 4.17, можно найти других пользователей системы по адресу электронной почты, номеру телефона или коду друга и отправить им запрос в контакты.

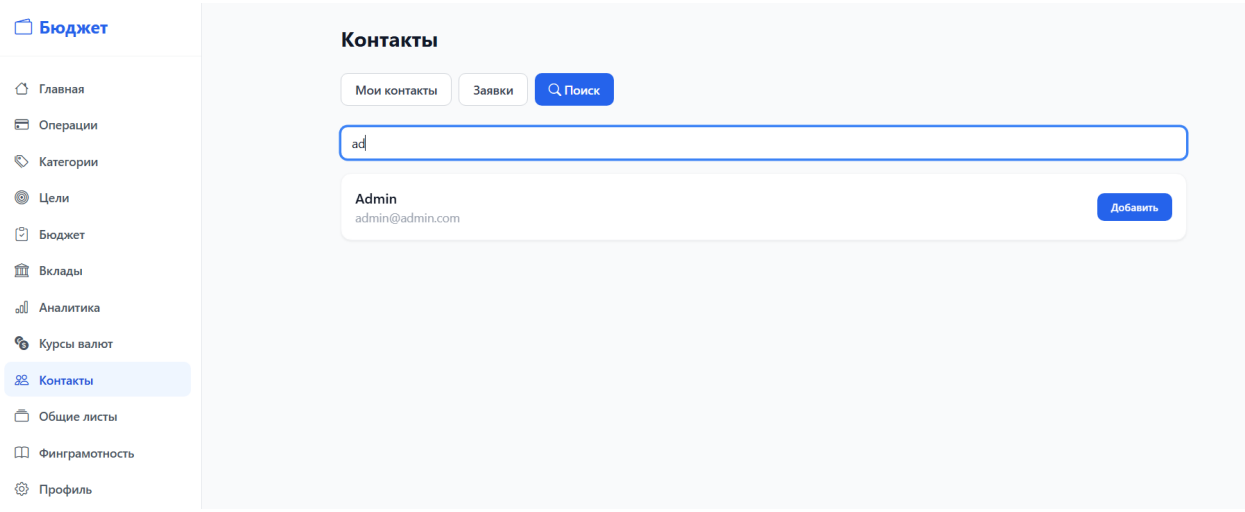


Рисунок 4.17 – Страница поиска

На вкладке «Заявки», представленной на рисунке 4.18, отображаются входящие запросы, каждый из которых можно принять или отклонить. О новом входящем запросе сигнализирует счётчик на иконке раздела в навигационной панели.

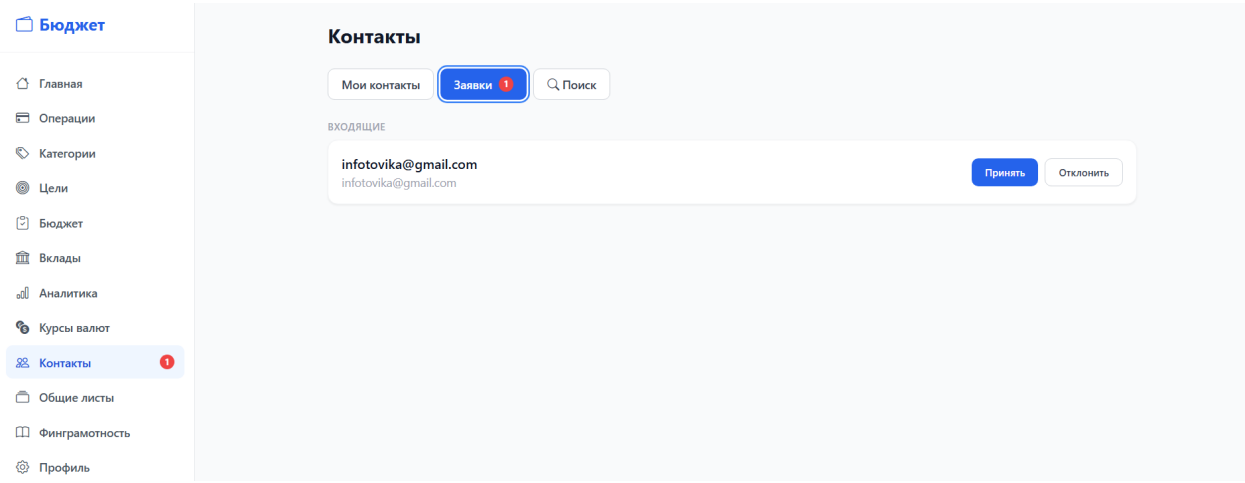


Рисунок 4.18 – Входящие заявки

Раздел «Общие листы» позволяет вести совместный финансовый учёт с другими пользователями. Для создания нового листа необходимо нажать кнопку «Создать» и указать название и необязательное описание, как показано на рисунке 4.19.

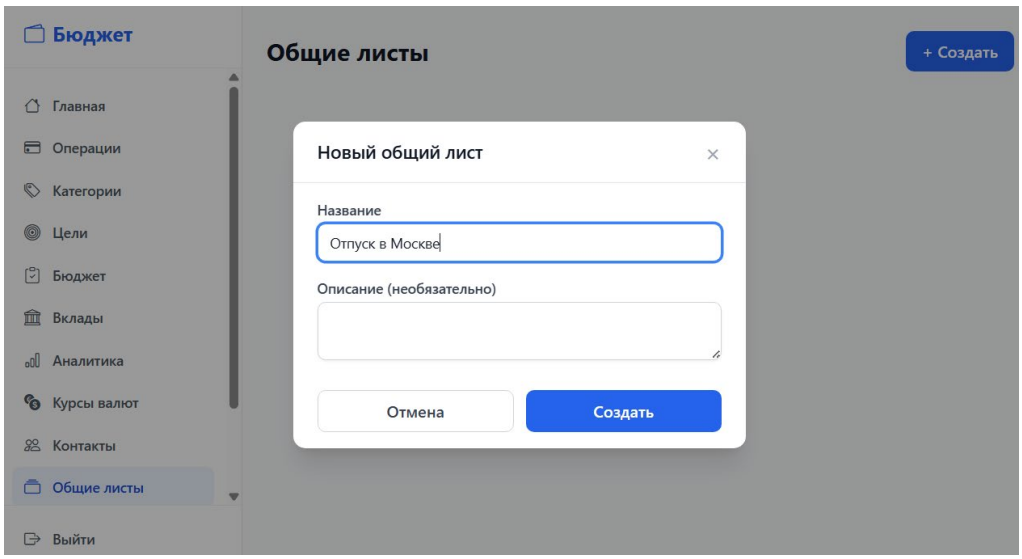


Рисунок 4.19 – Страница создания общего листа

После создания открывается страница листа, представленная на рисунке 4.20, содержащая четыре вкладки: «Операции», «Цели», «Аналитика» и «Участники». На вкладке «Операции» отображается список всех транзакций в листе с указанием даты, автора, типа, описания и суммы.

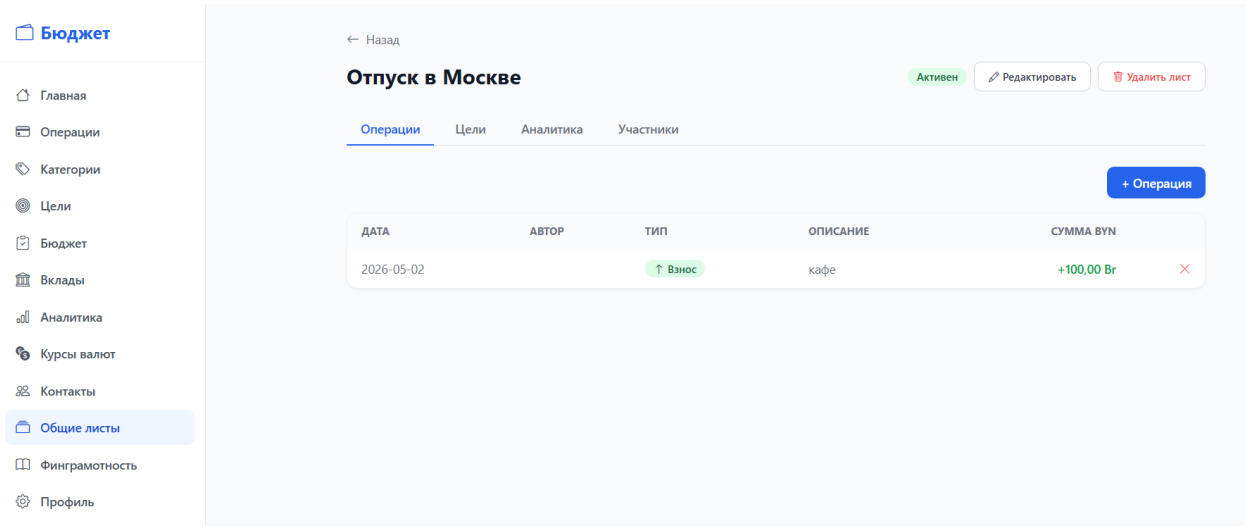


Рисунок 4.20 – Страница общего листа

На вкладке «Аналитика», представленной на рисунке 4.21, выводится сводка по листу: всего внесено, всего потрачено, баланс, доля расходов и доля взносов на каждого участника, а также таблица индивидуального вклада каждого участника.

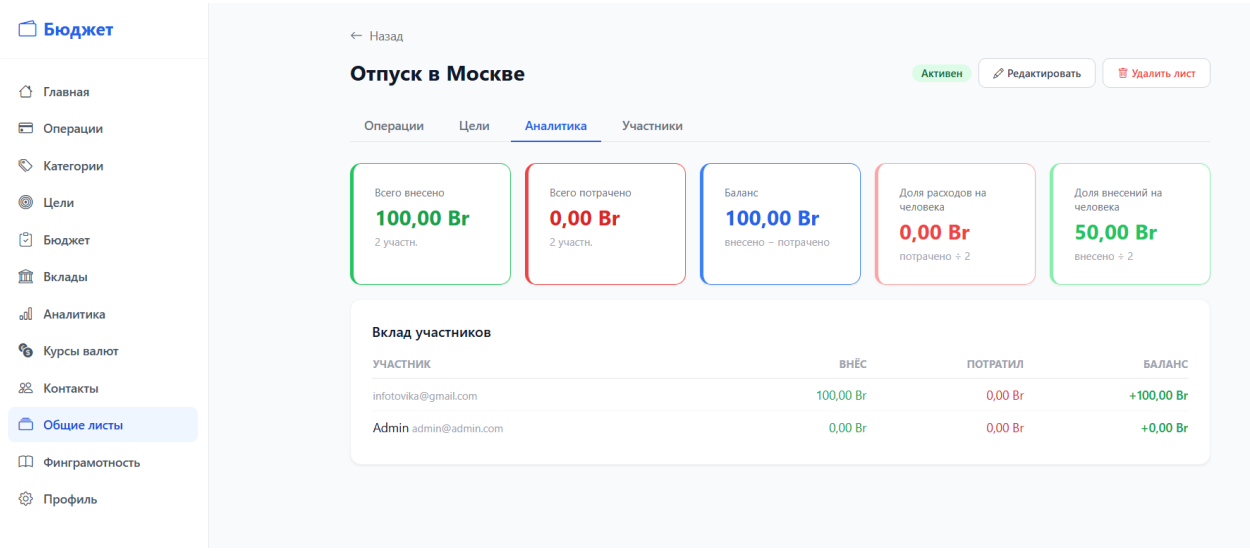


Рисунок 4.21 – Страница аналитика по общему листу

Раздел «Финграмотность» содержит обучающие материалы по управлению личными финансами, представленные на рисунке 4.22. Материалы отображаются в виде карточек с указанием категории, заголовка и краткого текста. Доступны поиск по заголовку и фильтрация по категории. Понравившиеся материалы можно добавить в избранное, нажав на иконку звезды, и впоследствии просматривать их на вкладке «Избранное».

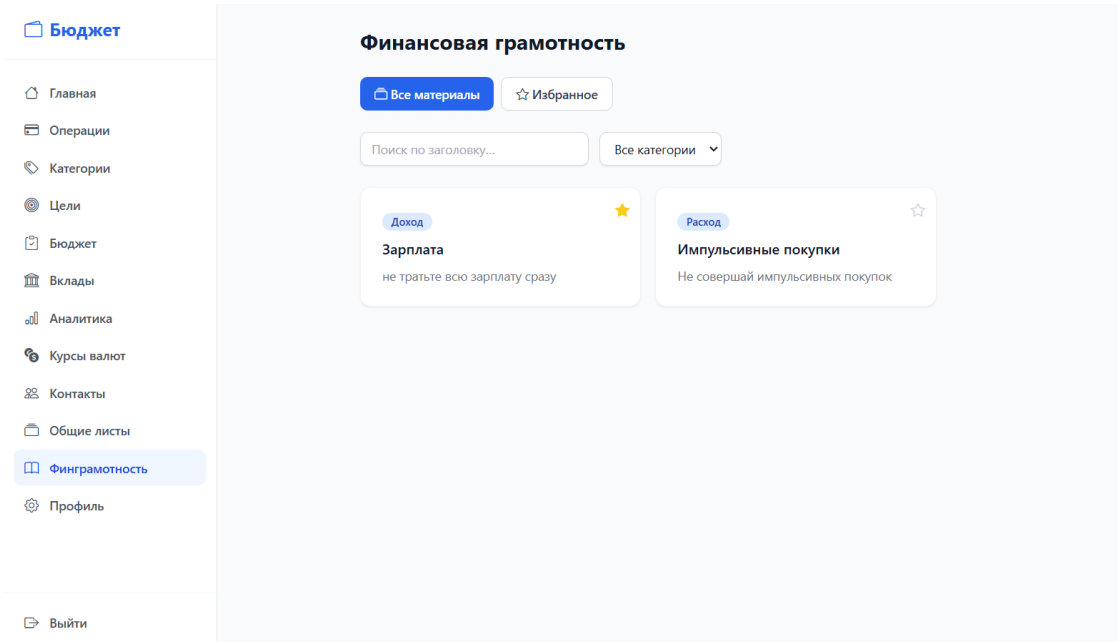


Рисунок 4.22 – Страница финграмотность у пользователя

Администратор системы имеет возможность создавать новые материалы и управлять категориями непосредственно через интерфейс раздела, как показано на рисунке 4.23.

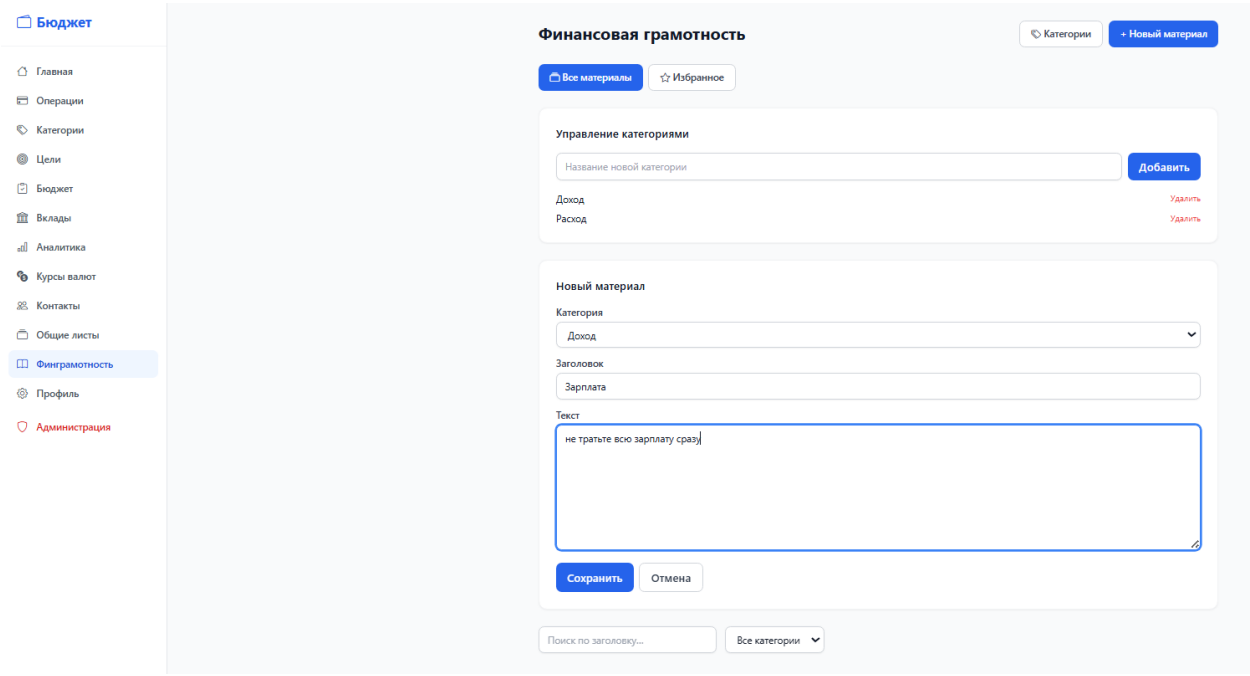


Рисунок 4.23 – Страница финграмотность у администратора

Раздел «Профиль» позволяет управлять личными данными аккаунта. В блоке личных данных, представленном на рисунке 4.24, пользователь может указать имя, фамилию, адрес электронной почты, номер телефона, дату рождения и валюту по умолчанию. Также на этой странице отображается личный код друга, который можно использовать для поиска в разделе «Контакты».

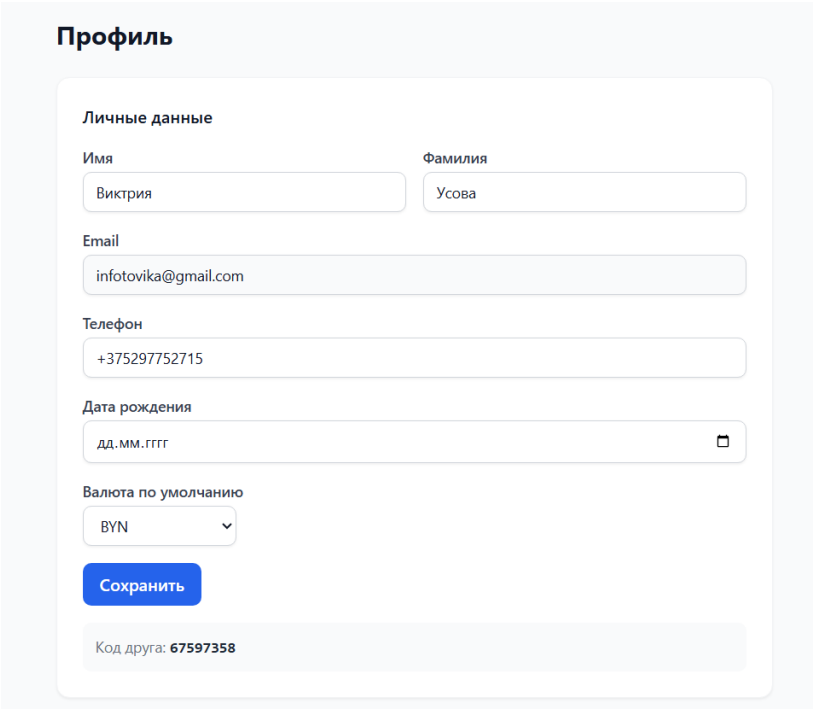


Рисунок 4.24 – Страница профиль

В блоке уведомлений на почту, представленном на рисунке 4.25, пользователь может гибко настроить, о каких событиях он хочет получать письма: достижение цели, превышение бюджета, операция в общем листе, запрос в контакты, закрытие вклада. В блоке настроек приватности задаётся видимость профиля при поиске другими пользователями: по email, по номеру телефона, по коду друга, а также отображение полного имени.

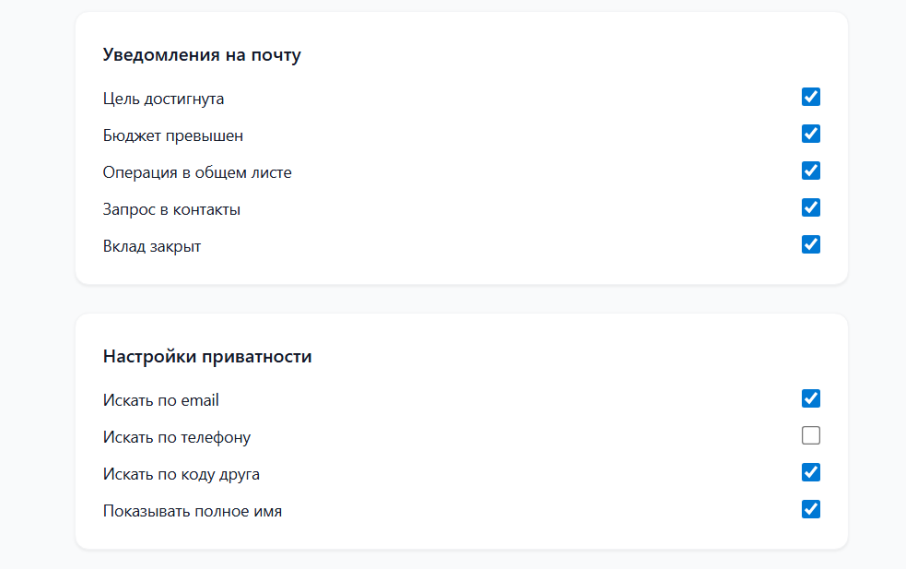


Рисунок 4.25 – Выбор уведомлений на почту

Для смены пароля предусмотрен отдельный блок, представленный на рисунке 4.27, в котором необходимо ввести текущий пароль, новый пароль, соответствующий требованиям надёжности, и его подтверждение.

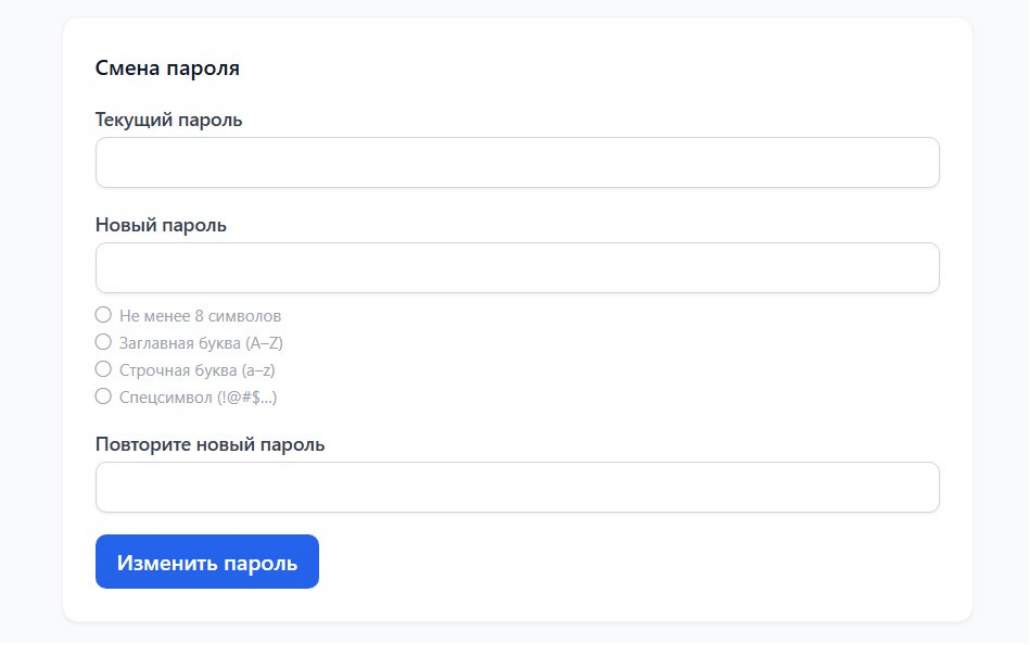


Рисунок 4.26 – Смена пароля

Таким образом, в ходе тестирования и разработки программной документации к платформе управления личными финансами с модулем категоризации расходов и планирования бюджета был пройден полный цикл верификации разработанного программного средства.

Функциональное тестирование, проводившееся на уровне серверной части приложения с использованием инструментов JUnit 5, Spring Boot Test, MockMvc и Mockito, подтвердило корректность обработки HTTP-запросов, работоспособность механизмов аутентификации и авторизации, а также соответствие бизнес-логики установленным требованиям. В ходе тестирования особое внимание уделялось негативным сценариям: обращению к защищённым эндпоинтам без JWT-токена, попыткам доступа к чужим сущностям и обработке запросов с некорректными входными данными.

Модульное тестирование охватило тест-кейсы для сервисов CategoryService, TransactionService и GoalService, покрывающих все ключевые операции: создание, обновление и мягкое удаление сущностей, проверку прав владельца, а также граничные условия логики пополнения финансовых целей, включая автоматический перевод статуса в COMPLETED и публикацию доменного события GoalReachedEvent. Дополнительно выполнено модульное тестирование фасадного слоя CategoryFacade, верифицирующих логику частичного обновления и делегирования вызовов. Интеграционное тестирование контроллера категорий (CategoryController) включало 4 тест-кейса, проверяющих корректность HTTP-статусов и защищённость эндпоинтов. Все тест-кейсы завершились успешно.

Кроме того, было описано руководство по развёртыванию программного средства, включающее последовательную настройку всех компонентов системы: базы данных PostgreSQL, а также серверной и клиентской частей приложения с указанием конфигурационных параметров. Руководство пользователя охватило полный пользовательский путь – от регистрации и входа в систему, в том числе через аккаунт Google, до работы со всеми модулями платформы: учётом операций и категоризацией расходов, планированием бюджета, управлением целями накопления и вкладами, просмотром аналитики и курсов валют, ведением совместных финансовых листов с контактами, изучением материалов по финансовой грамотности, а также настройкой профиля, уведомлений и приватности.

ЗАКЛЮЧЕНИЕ

В результате выполнения данного курсового проекта была разработана платформа для управления личными финансами с модулем категоризации расходов и планирования бюджета. Программное средство предназначено для автоматизации процессов учёта доходов и расходов, их классификации по категориям, планирования бюджета, управления накоплениями и финансовыми целями, а также для предоставления пользователям наглядной аналитики и инструментов совместного финансового планирования. Система обеспечивает удобное взаимодействие со всеми модулями платформы через веб-интерфейс.

Разработка инструментов управления личными финансами является актуальной задачей в условиях цифровой экономики, когда увеличение числа банковских продуктов и электронных платёжных систем усложняет структуру доходов и расходов граждан. Проведённый анализ аналогов показал, что существующие решения либо ограничены в гибкости настройки категорий и бюджетных параметров, либо требуют высокой стоимости подписки и не обеспечивают баланса между простотой интерфейса и глубокой аналитикой. Это подтвердило актуальность разработки собственной платформы.

Проект сопровождался разработкой и анализом моделей данных, бизнес-процессов в нотациях BPMN и IDEF0, архитектуры клиент-серверного взаимодействия, UML-диаграмм и алгоритмов проверки корректности входных данных. Реализованы современные механизмы безопасности, включая JWT-аутентификацию, OAuth 2.0 через Google, хеширование паролей алгоритмом BCrypt и ролевую модель разграничения доступа, что обеспечивает надёжную защиту персональных финансовых данных пользователей.

Основная цель проекта была достигнута. Все поставленные задачи, включая анализ аналогов, проектирование, реализацию функциональных модулей, разработку пользовательского интерфейса и тестирование, успешно выполнены. По результатам проведённого тестирования все тест-кейсы завершились успешно, а функциональное тестирование подтвердило корректность обработки всех ключевых пользовательских сценариев.

Разработанная система обеспечивает комплексную автоматизацию процессов личного финансового учёта, объединяя в рамках единой платформы учёт операций, категоризацию расходов, бюджетное планирование, управление целями накопления, вкладами и совместными листами. Это минимизирует влияние человеческого фактора при ведении финансовой отчётности, способствует повышению финансовой грамотности пользователей и поддерживает принятие обоснованных решений в области управления личными денежными ресурсами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Личные финансы и управление бюджетом в эпоху цифровой экономики / А. В. Лукасевич. – Москва : Финансы и статистика, 2021. – 304 с.

[2] Финансовая грамотность и поведение потребителей на рынке финансовых услуг : монография / О. И. Лаврушин [и др.]. – Москва : КноРус, 2022. – 248 с.

[3] Spring Security Reference [Электронный ресурс] / VMware, Inc. – Режим доступа: https://docs.spring.io/spring-security/reference. – Дата доступа: 25.02.2026.

[4] Spring Boot Reference Documentation [Электронный ресурс] / Pivotal Software. – Режим доступа: https://docs.spring.io/spring-boot/docs/current/reference/html. – Дата доступа: 25.02.2026.

[5] React – A JavaScript library for building user interfaces [Электронный ресурс] / Meta Platforms, Inc. – Режим доступа: https://react.dev. – Дата доступа: 25.02.2026.

[6] PostgreSQL 15 Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. – Режим доступа: https://www.postgresql.org/docs/15. – Дата доступа: 25.02.2026.

[7] CoinKeeper – приложение для учёта личных финансов [Электронный ресурс]. – Режим доступа: https://coinkeeper.me. – Дата доступа: 25.02.2026.

[8] Zenmoney – сервис для учёта финансов и бюджетирования [Электронный ресурс]. – Режим доступа: https://zenmoney.ru. – Дата доступа: 25.02.2026.

[9] Money Manager Expense & Budget [Электронный ресурс]. – Режим доступа: https://www.realbyteapps.com. – Дата доступа: 25.02.2026.

[10] YNAB (You Need A Budget) – личное бюджетирование [Электронный ресурс]. – Режим доступа: https://www.youneedabudget.com. – Дата доступа: 25.02.2026.

[11] Краткое описание нотации BPMN [Электронный ресурс]. – Режим доступа: https://habr.com/ru/companies/auriga/articles/667084/. – Дата доступа: 24.03.2026.

[12] IDEF0. Знакомство с нотацией и пример использования [Электронный ресурс]. − Режим доступа: https://www.trinion.org/blog/idef0 znakomstvo-s-notaciey-i-primer-ispolzovaniya Дата доступа: 25.02.2026.

[13] Моисеев А.Н., Литовченко М.И. Основы языка UML. // Учебное пособие. – 2023.– С.86-90

[14] Функциональная архитектура цифровых продуктов [Электронный ресурс]. − Режим доступа: https://habr.com/ru/articles/818547/?ysclid=mm83f4 Дата доступа: 26.02.2026.

[15] Урок по диаграммам развертывания [Электронный ресурс]. − Режим доступа: https://www.lucidchart.com/pages/ru/uml-deployment Дата доступа: 05.03.2026.

[16] UML: обзор основных типов диаграмм, диаграмма компонентов [Электронный ресурс]. − Режим доступа: https://habr.com/ru/articles/756552/ Дата доступа: 05.03.2026.

[17] UI – что тако. [Электронный ресурс]. − Режим доступа: https://skyeng.ru/magazine/wiki/it-industriya/chto-takoe-ui/?ysclid=miggfjas Дата доступа: 09.03.2026.

[18] Сущности и связи: как и для чего системные аналитики создают ER диаграммы [Электронный ресурс]. − Режим доступа: https://practicum.yandex.ru/blog/chto-takoe-er-diagramma/ Дата доступа: 09.03.2026.

[19] Использование диаграммы классов UML при проектировании и документировании программного обеспечения [Электронный ресурс]. − Режим доступа: https://habr.com/ru/articles/572234/ Дата доступа: 10.03.2026.

[20] Диаграмма последовательности [Электронный ресурс]. − Режим доступа: https://flexberry.github.io/ru/fd_sequence-diagram.html Дата доступа: 12.03.2026.

[21] Диаграмма состояний [Электронный ресурс]. − Режим доступа: https://ensi-platform.gitlab.io/analyst-guides/tools/diagrams/uml/ Дата доступа: 13.03.2026.

[22] Моисеев А.Н., Литовченко М.И. Основы языка UML. // Учебное пособие. – 2023.– С.86-90.

[23] Блок-схема алгоритма: разбираемся в особенностях [Электронный ресурс]. − Режим доступа: https://gb.ru/blog/blok-shemaalgoritma/? Дата доступа: 10.04.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода алгоритмов, реализующих основную бизнес-логику

```
package server.server.business.service.auth;

import
org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import server.server.business.service.profile.ProfileService;
import server.server.common.dto.auth.key.CheckSignUpKeyRequest;
import server.server.common.entity.auth.Account;
import server.server.common.repository.auth.AccountRepository;
import server.server.common.repository.auth.RefreshTokenRepository;
import
server.server.common.service.auth.SignUpKeyAuthorizationService;
import server.server.common.service.entity.SignUpKeyService;
import server.server.common.service.util.MailService;
import server.server.common.util.jwt.JwtUtils;

@Service
public class AuthorizationService extends
SignUpKeyAuthorizationService<CheckSignUpKeyRequest> {

    private final ProfileService profileService;

    public AuthorizationService(AccountRepository repository,
                                PasswordEncoder passwordEncoder,
                                AuthenticationManager
authenticationManager,
                                JwtUtils jwtUtils,
                                RefreshTokenRepository
refreshTokenRepository,
                                MailService mailService,
                                SignUpKeyService signUpKeyService,
                                ProfileService profileService) {
        super(repository, passwordEncoder, authenticationManager,
jwtUtils,
                refreshTokenRepository, mailService, signUpKeyService);
        this.profileService = profileService;
    }

    @Override
    protected void createRelatedEntities(Account account) {
        profileService.createWithDefaults(account, "", "");
    }
}
package server.server.business.service.auth;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import server.server.business.service.profile.ProfileService;
import server.server.common.entity.auth.Account;
import server.server.common.repository.auth.AccountRepository;
```

```
import server.server.common.service.auth.BaseInitService;

@Service
public class InitService extends BaseInitService {

    private final ProfileService profileService;

    public InitService(AccountRepository repository,
                       PasswordEncoder passwordEncoder,
                       ProfileService profileService) {
        super(repository, passwordEncoder);
        this.profileService = profileService;
    }

    @Override
    protected void afterAdminCreated(Account admin) {
        profileService.createWithDefaults(admin, "Admin", "");
    }
}
package server.server.business.service.finance;

import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import server.server.business.entity.enums.CategoryType;
import server.server.business.entity.finance.Category;
import server.server.business.entity.profile.Profile;
import server.server.business.repository.finance.CategoryRepository;
import server.server.common.exception.BaseKeyException;
import server.server.common.exception.common.EntityNotFoundException;

import java.util.List;

@Service
@RequiredArgsConstructor
@Transactional
public class CategoryService {

    private final CategoryRepository categoryRepository;

    public Page<Category> findByProfile(Long profileId, CategoryType
type, Pageable pageable) {
        return type != null
                ?
categoryRepository.findByProfile_IdAndTypeAndDeletedFalse(profileId,
type, pageable)
                :
categoryRepository.findByProfile_IdAndDeletedFalse(profileId,
pageable);
    }

    public List<Category> findAllByProfile(Long profileId) {
        return
```

```
categoryRepository.findByProfile_IdAndDeletedFalse(profileId);
    }

    public Category findById(Long id) {
        Category category = categoryRepository.findById(id)
                .orElseThrow(() -> new EntityNotFoundException("id=" +
id, "CATEGORY_NOT_FOUND", Category.class));
        if (Boolean.TRUE.equals(category.getDeleted())) {
            throw new EntityNotFoundException("id=" + id,
"CATEGORY_NOT_FOUND", Category.class);
        }
        return category;
    }

    public Category create(Profile profile, String name, String color,
CategoryType type) {
        if
(categoryRepository.existsByProfile_IdAndNameIgnoreCaseAndTypeAndDelet
edFalse(profile.getId(), name, type)) {
            throw new DuplicateCategoryException(name, type);
        }
        return categoryRepository.save(Category.builder()
                .profile(profile)
                .name(name)
                .color(color)
                .type(type)
                .build());
    }
    public Category update(Category category, String name, String
color) {
        category.setName(name);
        category.setColor(color);
        return categoryRepository.save(category);
    }

    public void delete(Category category) {
        category.setDeleted(true);
        categoryRepository.save(category);
    }
    public void assertOwnership(Category category, Long profileId) {
        if (!category.getProfile().getId().equals(profileId)) {
            throw new CategoryAccessDeniedException();
        }
    }
    // Inner exception classes
    public static class DuplicateCategoryException extends
BaseKeyException {
        public DuplicateCategoryException(String name, CategoryType
type) {
            super("Category '" + name + "' of type " + type + "
already exists", "DUPLICATE_CATEGORY");        }   }
```

```
 public static class CategoryAccessDeniedException extends
BaseKeyException {
        public CategoryAccessDeniedException() {
            super("You do not own this category",
"CATEGORY_ACCESS_DENIED");
        }
    }
}package server.server.business.service.finance;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import server.server.business.entity.enums.DepositStatus;
import server.server.business.entity.finance.Deposit;
import server.server.business.entity.finance.Transaction;
import server.server.business.event.DepositMaturedEvent;
import server.server.business.factory.TransactionFactory;
import server.server.business.repository.finance.DepositRepository;
import
server.server.business.repository.finance.TransactionRepository;
import java.time.LocalDate;
import java.util.List;
@Slf4j
@Service
@RequiredArgsConstructor
public class DepositMaturityService {

    private final DepositRepository      depositRepository;
    private final TransactionRepository  transactionRepository;
    private final TransactionFactory     transactionFactory;
    private final ApplicationEventPublisher eventPublisher;
    @Transactional
    public int processMatureDeposits() {
        List<Deposit> matured = depositRepository

.findByStatusAndEndDateLessThanEqualAndDeletedFalse(DepositStatus.ACTI
VE, LocalDate.now());

        int count = 0;
        for (Deposit deposit : matured) {
            try {
                Transaction income =
transactionFactory.createFromMaturedDeposit(deposit);
                transactionRepository.save(income);

                deposit.setStatus(DepositStatus.COMPLETED);

 deposit.setIncomeTransaction(income);
                depositRepository.save(deposit);

                eventPublisher.publishEvent(new
DepositMaturedEvent(this, deposit));
                log.info("Deposit id={} matured → income transaction
```

```
id={} created", deposit.getId(), income.getId());
                count++;
            } catch (Exception ex) {
                log.error("Failed to process matured deposit id={}:
{}", deposit.getId(), ex.getMessage(), ex);
            }
        }
        return count;
    }
}
package server.server.business.service.finance;

import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import server.server.business.entity.enums.DepositStatus;
import server.server.business.entity.enums.DepositType;
import server.server.business.entity.finance.Deposit;
import
server.server.business.repository.finance.DepositRepository;
import server.server.common.exception.BaseKeyException;
import
server.server.common.exception.common.EntityNotFoundException;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;
@Service
@RequiredArgsConstructor
@Transactional
public class DepositService {
    private final DepositRepository depositRepository;

    public Page<Deposit> findByProfile(Long profileId,
DepositStatus status, DepositType type, Pageable pageable) {
        if (status != null) return
depositRepository.findByProfile_IdAndStatusAndDeletedFalse(profi
leId, status, pageable);
        if (type != null)   return
depositRepository.findByProfile_IdAndTypeAndDeletedFalse(profile
Id, type, pageable);
        return
depositRepository.findByProfile_IdAndDeletedFalse(profileId,
pageable);
    }
public Deposit findById(Long id) {
```

```
Deposit deposit = depositRepository.findById(id)
                .orElseThrow(() -> new EntityNotFoundException("id=" +
id, "DEPOSIT_NOT_FOUND", Deposit.class));
        if (Boolean.TRUE.equals(deposit.getDeleted())) {
            throw new EntityNotFoundException("id=" + id,
"DEPOSIT_NOT_FOUND", Deposit.class);
        }
        return deposit;
    }
    public Deposit save(Deposit deposit) {
        return depositRepository.save(deposit);
    }
    public void delete(Deposit deposit) {
        deposit.setDeleted(true);
        depositRepository.save(deposit);
    }
    public Deposit addFunds(Deposit deposit, BigDecimal amount) {
        if (deposit.getType() != DepositType.PIGGY_BANK) {
            throw new DepositTypeException("Only PIGGY_BANK deposits
support manual fund addition");
        }

deposit.setCurrentAmount(deposit.getCurrentAmount().add(amount));
        return depositRepository.save(deposit);
    }

    /** Returns all ACTIVE deposits with end_date <= today. Used by
Quartz job. */
    public List<Deposit> findMaturedDeposits() {
        return
depositRepository.findByStatusAndEndDateLessThanEqualAndDeletedFalse(D
epositStatus.ACTIVE, LocalDate.now());
    }

    public void assertOwnership(Deposit deposit, Long profileId) {
        if (!deposit.getProfile().getId().equals(profileId)) {
            throw new DepositAccessDeniedException();
        }
    }

    public static class DepositAccessDeniedException extends
BaseKeyException {
        public DepositAccessDeniedException() {
            super("You do not own this deposit",
"DEPOSIT_ACCESS_DENIED");
        }
    }
    public static class DepositTypeException extends BaseKeyException
{
        public DepositTypeException(String msg) {
            super(msg, "DEPOSIT_TYPE_ERROR");
        }
    }
}package server.server.business.service.finance;
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Листинг скрипта генерации базы данных

```
create table public.flyway_schema_history
(
    installed_rank integer                 not null
        constraint flyway_schema_history_pk
            primary key,
    version        varchar(50),
    description    varchar(200)            not null,
    type           varchar(20)             not null,
    script         varchar(1000)           not null,
    checksum       integer,
    installed_by   varchar(100)            not null,
    installed_on   timestamp default now() not null,
    execution_time integer                 not null,
    success        boolean                 not null
);

alter table public.flyway_schema_history
    owner to postgres;

create index flyway_schema_history_s_idx
    on public.flyway_schema_history (success);

create table public.accounts
(
    id         bigserial
        primary key,
    username   varchar(255)            not null
        unique,
    password   varchar(255)            not null,
    role       varchar(10)             not null
        constraint accounts_role_check
            check ((role)::text = ANY ((ARRAY ['ADMIN'::character
varying, 'USER'::character varying])::text[])),
    active     boolean   default true  not null,
    created_at timestamp default now() not null,
    updated_at timestamp default now() not null
);

alter table public.accounts
    owner to postgres;

create table public.sign_up_keys
(
    id         bigserial
        primary key,
    username   varchar(255)            not null,
    reg_key    varchar(8)              not null,
    role       varchar(10)             not null
        constraint sign_up_keys_role_check
            check ((role)::text = ANY ((ARRAY ['ADMIN'::character
varying, 'USER'::character varying])::text[])),
    created_at timestamp default now() not null,
```

```
  updated_at timestamp default now() not null
);

alter table public.sign_up_keys
    owner to postgres;

create table public.profiles
(
    id               bigserial
        primary key,
    account_id       bigint                                          not
null
        unique
        references public.accounts,
    first_name       varchar(100)                                    not
null,
    last_name        varchar(100)                                    not
null,
    phone            varchar(20),
    birth_date       date,
    friend_code      varchar(12)                                     not
null
        unique,
    default_currency varchar(3) default 'BYN'::character varying not
null,
    created_at       timestamp  default now()                        not
null,
    updated_at       timestamp  default now()                        not
null
);

alter table public.profiles
    owner to postgres;

create table public.privacy_settings
(
    id                        bigserial
        primary key,
    profile_id                bigint                  not null
        unique
        references public.profiles,
    searchable_by_email       boolean   default true  not null,
    searchable_by_phone       boolean   default false not null,
    searchable_by_friend_code boolean   default true  not null,
    show_full_name            boolean   default true  not null,
    created_at                timestamp default now() not null,
    updated_at                timestamp default now() not null
);

alter table public.privacy_settings
    owner to postgres;

create table public.notification_settings
(
    id                    bigserial
```

```
  primary key,
    profile_id            bigint                  not null
        unique
        references public.profiles,
    on_goal_reached       boolean   default true  not null,
    on_budget_exceeded    boolean   default true  not null,
    on_shared_transaction boolean   default true  not null,
    on_contact_request    boolean   default true  not null,
    on_deposit_matured    boolean   default true  not null,
    created_at            timestamp default now() not null,
    updated_at            timestamp default now() not null
);

alter table public.notification_settings
    owner to postgres;

create table public.contacts
(
    id           bigserial
        primary key,
    requester_id bigint                                                 not
null
        references public.profiles,
    addressee_id bigint                                                 not
null
        references public.profiles,
    status       varchar(20) default 'PENDING'::character varying not
null
        constraint contacts_status_check
            check ((status)::text = ANY
                   ((ARRAY ['PENDING'::character varying,
'ACCEPTED'::character varying, 'DECLINED'::character
varying])::text[])),
    created_at   timestamp   default now()                             not
null,
    updated_at   timestamp   default now()                             not
null,
    constraint uq_contacts_requester_addressee
        unique (requester_id, addressee_id),
    constraint chk_contacts_no_self
        check (requester_id <> addressee_id)
);

alter table public.contacts
    owner to postgres;

create table public.categories
(
    id         bigserial
        primary key,
    profile_id bigint                  not null
        references public.profiles,
    name       varchar(100)            not null,
    color      varchar(7),
    type       varchar(10)             not null
```

```
 constraint categories_type_check
            check ((type)::text = ANY ((ARRAY ['INCOME'::character
varying, 'EXPENSE'::character varying])::text[])),
    deleted    boolean   default false not null,
    created_at timestamp default now() not null,
    updated_at timestamp default now() not null
);

alter table public.categories
    owner to postgres;

create table public.transactions
(
    id                   bigserial
        primary key,
    profile_id           bigint                  not null
        references public.profiles,
    category_id          bigint
        references public.categories,
    amount               numeric(19, 4)          not null,
    currency             varchar(3)              not null,
    exchange_rate_to_byn numeric(19, 6)          not null,
    amount_byn           numeric(19, 4)          not null,
    type                 varchar(10)             not null
        constraint transactions_type_check
            check ((type)::text = ANY ((ARRAY ['INCOME'::character
varying, 'EXPENSE'::character varying])::text[])),
    date                 date                    not null,
    description          text,
    deleted              boolean   default false not null,
    created_at           timestamp default now() not null,
    updated_at           timestamp default now() not null
);

alter table public.transactions
    owner to postgres;

create index idx_transactions_profile_date
    on public.transactions (profile_id, date);

create index idx_transactions_profile_type
    on public.transactions (profile_id, type);

create table public.goals
(
    id             bigserial
        primary key,
    profile_id     bigint
not null
        references public.profiles,
    name           varchar(200)
not null,
    target_amount  numeric(19, 4)
not null,
```

```
currency       varchar(3)                                              not
null,
    current_amount numeric(19, 4) default 0
not null,
    deadline       date,
    status         varchar(20)    default 'ACTIVE'::character varying
not null
        constraint goals_status_check
            check ((status)::text = ANY
                   ((ARRAY ['ACTIVE'::character varying,
'COMPLETED'::character varying, 'CANCELLED'::character
varying])::text[])),
    deleted        boolean        default false
not null,
    created_at     timestamp      default now()
not null,
    updated_at     timestamp      default now()
not null
);

alter table public.goals
    owner to postgres;

create table public.planned_expenses
(
    id           bigserial
        primary key,
    profile_id   bigint                  not null
        references public.profiles,
    category_id  bigint
        references public.categories,
    name         varchar(200)            not null,
    amount       numeric(19, 4)          not null,
    currency     varchar(3)              not null,
    type         varchar(20)             not null
        constraint planned_expenses_type_check
            check ((type)::text = ANY
                   ((ARRAY ['BUDGET_LIMIT'::character varying,
'PLANNED_PURCHASE'::character varying])::text[])),
    period_type  varchar(15)
        constraint planned_expenses_period_type_check
            check ((period_type)::text = ANY
                   ((ARRAY ['MONTHLY'::character varying,
'QUARTERLY'::character varying, 'YEARLY'::character
varying])::text[])),
    planned_date date,
    is_fulfilled boolean   default false not null,
    deleted      boolean   default false not null,
    created_at   timestamp default now() not null,
    updated_at   timestamp default now() not null
);

alter table public.planned_expenses
    owner to postgres;
```

```
create table public.deposits
(
    id                    bigserial
        primary key,
    profile_id            bigint
not null
        references public.profiles,
    name                  varchar(200)
not null,
    type                  varchar(15)
not null
        constraint deposits_type_check
            check ((type)::text = ANY
                   ((ARRAY ['DEPOSIT'::character varying,
'PIGGY_BANK'::character varying])::text[])),
    initial_amount        numeric(19, 4)
not null,
    currency              varchar(3)
not null,
    interest_rate         numeric(5, 2),
    start_date            date
not null,
    end_date              date,
    current_amount        numeric(19, 4)
not null,
    status                varchar(15) default 'ACTIVE'::character
varying not null
        constraint deposits_status_check
            check ((status)::text = ANY
                   ((ARRAY ['ACTIVE'::character varying,
'COMPLETED'::character varying])::text[])),
    income_transaction_id bigint
        references public.transactions,
    deleted               boolean     default false
not null,
    created_at            timestamp   default now()
not null,
    updated_at            timestamp   default now()
not null
);

alter table public.deposits
    owner to postgres;

create table public.shared_sheets
(
    id           bigserial
        primary key,
    creator_id   bigint                                          not
null
        references public.profiles,
    name         varchar(200)                                    not
null,
    description  text,
```

```
status       varchar(15) default 'ACTIVE'::character varying not null
        constraint shared_sheets_status_check
            check ((status)::text = ANY
                   ((ARRAY ['ACTIVE'::character varying,
'COMPLETED'::character varying])::text[])),
    completed_at timestamp,
    deleted      boolean     default false                      not
null,
    created_at   timestamp   default now()                      not
null,
    updated_at   timestamp   default now()                      not
null
);
alter table public.shared_sheets
    owner to postgres;

create table public.shared_sheet_members
(
    id         bigserial
        primary key,
    sheet_id   bigint                  not null
        references public.shared_sheets,
    profile_id bigint                  not null
        references public.profiles,
    joined_at  timestamp               not null,
    created_at timestamp default now() not null,
    updated_at timestamp default now() not null,
    constraint uq_sheet_member
        unique (sheet_id, profile_id)
);

alter table public.shared_sheet_members
    owner to postgres;

create table public.shared_transactions
(
    id                   bigserial
        primary key,
    sheet_id             bigint                  not null
        references public.shared_sheets,
    profile_id           bigint                  not null
        references public.profiles,
    amount               numeric(19, 4)          not null,
    currency             varchar(3)              not null,
    exchange_rate_to_byn numeric(19, 6)          not null,
    amount_byn           numeric(19, 4)          not null,
    type                 varchar(15)             not null
        constraint shared_transactions_type_check
            check ((type)::text = ANY
                   ((ARRAY ['EXPENSE'::character varying,
'CONTRIBUTION'::character varying])::text[])),
    description          text,
    date                 date                    not null,
    deleted              boolean   default false not null,
    created_at           timestamp default now() not null,
```

```
updated_at           timestamp default now() not null
);

alter table public.shared_transactions
    owner to postgres;

create index idx_shared_transactions_sheet
    on public.shared_transactions (sheet_id);

create table public.shared_goals
(
    id             bigserial
        primary key,
    sheet_id       bigint
not null
        references public.shared_sheets,
    name           varchar(200)
not null,
    target_amount  numeric(19, 4)
not null,
    currency       varchar(3)
not null,
    current_amount numeric(19, 4) default 0
not null,
    deadline       date,
    status         varchar(20)    default 'ACTIVE'::character varying
not null
        constraint shared_goals_status_check
            check ((status)::text = ANY
                   ((ARRAY ['ACTIVE'::character varying,
'COMPLETED'::character varying, 'CANCELLED'::character
varying])::text[])),
    deleted        boolean        default false
not null,
    created_at     timestamp      default now()
not null,
    updated_at     timestamp      default now()
not null
);

alter table public.shared_goals
    owner to postgres;

create table public.resource_categories
(
    id         bigserial
        primary key,
    name       varchar(100)            not null
        unique,
    deleted    boolean   default false not null,
    created_at timestamp default now() not null,
    updated_at timestamp default now() not null
);
```

```
alter table public.resource_categories
    owner to postgres;
create table public.financial_resources
(
    id                  bigserial
        primary key,
    author_id           bigint                  not null
        references public.accounts,
    resource_category_id bigint                 not null
        references public.resource_categories,
    title               varchar(300)            not null,
    content             text                    not null,
    deleted             boolean   default false not null,
    created_at          timestamp default now() not null,
    updated_at          timestamp default now() not null
);
alter table public.financial_resources
    owner to postgres;

create table public.resource_attachments
(
    id          bigserial
        primary key,
    resource_id bigint                  not null
        references public.financial_resources,
    file_name   varchar(255)            not null,
    file_path   varchar(500)            not null,
    file_type   varchar(10)             not null
        constraint resource_attachments_file_type_check
            check ((file_type)::text = ANY
                   ((ARRAY ['IMAGE'::character varying,
'DOCUMENT'::character varying])::text[])),
    created_at  timestamp default now() not null,
    updated_at  timestamp default now() not null
);
alter table public.resource_attachments
    owner to postgres;

create table public.refresh_tokens
(
    id         bigserial
        primary key,
    account_id bigint                  not null
        unique
        references public.accounts
            on delete cascade,
    token      varchar(512)            not null
        unique,
    expires_at timestamp with time zone not null,
    revoked    boolean   default false  not null,
    created_at timestamp default now()  not null,
    updated_at timestamp default now()  not null
);
alter table public.refresh_tokens
    owner to postgres;
```

ПРИЛОЖЕНИЕ В
(не обязательное)
BPMN-модель процессов предметной области

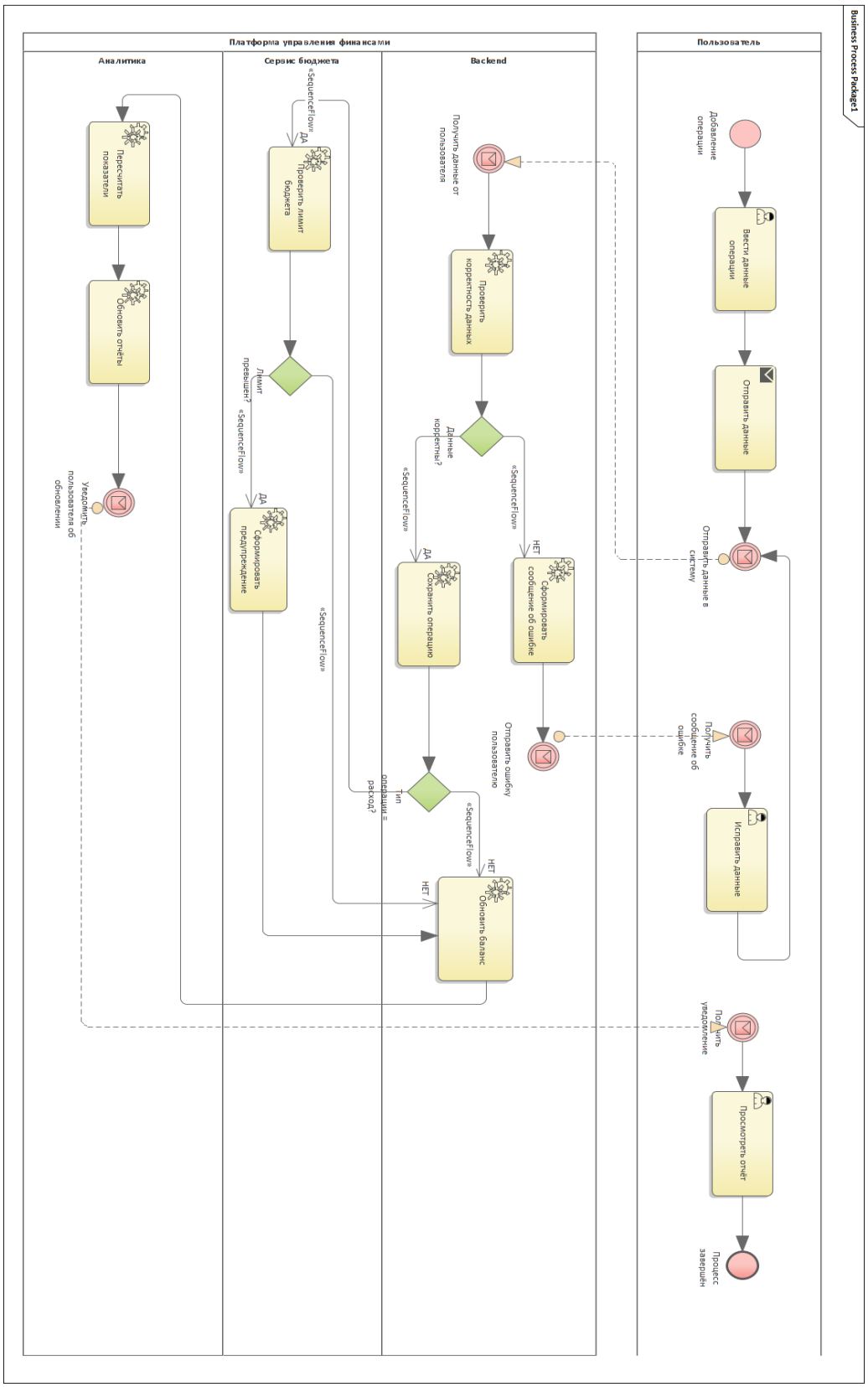


Рисунок В.1 – BPMN-модель

ПРИЛОЖЕНИЕ Г
(не обязательное)
Диаграмма классов

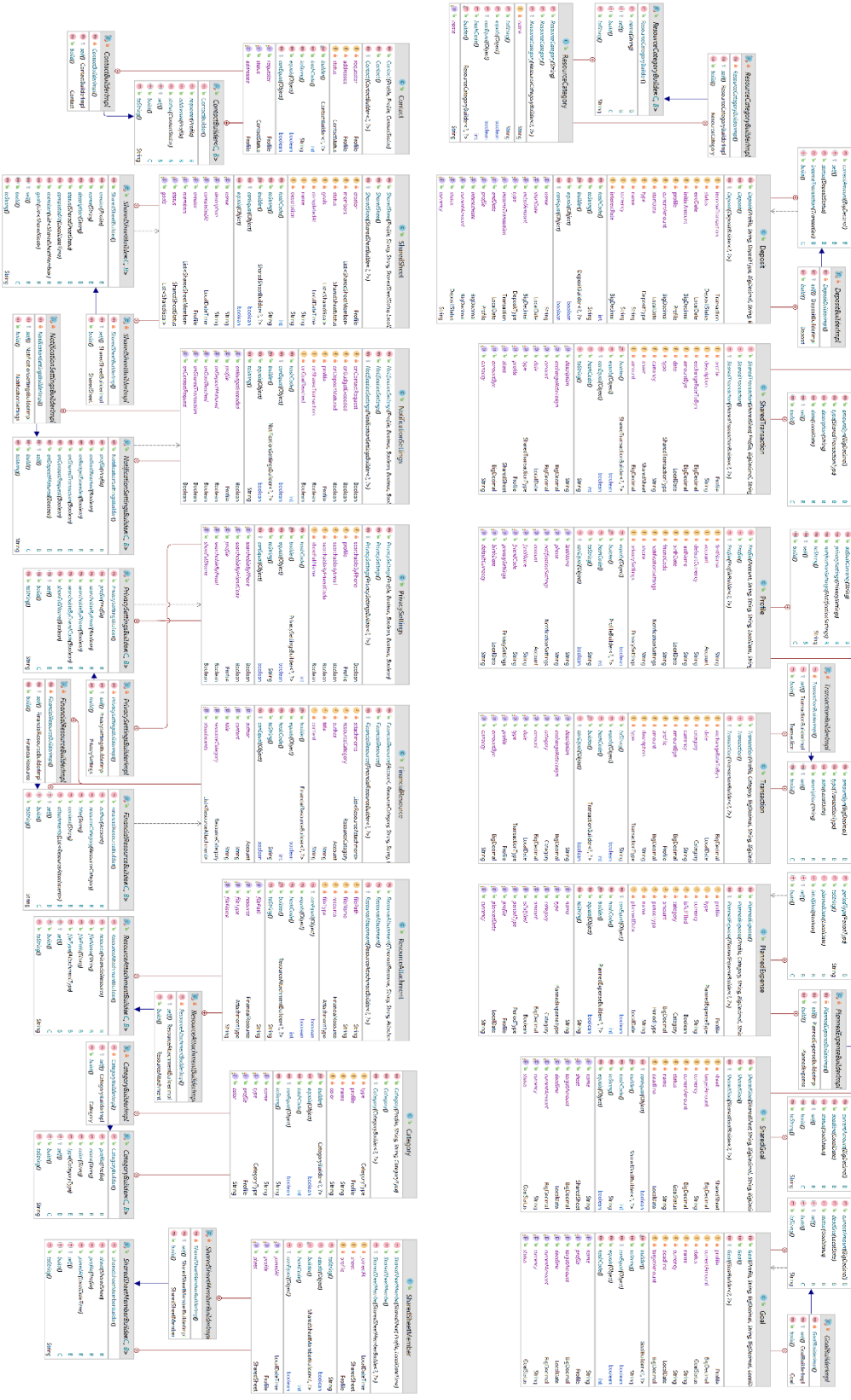


Рисунок Г.1 – Диаграмма классов

ВЕДОМОСТЬ ДОКУМЕНТОВ КУРСОВОГО ПРОЕКТА

Обозначение	Наименование	Дополнительные сведения
	Текстовые документы	
ГУИР КП 6-05 06 11-01 028 ПЗ	Пояснительная записка	103 с.
	Графические документы	
ГУИР.425196.001 ПЛ	BPMN-модель процессов	Формат А4
	предметной области	
ГУИР.425196.002 ПЛ	Архитектура программного	Формат А4
	средства	
ГУИР.425196.003 ПЛ	Схема модели базы данных	Формат А4
ГУИР.425196.004 ПЛ	Модели представления	Формат А4
	программного средства	
ГУИР.425196.005 ПД	Схема алгоритма, реализующая	Формат А4
	бизнес-логику программного средства	
ГУИР.425196.006 ПЛ	Скриншоты рабочих окон	Формат А4
	программного средства	

Перв. примен.

Справ. №

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.		Усова		
Пров.		Русецкий		
Т.контр.				
Н.контр.				
Утв.				

ГУИР КП 6-05 0611-01 028 ПЗ

Платформа для управления личными финансами с модулем категоризации расходов и планирования бюджета

Ведомость курсового проекта

Лит.	Лист	Листов
Т	103	103

Кафедра ЭИ
гр. 372302

Формат А4